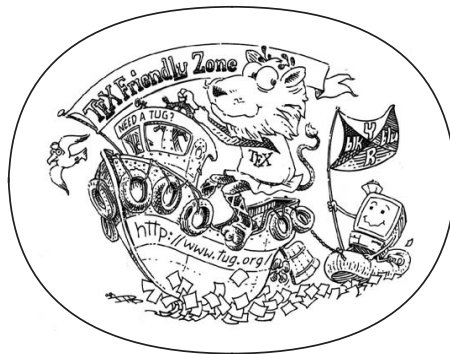


ANDRÉ MIEDE & IVO PLETIKOSIĆ
A CLASSIC THESIS STYLE

A CLASSIC THESIS STYLE

ANDRÉ MIEDE & IVO PLETIKOSIĆ



An Homage to The Elements of Typographic Style

February 2025 – classicthesis v4.8

André Miede & Ivo Pletikosić: *A Classic Thesis Style*, An Homage to
The Elements of Typographic Style, © February 2025

Ohana means family.
Family means nobody gets left behind, or forgotten.
— Lilo & Stitch

Dedicated to the loving memory of Rudolf Miede.
1939–2005

ABSTRACT

Short summary of the contents in English... a great guide by Kent Beck how to write good abstracts can be found here:

<https://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>

ZUSAMMENFASSUNG

Kurze Zusammenfassung des Inhaltes in deutscher Sprache...

PUBLICATIONS

This might come in handy for PhD theses: some ideas and figures have appeared previously in the following publications:

This is just an example.

- [1] Tobias Isenberg, André Miede, and Sheelagh Carpendale. “A Buffer Framework for Supporting Responsive Interaction in Information Visualization Interfaces.” In: *Proceedings of the Fourth International Conference on Creating, Connecting, and Collaborating through Computing (C⁵ 2006)*. IEEE, 2006, pp. 262–269. ISBN: 978-0-7695-2563-1.
- [2] Ulrich Lampe, Markus Kieselmann, André Miede, Sebastian Zöller, and Ralf Steinmetz. “A Tale of Millis and Nanos: On the Accuracy of Time Measurements in Virtual Machines.” In: *Proceedings of the Second European Conference on Service-Oriented and Cloud Computing (ESOCC 2013)*. Springer, 2013, pp. 172–179. ISBN: 978-3-642-40650-8.
- [3] Ulrich Lampe, Qiong Wu, Ronny Hans, André Miede, and Ralf Steinmetz. “To Frag Or To Be Fraggd – An Empirical Assessment of Latency in Cloud Gaming.” In: *Proceedings of the Third International Conference on Cloud Computing and Services Science (CLOSER 2013)*. 2013, pp. 5–12. ISBN: 978-898-8565-52-5.
- [4] André Miede. “Theses and other Beautiful Documents with classicthesis.” In: *TUGboat – The Communications of the T_EX Users Group* 31.1 (2010), pp. 18–20. ISSN: 0896-3207.
- [5] André Miede, Gökhan Şimşek, Stefan Schulte, Daniel F. Abawi, Julian Eckert, and Ralf Steinmetz. “Revealing Business Relationships – Eavesdropping Cross-organizational Collaboration in the Internet of Services.” In: *Proceedings of the Tenth International Conference Wirtschaftsinformatik (WI 2011)*. Vol. 2. 2011, pp. 1083–1092. ISBN: 978-1-4467-9236-0.
- [6] Hsin-Yi Tsai, Melanie Siebenhaar, André Miede, Yu-Lun Huang, and Ralf Steinmetz. “Threat as a Service? Virtualization’s Impact on Cloud Security.” In: *IEEE IT Professional* 14.1 (2012), pp. 32–37. ISSN: 1520-9202.

Attention: This requires a separate run of bibtex for your refsection, e.g., `ClassicThesis1-b1x` for this file. You might also use biber as the backend for biblatex. See also <http://tex.stackexchange.com/questions/128196/problem-with-refsection>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth [73]

ACKNOWLEDGMENTS

Put your acknowledgments here.

Many thanks to everybody who already sent me a postcard!

Regarding the typography and other help, many thanks go to Marco Kuhlmann, Philipp Lehman, Lothar Schlesier, Jim Young, Lorenzo Pantieri and Enrico Gregorio, Jörg Sommer, Joachim Köstler, Daniel Gottschlag, Denis Aydin, Paride Legovini, Steffen Prochnow, Nicolas Repp, Hinrich Harms, Roland Winkler, Jörg Weber, Henri Menke, Claus Lahiri, Clemens Niederberger, Stefano Bragaglia, Jörn Hees, Scott Lowe, Dave Howcroft, José M. Alcaide, David Carlisle, Ulrike Fischer, Hugues de Lassus, Csaba Hajdu, Dave Howcroft, Anonymous, Konrad Höffner, and the whole L^AT_EX-community for support, ideas and some great software.

Regarding L_YX: The L_YX port was initially done by *Nicholas Mariette* in March 2009 and continued by *Ivo Pletikosić* in 2011. Thank you very much for your work and for the contributions to the original style.

CONTENTS

I SOME KIND OF MANUAL

1	INTRODUCTION	3
1.1	Modeling the Language	3
1.1.1	A dialog between Shannon and Chomsky	3
1.1.2	The Transformer, the best machine language learner?	6
1.1.3	A Transformer is never enough large	14
2	EFFICIENT LANGUAGE MODELS	17
2.1	Model compression	18
2.1.1	Llama small models or how to account inference-optimality	19
2.1.2	Layer Looping: Increasing expressivity without increasing model parameters	20
2.1.3	Unstructured Pruning	20
2.1.4	Sturcured Pruning	22
2.1.5	The best training objective for compression	24
2.2	Activation Sparsity	28
2.2.1	Sparse Mixture of Experts (MoE)	29
2.2.2	Sparse Memory Layers	35
2.2.3	Embedding Memory Layers	39
2.2.4	Post-training Activation Sparsity	41
2.2.5	Matryoshka Learning	42
2.2.6	Speculative Decoding and Diffusion LLMs: Trading Compute for Memory	43

II COMPRESSION OF TRANSFORMER MODELS

3	LILLAMA	47
3.1	Introduction	47
3.2	Motivation: Activations are more low-rank than weights	49
3.3	Background: Low-Rank Approximation of Transformer Models	51
3.3.1	Low-Rank Approximation from Weights	51
3.3.2	Low-Rank Approximation from Feature	51
3.4	Proposed Approach	52
3.4.1	Non-linear Feature Approximation.	52
3.4.2	Beyond Teacher-only Activations.	54
3.4.3	Module-wise Distillation.	54
3.5	Fast and Memory-Efficient Compression	55
3.5.1	Uniform Weights Compression	55
3.5.2	Top Layers First Compression	56
3.5.3	Bottom Layers First Compression	56
3.5.4	Ease of Implementation of the approach	57

3.6	Experiments on Transformers	57
3.6.1	Setup	57
3.6.2	Results	58
3.7	Other Architectures and Modalities	59
3.7.1	Mamba Architecture	59
3.7.2	Speech modality: Whisper	61
3.8	Analysis	62
3.8.1	Which distillation loss to use?	63
3.8.2	How to choose the rank?	65
3.8.3	Is SVD initialization necessary?	65
3.8.4	Cost.	65
3.8.5	Examples of generated texts	66
3.8.6	Comparisons.	67
3.8.7	Can this approach work for low-resource and under-represented languages?	67
3.9	Limitations	70
3.9.1	Compatibility with quantization:	70
3.9.2	Compromise between specificity and genericity:	70
3.10	Conclusion	71
3.11	Open Questions	71
3.11.1	If the weights are full rank, why still compressing them?	71
3.11.2	Can an LLM be pre-trained from scratch with low-rank weights?	71
4	BALDWHISPER	73
4.1	Introduction	73
4.2	Related Work	74
4.3	BaldWhisper Approach	76
4.3.1	Stage 1: Layer Merging	76
4.3.2	Stage 2: Activation-Aware Embedding Decomposition	77
4.4	Experiments	77
4.4.1	Experimental Setup	77
4.4.2	Results	78
4.5	Analysis	79
4.6	Conclusion	80
III ACTIVATION SPARSITY OF TRANSFORMER MODELS		
5	CROSS-LINGUAL MRL	85
5.1	Introduction	85
5.1.1	Motivation	85
5.1.2	Related Work	85
5.2	Building a Wolof Speech Language Model	86
5.2.1	Background and Motivation	86
5.2.2	Wolof-Specific HuBERT Continued Pretraining	87

5.2.3	Supervised ASR Fine-Tuning and Evaluation	88
5.2.4	From a Speech Encoder to a Wolof Speech LLM	89
5.2.5	Implications for Cross-modal Retrieval	92
5.3	Datasets	92
5.3.1	Training Datasets	92
5.3.2	Evaluation Datasets	93
5.4	Modeling	94
5.4.1	Text-only Embeddings	95
5.4.2	Late Modality Fusion	95
5.4.3	Dual Modality	95
5.5	Experiments	95
5.5.1	Setup	95
5.5.2	Document Retrieval	96
5.5.3	Speech Keyword Spotting	97
5.5.4	Generalization to an Unseen Task	97
5.6	Analysis	98
5.6.1	Speech Quality in FLEURS	98
5.6.2	Prompting	99
5.6.3	Rank of Matryoshka Dimensions	99
5.6.4	Deployment Costs	100
5.7	Limitations	101
5.8	Conclusion	101

IV APPENDIX

A	APPENDIX TEST	105
A.1	Appendix Section Test	105
A.2	Another Appendix Section Test	105

BIBLIOGRAPHY	107
--------------	-----

LIST OF FIGURES

Figure 1.1	Decoder-only Transformer architecture.	7
Figure 1.2	Sparse-attention decoding costs and computation savings. (a) DeepSeek Sparse Attention (DeepSeek-v3.2) vs DeepSeek with full attention (DeepSeek-v3.1-Terminus) [33]. (b) How Sliding Window Attention reduces computation, from https://mbrenndoerfer.com/writing/sliding-window-attention	9
Figure 1.3	Proportion of the size in MB of embedding parameters across different model sizes. Data taken from https://blog.squeezebits.com/vocabulary-trimming-methods .	13
Figure 1.4	Effect of reducing the LM Head rank on accuracy and cross-entropy. Figures from Godey, Clergerie, and Sagot [51].	14
Figure 1.5	Examples of emergent abilities in large language models. Figure from Wei et al. [140]. A capacity is emergent when the score surpasses the random baseline while it wasn't at smaller scale.	15
Figure 2.1	Fixed-token scaling view of Chinchilla and Llama 3.1 models.	21
Figure 2.2	Illustration of unstructured pruning. The architecture is unchanged, but individual weights are removed independently.	21
Figure 2.3	Convergence of Sheared LLaMA after structured pruning and continued pre-training, compared to other models of similar sizes. Plot taken from the original paper [147]	23
Figure 2.4	Illustration of structured pruning. A full layer is removed, together with all its incoming and outgoing weights.	24
Figure 2.5	Sequence-level distillation. The teacher first generates a complete answer, and the student is then trained on this synthetic sequence.	25
Figure 2.6	Off-policy distillation. The student and the teacher are evaluated on the same data, and the student distribution is trained to match the teacher distribution.	26

Figure 2.7	On-policy distillation. The student first samples an output, then the teacher evaluates this student-generated output. 28
Figure 2.8	Switch Transformer encoder block, where the dense feed-forward layer is replaced by a sparse Switch FFN layer. Figure adapted from Fedus, Zoph, and Shazeer [39]. 29
Figure 2.9	Scaling properties of Switch Transformer. Increasing the number of experts improves performance and sample efficiency at equal computational budget. Figure from Fedus, Zoph, and Shazeer [39]. 30
Figure 2.10	Effect of MoE granularity on loss for fixed model size and number of training tokens. Figure adapted from Krajewski et al. [74]. 31
Figure 2.11	Expert selection proportions in Mixtral across domains and layers. The dashed line corresponds to uniform routing over the eight experts. Figure adapted from Jiang et al. [65]. 34
Figure 2.12	Cosine similarity between languages based on expert-choice patterns in NLLB-200. Figure adapted from NLLB Team et al. [99]. 35
Figure 2.13	LongCat-Flash architecture with zero-computation experts. The router still selects K experts, but some selected experts are identity paths, which makes the effective compute adaptive. Figure from Meituan LongCat Team et al. [91]. 37
Figure 2.14	Overview of a key-value memory layer. The input is projected into a query, compared to the memory keys, and only the values associated with the selected keys are used for computation. Figure from Lample et al. [77]. 37
Figure 2.15	Architecture of an n -gram embedding layer. The token embedding is augmented with hashed n -gram embedding branches. Figure from Liu et al. [86]. 39
Figure 2.16	Matryoshka Representation Learning trains a representation to be useful at several embedding dimensions. Figure adapted from Kusupati et al. [75]. 43
Figure 2.17	MatFormer nests several FFN sizes inside the Transformer block and trains the corresponding submodels jointly. Figure from Devvrit et al. [35]. 43

Figure 3.1	Lillama approach: STEP 1 selects layers to compress for a target compression ratio (e.g., N%) using various strategies (see Section 3.5). STEP 2 compresses and initializes the chosen parameters via SVD. STEP 3 distills the low-rank weights with a small calibration dataset. 47
Figure 3.2	Activations are low-rank. Comparison of the stable rank of weights (<code>*_proj</code>) and layer activations (<code>layer_activations</code>). Each bar is the average stable rank across all layers. 50
Figure 3.3	Accelerating convergence through a joint loss 53
Figure 3.4	Illustration of 20% compression when using the three strategies: bottom, top, and uniform. Green boxes are compressed layers with low-rank matrices while gray boxes are non-compressed ones. $l_1, l_2, \dots, l_{n-1}, l_n$ are the layer indexes. 55
Figure 3.5	Example of PyTorch implementation of our approach using the Teacher-only distillation strategy. 57
Figure 3.6	Higher ranks give generally better results. Mean accuracy over our test benchmarks as a function of the minimum rank k in Algorithm 1 63
Figure 3.7	SVD Initialization improves the sample-efficiency. Convergence when initializing low-rank weights randomly (without SVD) or with SVD. 64
Figure 3.8	The joint loss converges generally better. Convergence of the three losses illustrated in Figure 3.3, evaluated on the Wikitext2 test corpus perplexity during distillation. 64
Figure 4.1	Layer merging in the Whisper decoder: each student layer is a weighted average of two consecutive teacher layers, then trained with cross-entropy and knowledge distillation. 74
Figure 4.2	Head shearing of the Whisper embedding through low-rank decomposition. The decomposition is activation-aware because the low-rank weights are subsequently feature-distilled. 76
Figure 4.3	Choice of α and β . Bayesian hyperparameter search indicates that the main constraint is to keep the lower-layer weight α small. 79
Figure 4.4	Activation similarities between all decoder-layer pairs. Consecutive layers tend to be more similar, suggesting that they can be merged. 80
Figure 5.1	Wolof speech collection and filtering 87
Figure 5.2	Wolof Speech LLM architecture 90

Figure 5.3	Speech data processing pipeline	92
Figure 5.4	Late-fusion and dual architectures. Late fusion projects downsampled HuBERT features into the LLM embedding space and concatenates them with prompt embeddings. The dual architecture pools speech features and uses dimension-specific projections.	94
Figure 5.5	Dimensions needed for an energy ratio	98
Figure 5.6	Dimensions needed for full energy.	99
Figure 5.7	Recall by dimension and examples.	100
Figure 5.8	Indexing, storage, query-time, and accuracy tradeoffs across dimensions.	101

LIST OF TABLES

Table 1.1	Average SuperGLUE score for gated MLP variants.	11
Table 1.2	Examples of MLP key-value memory pairs identified by Geva et al. [49]. For each value vector $\mathbf{v}_i^\ell$, <i>Top prediction</i> is the vocabulary token receiving the highest score when the value is projected through the model’s output embedding. <i>Precision@50</i> is the percentage of the 50 training prefixes that most strongly activate the paired key $\mathbf{k}_i^\ell$ whose ground-truth next token equals that prediction. The final column presents one such matching prefix, with its ground-truth next token shown in blue.	12
Table 2.1	Main efficiency metrics for LLM training and inference. Arrows indicate the optimization direction.	19
Table 2.2	Model sizes prescribed by different scaling laws for a fixed training-data budget of 16.55T tokens.	20
Table 2.3	Vocabulary specialization in the seventh layer of OLMoE-1B-7B. Table adapted from Muenighoff et al. [96].	36
Table 2.4	Effect of n -gram vocabulary scaling in Qwen3.8-Flash-Next. Vocabulary scales are measured relative to the base tokenizer vocabulary size. Larger vocabularies add parameters. Table adapted from Qwen Team [106].	41

Table 3.1	Positioning of our approach within the compression literature. No Custom Kernel indicates that a custom GPU kernel is not required to observe speedup and No Ampere-only is whether the method requires Ampere GPU to achieve speedup. Calib. Data is the total number of tokens in billions used for compression. 48
Table 3.2	Benchmarks and corresponding metrics that were used to evaluate the models. 59
Table 3.3	Zero-shot evaluation results for the base non-compressed models (0%), and the 20%, 25%, and 30% compressed models. 60
Table 3.4	Compressed models save memory and speedup computation. Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 using a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All the models were loaded in bf16 and used Flash-Attention2. OOM stands for Out-Of-Memory. 60
Table 3.5	40% compressed Phi-2 3B competes with models of similar size while being compressed using only 13M tokens. Compressing Phi-2 3B to 1.7B (by removing 40% of the parameters) and comparing its zero-shot performance to other recent language models of similar sizes. 61
Table 3.6	Compressed models recover well when fine-tuned. Zero-shot performances of the base non-compressed models (0%) and the 40% compressed models, without and with fine-tuning (FT). Models were fine-tuned on the whole Slim-Orca dataset (191 million tokens). 61
Table 3.7	Mamba-based Language Models can be efficiently compressed. Zero-shot evaluation results for the non-compressed (0%) and for the compressed Mamba-based Language Models (20%) 61
Table 3.8	Compressing mamba models saves memory and speeds up computation. Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All models were loaded in bf16 and used Flash-Attention2. 62

Table 3.9	The 37% compressed Whisper is faster, 1.46× lighter. The WER is computed on the English test subset of the Fleurs dataset. The speed (tokens/second) corresponds to the transcription of 400 tokens from a dataset containing 2,048 speech examples, using a batch size of 128. A single A100 GPU is used and the model is loaded in FP32, without Flash-Attention. We also give the GPU Memory (VRAM) occupied by each model. 62
Table 3.10	The joint loss generally performs better: Zero-shot performances of the three losses using the same setup for each model, as described in Section 3.6. 63
Table 3.11	Comparing the Wikitext perplexity convergence of the three losses for 2M training tokens. 65
Table 3.12	Compression times for the three compression strategies at 20% parameter reduction on a single A100 GPU. For the bottom and top-first compression strategies, we used the same hyperparameters as in Section 3.6. 66
Table 3.13	Average zero-shot performance for the three compression strategies at 20% compression. For the bottom and top-first compression strategies, we used the same hyperparameters as in Section 3.6. 67
Table 3.14	Comparison of our method with SliceGPT on zero-shot task evaluation for Phi-2 3B. We give the evaluation scores before compression (0%), both as reported by SliceGPT and as reproduced by us. Both methods use the Alpaca calibration dataset for compression. 67
Table 3.15	Steve Jobs Story by the 20% and 40% compressed Phi-2 3B models, without <i>any</i> fine-tuning. Prompt: "Here the story of Steve Jobs:" 68
Table 3.16	The Civil War Story by the 20% and 40% compressed models, without <i>any</i> finetuning. Prompt: "What is the cause of the Civil War? Here is the story." 69
Table 3.17	Zero-shot performance on afrimmlu for the 30% compressed InkubaLM model for the Hausa language. 70

Table 4.1	Main results. Accuracy and efficiency after layer merging and head shearing (embedding decomposition). Speed is measured on a MacBook Air M1 while decoding 256 tokens for each utterance in a batch of 10. Bambara’s less standardized orthography may partly explain the high absolute WER. 78
Table 5.1	Hours of supervised Wolof speech used for ASR fine-tuning and evaluation. 88
Table 5.2	ASR results after identical supervised fine-tuning. The arrow denotes English pretraining followed by Wolof continued pretraining. 89
Table 5.3	ASR results of the Wolof Speech LLM. 91
Table 5.4	Speech-to-text translation results of the Wolof Speech LLM. 91
Table 5.5	Kallaama retrieval results 96
Table 5.6	Document-retrieval nDCG on Fleurs-Retrieval-Eval. 96
Table 5.7	Keyword-spotting performance across embedding dimensions. 97
Table 5.8	Few-shot speech intent detection at dimension 1024. 98
Table A.1	Autem usu id 105

LISTINGS

Listing 2.1	Pseudo-code implementation of a PEER layer forward pass. 40
Listing A.1	A floating example (listings manual) 105

ACRONYMS

Part I

SOME KIND OF MANUAL

INTRODUCTION

1.1 MODELING THE LANGUAGE

We discuss here the origin of the notion of *language model* and why calling such systems as that rises important questions on the long-standing debate between formalists and cognitivist and corpus-based linguistics. We then present the Transformer as probably the best current machine language and knowledge learner in that each component of the systems seems to learn different properties of the language and human knowledge broadly. We then propose the claims and work of the thesis.

1.1.1 *A dialog between Shannon and Chomsky*

It is curious that we still don't know where the expression *language model* come from and when it establishes. Not the notion of what is a *language model* but the expression itself. Because for the notion, we know that the idea of language model is quite old and one of the earliest formulations closely related to what we now call language modeling can be found in Shannon's work on information theory Shannon [119]. Shannon starts from a simple observation: sequences of linguistic symbols are not random, but exhibit regularities characteristic of the language.

"[...] the messages to be transmitted consist of sequences of letters. These sequences, however, are not completely random. In general, they form sentences and have the statistical structure of, say, English."

— Claude E. Shannon, *A Mathematical Theory of Communication*
(1948)

If English sentences have a statistical structure, then a statistical model that captures this structure should be able to generate sequences that look like English. But what is this statistical model? How can we estimate it? Later on, again Shannon gives a more operational view:

"We can think of a discrete source as generating the message, symbol by symbol. It will choose successive symbols according to certain probabilities depending, in general, on preceding choices [...]."

— Claude E. Shannon, *A Mathematical Theory of Communication*
(1948)

Shannon describes text generation as a sequential probabilistic process: the next symbol is sampled according to a probability distribution that depends on the preceding context. He then considers increasingly higher-order approximations, where the next symbol depends on a progressively longer context. These probabilities can be estimated from text by simply estimating how frequently a symbol occurs after a given context. This already contains the basic ingredients of what we would now recognize as an autoregressive language model: modeling the probability of the next symbol conditioned on its preceding context. We will discuss this more formally. Still, while Shannon describes what conceptually is the language model as we perceive it today, he doesn't use the expression *language model*. Calling these systems as such could suggest that they are a linguistic model of a language. But does a better statistical model of English necessarily mean a better model of its grammar? While this idea is natural from a corpus-based or cognitive view of language, it is strongly rejected by Chomsky. In *Three Models for the Description of Language*, he states Chomsky [21, p. 113]:

"We find that no finite-state Markov process that produces symbols with transition from state to state can serve as an English grammar. [...] such processes that produce n-order statistical approximations to English do not come closer, with increasing n, to matching the output of an English grammar."

— Noam Chomsky, *Three Models for the Description of Language*
(1956)

There are two main claims made by Chomsky in this paper. The first claim is structural and concerns the **bounded memory** of finite-state models. For finite-order Markov language models, this limitation manifests as a fixed context length: only a bounded portion of the preceding sequence can determine the next symbol. English, however, permits recursive constructions whose dependencies can require information from an arbitrarily large preceding structure. Modern language models have greatly increased the amount of preceding context they can exploit, making them much better at modeling long-range dependencies. Yet practical models still operate with bounded context and memory, so the distinction highlighted by Chomsky between unbounded linguistic structure and bounded computational memory remains relevant. Second, even if we consider a statistical model with unbounded memory, Chomsky's criticism goes beyond model capacity. His broader claim is that **language innate**, biologically programmed in the human brain Chomsky [22] and cannot be learned simply by accumulating statistics over observed linguistic data. The main argument is the *poverty of stimulus*: a child learns its native language rapidly, despite being exposed to limited, noisy, and incomplete examples. This is possible because the infants implement structural biases that

help learn and generalize linguistic structures from few examples. Chomsky's theory holds that human linguistic competence depends on internal constraints that cannot be inferred from experience alone.

So it is difficult to pinpoint when the expression *language modeling* became established, and what exactly was originally meant by it. From a Chomskian perspective, a model of a language should explain the structure of that language; Chomsky explicitly rejected the idea that a statistical model fitted to linguistic data could constitute such a model. Shannon, meanwhile, developed statistical models of English but, importantly, did not call them *language models*. The story takes a different turn in the early 1970s. In 1974, a team at IBM working on speech recognition described a statistical model closely related to Shannon's formulation and explicitly called it a *language model*. In their probabilistic decomposition of speech recognition, the language model is simply the component that assigns probabilities to possible word sequences—that is, a model of which utterances are more or less likely in a given language. As they put it, “it is the function of the language model (LM) to provide us with $\Pr\{W \mid L\}$,” where W denotes a word sequence and L the language. This work is therefore probably one of the earliest uses of the expression *language model* in essentially its modern statistical sense: a probability distribution over linguistic sequences estimated from language data.

Indeed, this is not a model of a language in the Chomskyan sense, since language is assumed to rely on innate structure and therefore cannot be modeled from experience alone. It is not straightforward, however, to translate this notion of innateness into the language of machine learning. Does Chomsky's argument imply that general-purpose learning architectures are inherently incapable of acquiring language from data alone? Or can we instead design computational architectures and general learning objective whose inductive biases make the acquisition of linguistic structure easier? ¹ Maybe we can also posit the question in the reverse: does learning from experience alone help learning the structure latent in language. Even though modern machine learning algorithms are still far from matching the data efficiency of human children, substantial progress has been made in language modeling. Let us now look at how language modeling architectures has evolved.

As discussed previously, a language model is exactly the one Shannon describes: a text is generated by predicting the next symbol from a fixed number of previous symbols, the n -gram context. These symbols can be letters or words, but nowadays they are usually tokens, units that can form a word or a subword. More formally, the probability of picking a token s_i given a sequence of previous tokens $s_{<i} = (s_1, s_2, \dots, s_{i-1})$ is written as $P(s_i \mid s_{<i})$. Using the chain rule,

¹ For Chomsky, this question is not really interesting, as reproducing linguistic behavior with an artificial architecture is not, by itself, an explanation of human language acquisition:

<https://www.youtube.com/watch?v=PZQ>

these individual probabilities can be multiplied together to give the probability of the whole text:

$$P(s) = p(s_1) \times p(s_2|s_1) \times p(s_3|s_1, s_2) \times \cdots \times p(s_t|s_{<t}) = \prod_{i=1}^t P(s_i | s_{<i}). \quad (1.1)$$

Each individual probability can be defined from a score over the vocabulary:

$$p(s_i | s_{<i}) = \frac{\text{score}(s_{<i}, s_i)}{\sum_{v \in \mathcal{V}} \text{score}(s_{<i}, v)}. \quad (1.2)$$

Here, $\text{score}_\theta(x, y)$ measures how strongly the context x is associated with the next token y , and $\mathcal{V}$ is the token vocabulary. The function $\text{score}(x, y)$ can be designed very freely but is often a counting function: how often y appears after the context x in the training corpus.

Using Maximum Likelihood Estimation, the probabilities of the sequence can be estimated by a statistical model parameterized by θ on a large training corpus $\mathcal{C}$ of texts:

$$\theta^* = \arg \max_{\theta} \sum_{s \in \mathcal{C}} \sum_{i=1}^{|s|} \log P_\theta(s_i | s_{<i}). \quad (1.3)$$

This learning objective does not really change for generative language models. What changes is how the probabilities are represented and learned. **MLE: compression, memorization and teacher-forcing**

Parler des tests psycholinguistique sur les modèles. Du fait que rajouter des biais linguistiques aident à certains niveau mais disparaît à l'échelle. Que l'imitation learning permet en grande partie aux modèles d'apprendre les structures linguistiques latentes, et des différentes études sur les LLMs à ce sujet.

Voir les travaux de Alex W., Tal Linzen, Lavechin, Dupoux, **file: :///Users/yysy/Downloads/Can_SL_bootstrap_ela.pdf**

BEYOND IMITATION LEARNING

1.1.2 The Transformer, the best machine language learner?

While Banhaudeau et al. improved performance by adding attention on top of RNN layers, Vaswani propose to fully remove the RNN and keep only the attention. This remove the sequential processing imposed by RNN, which make the training parallelisable and hence scalable, since attention is full parallelisable. Figure x presents the transformer architecture for language modeling. The architecture alternates between attention and MLP for N layers and then finally the output, the next token prediction, is computed. Let's detail and discuss these 3 components and how each of them acquires knowledge and generalize linguistics structures.

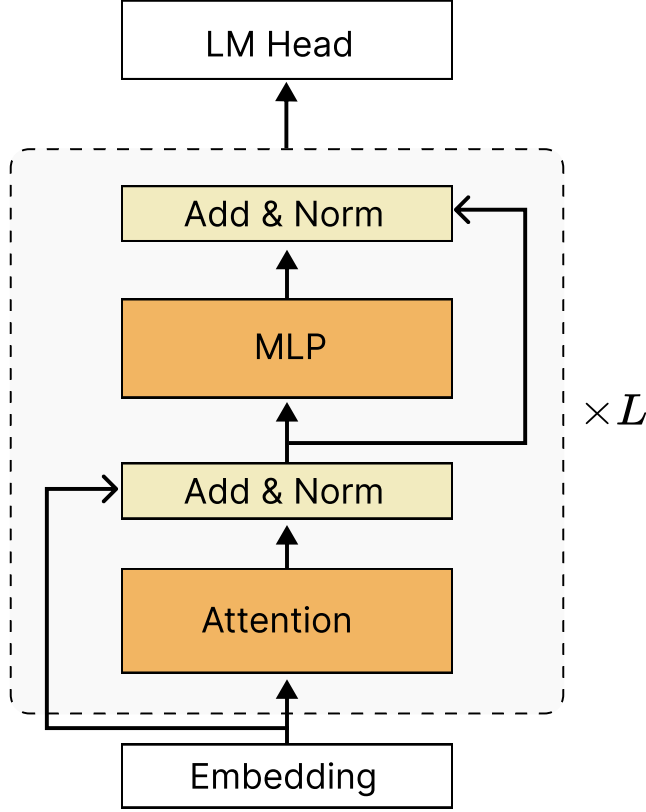


Figure 1.1: Decoder-only Transformer architecture.

1.1.2.1 Attention

The Transformer haven't introduced attention since we have seen that this has been used by Banhaudeau et al., in an RNN architecture. The innovation of Transformer is to remove the RNN and replace it fully with **Multi-Head Attention (MHA)**. Given $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, the sequence of token embeddings for T positions, MHA projects the sequence into queries, keys and values for H heads:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times H \times d_{\text{head}}}. \quad (1.4)$$

Equivalently, each head h has its own projections:

$$\mathbf{Q}_h = \mathbf{X} \mathbf{W}_h^Q, \quad \mathbf{K}_h = \mathbf{X} \mathbf{W}_h^K, \quad \mathbf{V}_h = \mathbf{X} \mathbf{W}_h^V, \quad (1.5)$$

where $\mathbf{W}_h^Q \in \mathbb{R}^{d_{\text{model}} \times d_q}$, $\mathbf{W}_h^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_h^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are learned projection matrices for head h . The head output is then computed with scaled dot-product self-attention:

$$\text{head}_h = \text{softmax} \left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top + \mathbf{M}}{\sqrt{d_k}} \right) \mathbf{V}_h. \quad (1.6)$$

For a decoder language model, $\mathbf{M}$ is a causal mask: entries corresponding to future positions are set to $-\infty$, so token i can only attend to

tokens $j \leq i$. Multi-head attention runs H such heads in parallel and concatenates their outputs:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O, \quad (1.7)$$

with $\mathbf{W}^O \in \mathbb{R}^{Hd_v \times d_{\text{model}}}$. Each head can therefore learn a different projection of the same context, while the output projection mixes these views back into the residual stream.

TODO: Linguistics specialization of heads

In standard MHA, as introduced in the original Transformer paper, the projection matrices $\mathbf{W}_h^Q$, $\mathbf{W}_h^K$ and $\mathbf{W}_h^V$ are usually chosen so that all heads together project back to a space of size d_{model} . However, these projections can become a bottleneck during generation. In a naive implementation, at each decoding step, all previous tokens would be projected again to recompute their keys and values. The **KV-Cache** reduces this computation and speeds up generation by caching the keys and values of previous tokens. However, this cache grows linearly with the sequence length, and it is easy to run out of memory when generating very long texts. To reduce the KV-Cache, we need to reduce the dimensionality of keys and values.

TODO: Redundancy of Heads and transition to GQA

Among the popular approaches to reduce the KV-Cache, **Group Query Attention (GQA)** shares key-value pairs to multiple query heads. Intuitively, the keys and values have fewer heads than the queries:

$$\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times h_g \times h_{\text{dim}}}, \quad (1.8)$$

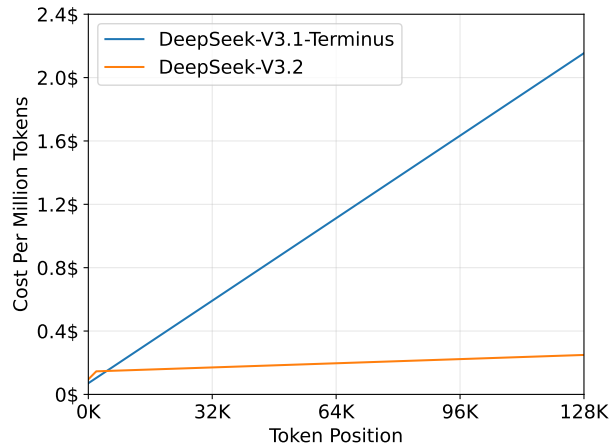
$$\mathbf{Q} \in \mathbb{R}^{T \times h_q \times h_{\text{dim}}}, \quad h_g < h_q. \quad (1.9)$$

The key-value heads are then broadcast to the query heads, and attention is computed as usual:

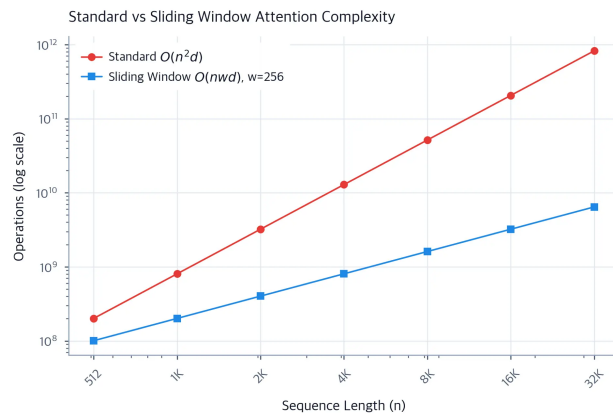
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top) \mathbf{V}. \quad (1.10)$$

Standard MHA corresponds to no sharing, while multi-query attention is the extreme case where all query heads share the same keys and values. GQA is the compromise between the two: it reduces the KV-Cache while keeping several different key-value heads.

While GQA reduces KV-cache memory, it does not reduce the attention computation. **Sparse Attention** reduces this compute by allowing each query to attend to only a subset of previous tokens. DeepSeek Sparse Attention [33] introduces a lightweight *lightning indexer* that cheaply scores all previous tokens using a small number of low-dimensional attention heads. The top- k scoring keys are then selected, and the expensive main attention is computed only over those key-value entries. The main attention therefore changes from $\mathcal{O}(L^2)$ to $\mathcal{O}(Lk)$, where $k \ll L$ is the number of selected tokens. The indexer itself remains $\mathcal{O}(L^2)$, but its constant cost is much smaller because it



(a) DeepSeek Sparse Attention.



(b) Sliding Window Attention.

Figure 1.2: Sparse-attention decoding costs and computation savings. (a) DeepSeek Sparse Attention (DeepSeek-v3.2) vs DeepSeek with full attention (DeepSeek-v3.1-Terminus) [33]. (b) How Sliding Window Attention reduces computation, from <https://mbrenndoerfer.com/writing/sliding-window-attention>

uses very small dimensions, few heads, and FP8 computation. While many recent models use a derivative of DeepSeek Sparse Attention (cite Qwen, GLM), it is not the first work to introduce sparse attention (*TODO: Attention Sink*), which has been widely studied [12, 72, 112, 129]. Sliding-Window Attention is a simple form of sparse attention where each query attends only to a fixed number of nearby tokens instead of the entire preceding sequence. For a query at position i and a window size w , attention is restricted to the key-value pairs $(k_{i-w}, v_{i-w}), \dots, (k_i, v_i)$. Each query therefore attends to only w tokens rather than all L previous tokens, reducing the attention complexity from $O(L^2)$ to $O(Lw)$, where typically $w \ll L$. This saves compute and save KV-Cache memory if the implementation discards KV states that fall outside the active window Sliding-window attention benefits from model depth because information can propagate from one local window to another across successive layers, even though attention is local within each individual layer. However, long-range information is accessed only indirectly through model depth, whereas full attention allows a query to directly attend to any previous token at each layer. For this reason, many LLMs use hybrid architectures that combine local-attention layers with occasional full-attention layers [165–167]. Many of the efficient attentions methods are orthogonal to each other, so they can be combined. For example, it is possible to combine sparse attention with local attention and sliding window attention. Or just as DeepSeek-v4 combine sparse attention with multi-latent attention which is another method for learning compressed kv-cache during pre-training.

1.1.2.2 Multilayer Perceptron

After the attention layer, the contextualized tokens are fed into the Multilayer Perceptron (MLP). While attention is a token-mixing operation, the MLP is a channel-mixing operation, each token is forwarded independantly into the MLP. In the original Transformer’s paper, the MLP is defined as:

$$\text{MLP}(x) = f(\mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$$

where $f(\cdot)$ is the non-linear activation function, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{inter}}}$ project the input token $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ in higher dimension, with $d_{\text{model}} \ll d_{\text{inter}}$. And finally, $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{inter}} \times d_{\text{model}}}$, projects backs to the model dimension. Modern LLMs often replace this simple MLP with a gated linear unit variant. Instead of applying only one non-linearity to the expanded representation, the block learns two projections: one

projection $\mathbf{V}_{\text{up}}$ produces the values, and the other projection $\mathbf{W}_{\text{up}}$ gates them. Omitting bias terms, these alternatives can be written as:

$$\text{MLP}_{\text{GLU}}(\mathbf{x}) = (\sigma(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.11)$$

$$\text{MLP}_{\text{GEGLU}}(\mathbf{x}) = (\text{GELU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.12)$$

$$\text{MLP}_{\text{Bilinear}}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_{\text{up}} \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.13)$$

$$\text{MLP}_{\text{SiLU}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.14)$$

$$\text{MLP}_{\text{ReGLU}}(\mathbf{x}) = (\text{ReLU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}. \quad (1.15)$$

Here $\odot$ denotes element-wise multiplication. 1.1 gives the performance of each activation on SuperGLUE benchmark. We can see that SiLU activation gives better results, this explains why many LLMs use it as activations: Mistral-v0.1, Granite, etc. It is difficult to explain why

Average SuperGLUE score	
MLP_{GLU}	73.95
$\text{MLP}_{\text{GEGLU}}$	73.96
$\text{MLP}_{\text{Bilinear}}$	73.81
$\text{MLP}_{\text{SwiGLU}}$	74.56
$\text{MLP}_{\text{ReGLU}}$	73.66

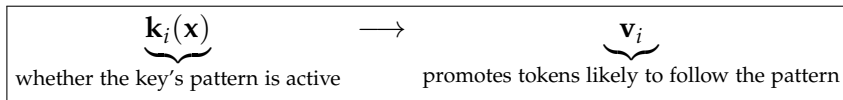
Table 1.1: Average SuperGLUE score for gated MLP variants.

gated MLP with work better with some activations than other. Citing Shazeer [120], the author of gated MLPs:

“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.”

— Noam Shazeer, *GLU Variants Improve Transformer* (2020)

In modern LLM architectures, the MLP parameters can occupy over to 90% of the total model parameters. It has been shown that MLP layers play an important role in storing knowledge acquired during web-scale pretraining. Geva et al. [49] interpret the MLP as a key-value memory: each column of $\mathbf{W}_{\text{up}}$ in the MLP represents a key $\mathbf{k}_i$ that detects linguistic patterns seen during training, while each row of $\mathbf{W}_{\text{down}}$ represents the corresponding value $\mathbf{v}_i$, which promotes the tokens likely to follow those patterns. The key $\mathbf{k}_i$ can be strongly or weakly activated depending on the input x . When $\mathbf{k}_i$ is strongly activated, its corresponding value $\mathbf{v}_i$ is assigned a large weight and therefore contributes more strongly to the MLP output. This mechanism can be schematized as follows:



Value	Top prediction	Precision@50	Matching trigger example
$\mathbf{v}_{222}^{15}$	<i>each</i>	68%	<i>But when bees and wasps resemble each</i>
$\mathbf{v}_{752}^{16}$	<i>played</i>	16%	<i>Her first role was in Vijay Lalwani's psychological thriller Karthik Calling Karthik, where Padukone was cast as the supportive girlfriend of a depressed man (<i>played</i>)</i>
$\mathbf{v}_{2601}^{13}$	<i>extratropical</i>	4%	<i>Most of the winter precipitation is the result of synoptic-scale, low-pressure weather systems (large-scale storms such as <i>extratropical</i>)</i>
$\mathbf{v}_{881}^{15}$	<i>part</i>	92%	<i>Comet served only briefly with the fleet, owing in large <i>part</i></i>
$\mathbf{v}_{2070}^{16}$	<i>line</i>	84%	<i>Sailing from Lorient in October 1805 with one ship of the <i>line</i></i>
$\mathbf{v}_{3186}^{12}$	<i>jail</i>	4%	<i>On May 11, 2011, four days after scoring 6 touchdowns for the Slaughter, Grady was sentenced to twenty days in <i>jail</i></i>

Table 1.2: Examples of MLP key–value memory pairs identified by Geva et al. [49]. For each value vector $\mathbf{v}_i^\ell$, *Top prediction* is the vocabulary token receiving the highest score when the value is projected through the model’s output embedding. *Precision@50* is the percentage of the 50 training prefixes that most strongly activate the paired key $\mathbf{k}_i^\ell$ whose ground-truth next token equals that prediction. The final column presents one such matching prefix, with its ground-truth next token shown in blue.

The MLP does not only store lexical knowledge. At some point, this lexical knowledge coincides with factual knowledge. When a key detecting the prefix *The capital of France is* is strongly associated with a value representing *Paris*, this lexical key–value memory turns out to also encode world knowledge. Hence, more broadly, it is widely believed that MLPs play a central role in storing the knowledge acquired during pretraining Dai et al. [29] and Geva et al. [49]. This view is also reflected in many knowledge-editing methods, which often directly target MLP weights Fang et al. [38] and Meng et al. [93, 94].

1.1.2.3 Language Modeling Head

Well, the tokens are contextualized thanks to the multi-head attention, and each token retrieves the relevant knowledge encoded within the

MLP. This process is repeated for L layers and at the end produces a context vector $\mathbf{c}$ that is ready for next token prediction using the language modeling head (LM Head or output embedding). The LM Head is another important component in large scale Transformer LLM. First, the token vocabulary becomes quite large due to the diversity of tokens that can be inferred from a web scale data. Hence, the LM Head weight can occupy a huge amount of parameters in the LLM, particularly for small LLMs. Figure 1.3 shows the memory occupied by the input/output embedding parameters for different model sizes, including dense and quantized ones. We can see that the embedding and LM Head parameters represent a larger fraction of the model when the Transformer backbone is small². This is mainly caused by the

² Quantization is usually not applied to the embedding weights.

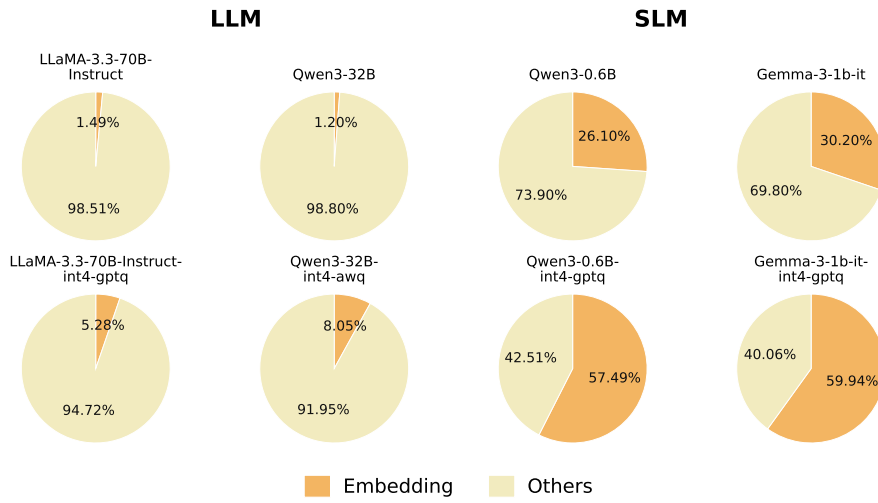


Figure 1.3: Proportion of the size in MB of embedding parameters across different model sizes. Data taken from <https://blog.squeezebits.com/vocabulary-trimming-methods>.

large vocabulary of the models. A common solution to reduce the size by half is weight tying, which shares the input and output embedding. Almost all small LLMs use weight tying, which haven't proven to hurt parameters, while still reducing a lot the model's parameters.

Another approach to reducing the number of parameters occupied by the LM head is to factorize its weight matrix into two low-rank matrices. This strategy was notably adopted in ALBERT [78], where the embedding dimension was reduced from 768 to 128 without sacrificing downstream performance. More precisely, given the input $\mathbf{x} \in \mathbb{R}^d$ and an LM-head weight matrix $\mathbf{W} \in \mathbb{R}^{d \times V}$, the factorized projection is

$$\mathbf{xW} \longrightarrow (\mathbf{xW}_1)\mathbf{W}_2, \quad (1.16)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times r}$, $\mathbf{W}_2 \in \mathbb{R}^{r \times V}$, and $r \ll H$. This reduces the LM-head parameter count from dV to $dr + rV$.

However, some works have identified what is known as the *softmax bottleneck* [152]: a low-dimensional LM head can only produce a limited variety of output distributions. Therefore, reducing its rank too

aggressively can limit the model’s ability to predict complex token distributions. Godey, Clergerie, and Sagot [51] studied this by compressing the LM Head rank dimension at different sizes and found that smaller rank hurts performance and limits the expressiveness of the probability distribution as shown in Figure 1.4. More importantly, this also hurts the learning signal during training Godey and Artzi [50]. There are many solutions to avoid the *softmax bottleneck*. Since this phenomenon arises mainly when $r \ll V$, an obvious solution is to augment r or V the vocabulary size. Reducing the vocabulary size can be achieved by using a tokenizer with few tokens or falling back to a character or bytes tokenizers. It is also technically possible after training to remove tokens in the input/embedding embedding. But this is risky as it can affect critical tokens. This discussion is related to the research we will present later, which proposes to reduce the rank of the LM Head instead of pruning tokens in multilingual transformer models.

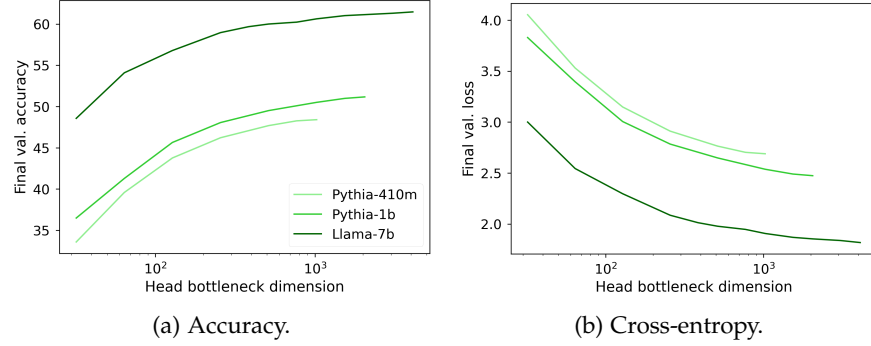


Figure 1.4: Effect of reducing the LM Head rank on accuracy and cross-entropy. Figures from Godey, Clergerie, and Sagot [51].

1.1.3 A Transformer is never enough large

What made the Transformer architecture particularly successful in language modeling is its ability to scale both the number of parameters and the amount of training data. Since Transformer training can be efficiently parallelized across multiple GPUs, increasingly large models can be trained on increasingly large datasets, leading to improved performance and the emergence of new capabilities. However, scaling cannot be done blindly: given the high cost of pretraining, the allocation between model size and training data must be estimated before the final training run. For a fixed compute budget, how large should the model be, and on how many tokens should it be trained?

Denoting the number of parameters by N , the number of training tokens by D , and the training compute by C , the compute cost of a dense Transformer can be approximated as $C \approx 6ND$. So given

a training budget C fixed (how many training flops is available for training), how much the model size should take in this compute and how much the training tokens should take in this budget. Both Kaplan [68] and Chinchilla [59] scaling find that the compute-optimal model size and dataset size follow power laws in the compute budget: $N_{\text{opt}} \propto C^\alpha$ and $D_{\text{opt}} \propto C^\beta$. **Kaplan scaling laws** estimate that the compute-optimal allocation with $N_{\text{opt}} \propto C^{0.73}$ and $D_{\text{opt}} \propto C^{0.27}$. Thus, model size should increase much faster than training data, which favors very large models trained on comparatively few tokens. On the other side, the **Chinchilla scaling laws** later find that model size and training data should instead scale approximately equally: $N_{\text{opt}} \propto C^{0.5}$ and $D_{\text{opt}} \propto C^{0.5}$. Since Kaplan and Chinchilla scaling laws, lot of types scaling laws are proposed for training LLMs. Often it is about adding new scaling axis such as sparsity and granularity in the case of sparse Transformer model. It can also be about accounting a constrain such as an inference budget.

Beyond guiding compute allocation, scaling laws also reveal how model capabilities evolve with scale. While performance often improves predictably as training compute increases, some abilities appear only after the model reaches a certain scale. The model may remain close to random performance across several scales before suddenly crossing a plateau and acquiring a new ability. Wei et al. [140] identify several such *emergent abilities*.

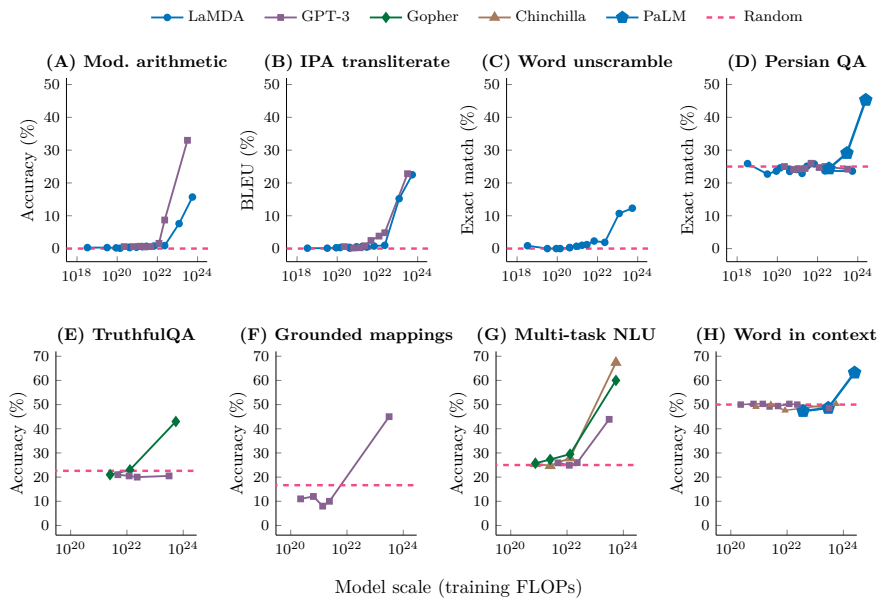


Figure 1.5: Examples of emergent abilities in large language models. Figure from Wei et al. [140]. A capacity is emergent when the score surpasses the random baseline while it wasn't at smaller scale.

Then a question arises. Is there a point beyond which increasing the number of parameters no longer produces meaningful improvements? Testing this hypothesis directly is difficult, since training models at

the largest scales is extremely costly. Classical scaling laws predict *diminishing returns*: test loss continues to decrease with scale, but each additional increase in model size produces a smaller improvement [59, 68]. However, whether this implies diminishing returns in model capabilities depends on how performance is measured. Sinha et al. [122] argue that short-task benchmarks may create an illusion of saturation. Even small improvements in single-step accuracy can compound into large improvements in the length of tasks that a model can reliably execute. Therefore, diminishing returns on loss or short-task accuracy do not necessarily imply diminishing returns in practical capabilities.

EFFICIENT LANGUAGE MODELS

We've discussed Transformer and how scaling them yields strong improvement in natural language tasks. However, scaling a Transformer-based language model is costly in training compute but also in inference budget. We discuss here LLM efficiency mainly through the lens of **model compression** and **activation sparsity**, since our contributions concern these two aspects. We present here common approaches to make LLMs efficient to scale during training, but also to serve efficiently during inference.

When talking about *efficiency* in LLMs, there are few concepts to have in head. The first distinction is between **compute-boundness** and **memory-boundness**. A model or a task is compute-bound when the bottleneck is the number of arithmetic operations that can be done per second. In that case, the limitation is not only the theoretical peak compute of the hardware, but also how well the model and the implementation can use this compute. A model or a task is memory-bound when the bottleneck is instead the movement of data: loading weights, reading activations, writing tensors, or moving the KV-cache from the GPU memory. Depending on whether we are training, doing prefill, or decoding tokens, these two concepts do not have the same consequences. This distinction explains why diffusion based LLMs such LLaDa or Gemma-Diffusion are much faster than autoregressive LLMs. Diffusion LLMs are compute-bound such that they are more FLOP-Intensive than autoregressive LLMs. So, since the decoding is parallel it reduces the data-movement compared to autoregressive LLMs, and leads to faster generation.

LLM training is mostly compute-bound, because training is highly parallel. Many tokens are processed at once, and the forward and backward passes are dominated by large matrix multiplications. So the key metrics to check are **FLOPS**, **MFU** and **Tokens/s**. **FLOPS** measures how many Floating Point Operations are executed per second, while **MFU** measures how much of the theoretical peak compute of the hardware is really used. The goal is to make the model perform a lot of useful operations per second while minimizing communication: between GPUs, between CPU and GPU, and between HBM and the compute units of the GPU. This is why training efficiency is not only about reducing the number of parameters. It is also about keeping the hardware busy. We will later see that small dense models, sparse models, and compressed models do not all use the hardware in the same way. FLOPS and MFU are hardware-oriented numbers, so they

are often completed by **Tokens/s**, which measures at which speed the model is consuming the training data.

LLM inference can be compute-bound, but is more often memory-bound because tokens are generated token by token. At each decoding step, the model has to read its weights and the KV-cache to produce only one new token, which creates a lot of memory traffic for a relatively small amount of computation. There are two phases in LLM inference. The first is the **prefill**, where the system prompt, the user prompt and the context are fed in parallel to the LLM. This phase is closer to training and is often compute-bound. The second phase is **decoding**, where the model generates the text token per token. This is usually the bottleneck, because the model repeatedly accesses the same weights and an increasingly large KV-cache. It is important to understand this difference because a model that requires more FLOPS can sometimes be faster if it uses the hardware better or moves less memory. The key metrics for inference follow these two phases. For prefill, we usually look at **Time To First Token (TTFT)**, which measures how long the user waits before seeing the first generated token, and **prefill Tokens/s**, which measures how fast the prompt is processed. These two metrics are related, but they are not exactly inverse because TTFT also includes overheads and the generation of the first token. For decoding, we look at **Time Per Output Token (TPOT)**, which measures the time needed to generate one new token, and **generated Tokens/s**, which measures how many new tokens are produced per second. These two metrics describe the same decoding speed with inverse units. Finally, we also care about the **memory footprint**, especially the weights and KV-cache memory.

This part is heavily inspired by the blogpost we wrote in collaboration with the Datacraft team in 2024: <https://medium.com/@yayasy-nlp/a-comprehensive-overview-of-large-language-models-compression-methods-8b59ddoff4f8>

2.1 MODEL COMPRESSION

An approach to reduce the memory footprint, and sometimes also accelerate inference by reducing the FLOPs, is model compression: making the model smaller while trying to preserve most of its performance. There is a tension here. Scaling laws usually advocate for more training compute, and often this means increasing the number of parameters. Inference efficiency, in contrast, advocates for less compute, less memory movement, and a smaller memory footprint.

We present here two ways of thinking about this tension. First, we discuss small LLMs such as Llama, where the objective is not only to be compute-optimal during training, but also to account for inference-optimality. The idea is to train smaller models much longer than what the training-optimal rule would suggest, so that they remain small

Phase	Metric	What it measures
Training	FLOPS ↑	Number of floating point operations per second.
	MFU ↑	Fraction of the theoretical hardware compute that is effectively used.
	Tokens/s ↑	Number of training tokens processed per second.
Inference - Prefill	TTFT ↓	Time before the first generated token is returned to the user.
	Prefill Tokens/s ↑	Speed at which the input prompt and context are processed.
Inference - Decoding	TPOT ↓	Time needed to generate one new token.
	Generated Tokens/s ↑	Number of new tokens generated per second. It is the inverse view of TPOT.
Training and inference	Memory footprint ↓	Memory occupied by weights, activations, optimizer states during training, and the KV-cache during inference.

Table 2.1: Main efficiency metrics for LLM training and inference. Arrows indicate the optimization direction.

at inference time while still performing well. We then present the approach of “*train large, then compress*”, which consists of training a large model first, then compress it by keeping the most important components of the model, or by distilling the knowledge of large models into smaller ones.

2.1.1 Llama small models or how to account inference-optimality

The Llama series of models were among the first to strongly push the idea that training should also account for inference compute. Instead of only following what Chinchilla compute-optimality suggests, they train small models on much more tokens. The result is a model that

Table 2.2: Model sizes prescribed by different scaling laws for a fixed training-data budget of 16.55T tokens.

Training tokens	Kaplan N_{opt}	Chinchilla N_{opt}	Llama 3 N_{opt}
16.55T	$\sim 100,000\text{T}$	$\sim 827.5\text{B}$	402B

is not training-optimal according to Chinchilla, but is much more efficient at inference time: it is smaller, cheaper to run, and still very performant. This point is important for open models. A smaller but strong model can be deployed more easily by independent users, especially those who do not have access to large compute resources. In 2023, Llama-1 and Llama-2 made the ecosystem of open-source tools for local LLM usage much more concrete, with projects such as Llama-cpp, Llama-index, and Ollama becoming central for running and using LLMs outside large GPUs clusters. Llama-3 pushed this idea further by training models such as the 8B model on 16.55 trillion tokens. In this work they develop their own scaling laws where their estimated model size and training tokens compute allocation advocates for training smaller models on more tokens. Table 2.2 presents the estimated number of parameters that should be trained for a fixed budget of 16.55 trillion tokens by Kaplan, Chinchilla and Llama scaling laws.

How can we explain these differences in compute-optimal allocation estimates? The history of scaling laws is also a history of scaling the compute used to study them and improving the methods used to fit the experimental results. The resulting scaling laws may also differ depending on the hyperparameters, model architecture, and data quality. This explains why the Llama 3 scaling-law estimates differ from those of Kaplan and Chinchilla. By the time Llama 3 was released, the field had evolved considerably in terms of data quality, model design, and optimization techniques for large-scale LLM training.

More importantly, the Llama 3 advocate training small models much longer, beyond what is compute-optimal. Figure 2.1 presents Llama 3 8B and Llama 3 70B in the chinchilla compute-optimal estimate. We can see the Llama 3 models are trained beyond of what is compute-optimal. While this is costly at training time, the resulting small models are stronger and efficient to serve for inference.

2.1.2 *Layer Looping: Increasing expressivity without increasing model parameters*

2.1.3 *Unstructured Pruning*

Instead of training small for longer, another approach to produce small models is to train large models first and then compress it or distill

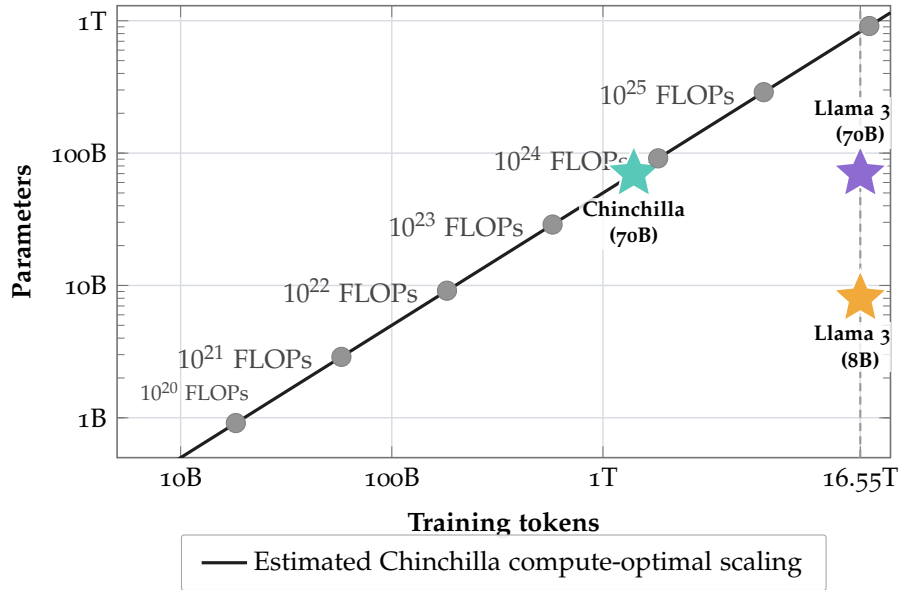


Figure 2.1: Fixed-token scaling view of Chinchilla and Llama 3.1 models.

it in efficient architecture. Compressing larger and more powerful pre-trained models is a viable alternative. This can be achieved using many complementary methods: pruning, distillation, quantization, and more.

The intuition of pruning is identifying unimportant weights in the network and removing them when possible or setting them to zero. There are two main pruning methods: **unstructured pruning** and **structured pruning**.

2.1.3.1 Unstructured Pruning

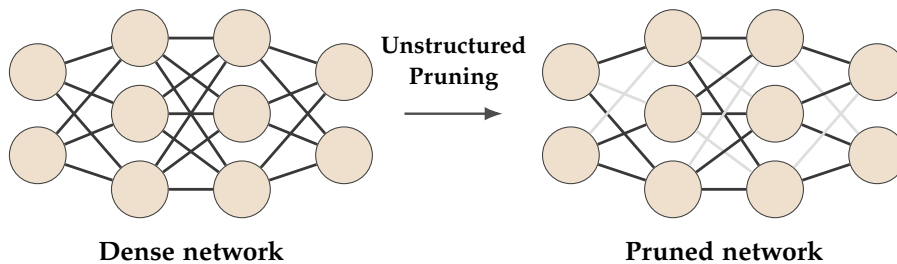


Figure 2.2: Illustration of unstructured pruning. The architecture is unchanged, but individual weights are removed independently.

Unstructured pruning identifies and removes individual unimportant weights from the model. Figure 2.2 illustrates this process, where some model weights are individually removed, producing a sparse model. It may be more accurate to call these approaches **weight sparsity**, because setting weights to zero does not directly reduce the dimensions of the model. The sparse weights can only reduce the model’s memory footprint when they are stored using a specific sparse

format. However, this format must also store the positions of the non-zero weights and therefore usually becomes beneficial only at high sparsity levels. In pretrained LLMs, achieving such high sparsity without significantly degrading performance remains difficult. Similarly, unstructured sparsity can provide speed-ups only at high sparsity levels and with specialized GPU kernels. Another approach is **semi-structured sparsity**, which constrains a *pattern* in the the positions of the removed weights. For example, 2:4 sparsity removes exactly two weights from every group of four. This regular pattern facilitates the development of GPU kernel that can accelerate computation in hardware.

SparseGPT [41] and Wanda [126] are two popular post-training methods that support both unstructured and semi-structured sparsity. SparseGPT uses activations collected from a small calibration dataset to estimate the effect of removing each weight. After pruning a weight, the remainings are adjusted to compensate for the resulting reconstruction error. Wanda is much simpler and cheaper: it removes weights that are both small and connected to weakly activated features. For each weight W_{ij} , Wanda computes the importance score $S_{ij} = |W_{ij}| \|\mathbf{X}_j\|_2$, where $|W_{ij}|$ is the magnitude of the weight and $\|\mathbf{X}_j\|_2$ is the ℓ_2 norm of its corresponding input feature over the calibration data. Weights with lower score are removed. Beyond SparseGPT and Wanda, many works proposed new unstructured/semi-structured algorithms. Some proposing improved weight-importance criteria, such as Plug-and-Play [161], non-uniform sparsity allocation across layers, such as OWL [153], or methods that iteratively refine the initial pruning mask, such as DSnoT [162].

2.1.4 Structured Pruning

Unstructured Pruning works well but the resulting models require specific implementations to accelerate inference or reduce memory requirements. To easily achieve acceleration and memory gain out-of-the box, it is better to shrink components of the model. This is the purpose of **Structured Pruning** which is illustrated in Figure 2.4. In structured pruning, entire substructures (or blocks) are removed directly, for example, the Attention Heads, Rows/Columns of the matrices, or even whole layers. Since entire substructures are pruned, Structure pruning imposes more constraints compared to unstructured pruning or semi-structured pruning where individual weights are remove more independently and freely. This explains why unstructured pruning usually perform better than structured pruning. However, since structured pruning has more potential in improving inference and reducing memory footprint.

LLM-Pruner [90] uses gradient information to estimate the importance and removes the least important groups. The method can remove

approximately 20% of the parameters of LLaMA-7B while retaining 89.8% of its original performance without post-training. However, more aggressive parameter reduction generally leads to a significant performance drop. Therefore, larger reductions often require continued training. This is illustrated by **Sheared Llama** [147], which learns binary masks over layers, attention heads, MLP dimensions, and hidden dimensions. Since binary masks are not directly differentiable, the method uses a continuous reparameterization to learn which structures should be removed. The authors prune LLaMA2-7B into models containing 2.7B and 1.3B parameters, and continue their training on 50B tokens using the language modeling loss. The pruned model converges fastly as shown in Figure 2.3, which shows the speed of convergence of the pruned model compared to from scratch training. Hence, pruning larger models then continued pre-training can be a good approach for creating small models than training them from scratch. Although the resulting models do not retain the full performance of LLaMA2-7B, they are competitive with models of similar sizes trained on much larger datasets. The Minitron approach similarly combines structured pruning with knowledge-distillation-based retraining [98, 124]. It removes complete layers or reduces the hidden, attention, and MLP dimensions of a pretrained model. The pruned model is then retrained on much more data than in Sheared-Llama, over 380 billion tokens for the pruning of Mistral NeMo 12B.

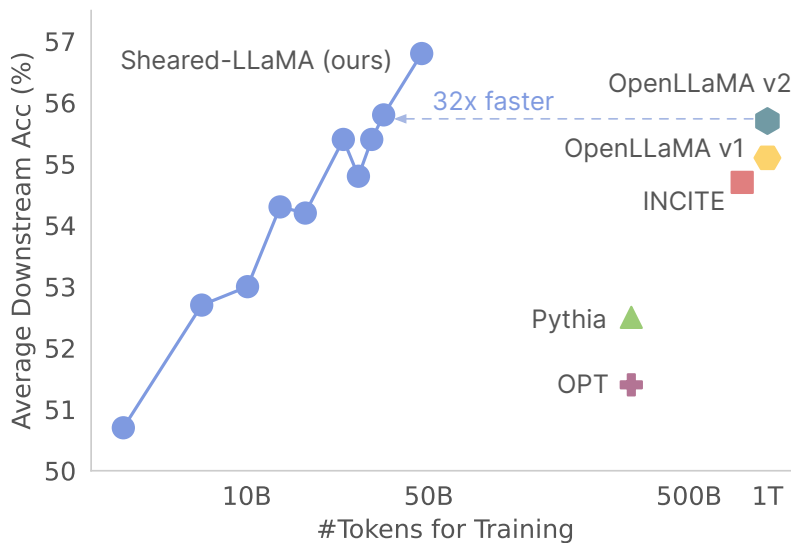


Figure 2.3: Convergence of Sheared LLaMA after structured pruning and continued pre-training, compared to other models of similar sizes. Plot taken from the original paper [147]

Some LLM releases have also experimented with pruning to produce smaller models from larger ones. For example, the Llama 3.2 1B and 3B models were obtained through one-shot structured pruning of Llama 3.1 8B.¹ The pruned models were then trained using knowledge

¹

<https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>

distillation, with the logits of Llama 3.1 8B and 70B used as training targets. On the other side, Ministral 3 [83] uses a cascaded pruning and distillation. Starting from Mistral Small 3.1 24B, the authors successively produce a smaller model at each stage which trained using the logits of the original model. This “prune–distill–repeat” procedure is similar to Minitron.

There are some approaches that focus specifically on layer pruning, based on the observation that several Transformer layers are redundant [92, 151]. For example, **ShortGPT** [92] measures the contribution of each layer using the cosine similarity between its input and output activations. The intuition is that an unimportant layer does not substantially modify its input representation. Therefore, layers whose input and output activations are highly similar can be removed. However, layer pruning incurs a big performance drop and the resulting model needs additional training. Layer pruning generally incurs a significant performance drop, and the resulting model often requires additional training to recover. It is therefore interesting to ask whether, in decoder-only LLMs, high layer similarity is a misleading signal for pruning. Information flows progressively through the network, and each layer may add only a small but important update through the residual connection. Although these updates are individually small, they accumulate across dozens of layers. Consequently, a high similarity between the input and output of a layer does not necessarily mean that the layer is useless. Removing several such layers can disrupt this accumulation and degrade performance unless a recovery stage is applied. Later evaluations of one-shot layer pruning have notably found that it can preserve performance on simple or multiple-choice benchmarks while substantially harming open-ended generation and long-chain reasoning capabilities [56, 136].²

² https://www.reddit.com/r/MachineLearning/comments/1bc1638/r_shortgpt_layers_in_large_language_models_are/

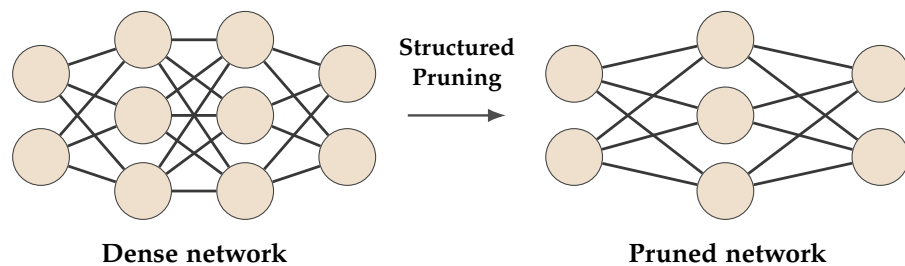


Figure 2.4: Illustration of structured pruning. A full layer is removed, together with all its incoming and outgoing weights.

2.1.5 The best training objective for compression

We mentioned distillation for compressing large models capacity in small models. What are the best training objectives for compressing LLMs and training small LLMs. The common training objective for

compression is maximum likelihood estimation, but small LLMs take time to learn. Hence, it has been proposed to use training objective that give more *learning signal* to the small LLM. Let's list them and see how they're used in recent LLM releases.

2.1.5.1 Sequence Level Distillation

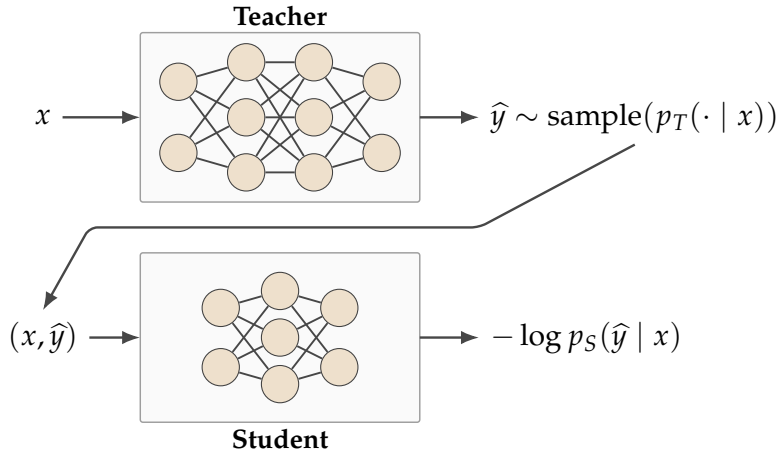


Figure 2.5: Sequence-level distillation. The teacher first generates a complete answer, and the student is then trained on this synthetic sequence.

In sequence-level distillation, the small model is trained on texts generated by a larger and more capable model. This approach is illustrated in Figure 2.5. The concept was introduced by Kim and Rush [70] for machine translation, where the translations generated by the teacher are used as targets for the student. In the context of LLMs, this approach became popular with Alpaca [131]. Alpaca trained a small LLaMA model on 52K instruction–response pairs generated by the closed-source text-davinci-003 model. However, generating useful synthetic data is not as that simple. The generated answers must be correct, diverse, sufficiently difficult, and representative of the capabilities that we want to model. The data must also be filtered to remove redundant, low-quality, or unsafe examples. This motivated the development of many methods for automating and improving synthetic-data generation. Orca [97], for example, uses GPT-4 to generate detailed explanations and reasoning traces rather than only short answers. Magpie [148], on the other hand, generates both instructions and responses from a chat LLM without requiring human-written seed instructions. Persona Hub [47] improves data diversity by conditioning the generation on a large collection of synthetic personas representing different backgrounds, occupations, and perspectives.

There has also been a debate about whether a student trained on teacher-generated data can outperform its teacher. This appears possible on specific tasks: Hsieh et al. [60], for example, show that a much smaller model can outperform a few-shot-prompted teacher by

learning from both its answers and its rationales. However, this does not mean that the student becomes generally more capable than the teacher. Such improvements often require generating multiple candidate responses and retaining only the best ones through correctness verification, rejection sampling [156], or learned quality and reward functions.

2.1.5.2 Off-policy Knowledge Distillation

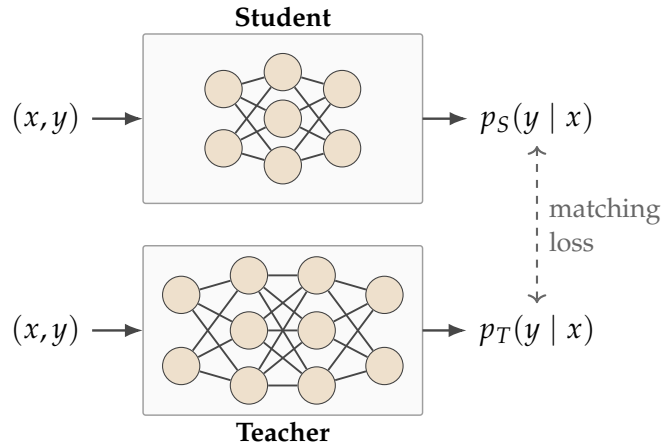


Figure 2.6: Off-policy distillation. The student and the teacher are evaluated on the same data, and the student distribution is trained to match the teacher distribution.

In sequence-level distillation, the training signal provided by each token is limited, as the student is trained only on the token sampled by the teacher. When the synthetic dataset is sufficiently large, the student can nevertheless approximate the teacher’s probability distribution underlying the generated texts. A more direct approach to mimicking the teacher’s probability distribution is **logit-based knowledge distillation**. This approach provides a richer training signal because the student learns from the teacher’s probability distribution over the vocabulary at every token position, rather than from only the generated token. This process is illustrated in Figure 2.6. The data is forwarded through both the student and the teacher, which produce a probability distribution over the vocabulary at each token position of the gold answer. The student distribution is then compared with the teacher distribution on the same data. The loss can be designed in different ways, but the most common choice is the *Kullback–Leibler divergence*, which encourages the student distribution to match the teacher distribution.

Logit-based distillation has been used to train several open-weight small language models. In particular, Gemma 2 and Gemma 3 were trained by distilling larger teacher models [48, 166]. And as discussed earlier, knowledge distillation is used as a recovery stage after pruning

in small model series such as Llama 3.2 and Ministral 3. Logit-based knowledge distillation usually outperform training on hard targets alone as the teacher’s probability distribution provides a richer training signal for every token [6, 48]. For example, the Gemma 2 ablations show that distillation improves both validation perplexity and downstream performance across several student sizes. This is particularly true in low-data regimes [89].

2.1.5.3 On-policy Knowledge Distillation

In sequence-level and off-policy knowledge distillation, the student remains passive: it is trained on sequences generated by the teacher or on fixed sequences for which the teacher provides feedback. This creates what is known as the *training–inference mismatch*. During training, the student is trained using *teacher forcing*, meaning that the prediction of the i -th token is conditioned on the previous tokens from the teacher-generated or gold sequence. At inference time, however, the student must generate its own tokens and condition subsequent predictions on its previous outputs. It may therefore encounter contexts resulting from its own mistakes that were never observed during training. **On-policy distillation** addresses this mismatch by allowing the student to generate sequences during training and using the teacher to provide feedback on the states visited by the student. Agarwal et al. [3] show that knowledge distillation is more effective when the student learns from feedback on its own generated sequences. Their method outperforms distillation based only on fixed teacher-generated data.

Figure 2.7 illustrates the process of on-policy distillation. The student takes a prompt and generates an answer, which is then evaluated by the teacher. More precisely, the teacher provides a probability distribution at each token position of the student-generated sequence. The training loss can be designed in different ways, using, for example, the forward or reverse Kullback–Leibler divergence or the Jensen–Shannon divergence. On-policy distillation can be understood through the lens of **reinforcement learning**: the student generations can be seen as actions, while the teacher provides the reward on those actions. Since the reward is differentiable, it not necessarily use policy-gradient algorithms such as REINFORCE. The student can instead be trained directly by minimizing the token-level divergence from the teacher distribution. Following studies have extended on-policy distillation beyond the compression of small models and used it as a post-training framework. Recently, **On-Policy Self-Distillation** has emerged as a new post-training approach [163]. In this setting, the student and the teacher are the same model but receive different information. The teacher is given *privileged information*, such as the gold answer or a verified reasoning trace, whereas the student receives only the original prompt. The student generates its own response, and the privileged version of the model provides dense token-level feedback on this

response. A detailed discussion of these new promising approaches goes beyond the scope of this thesis.

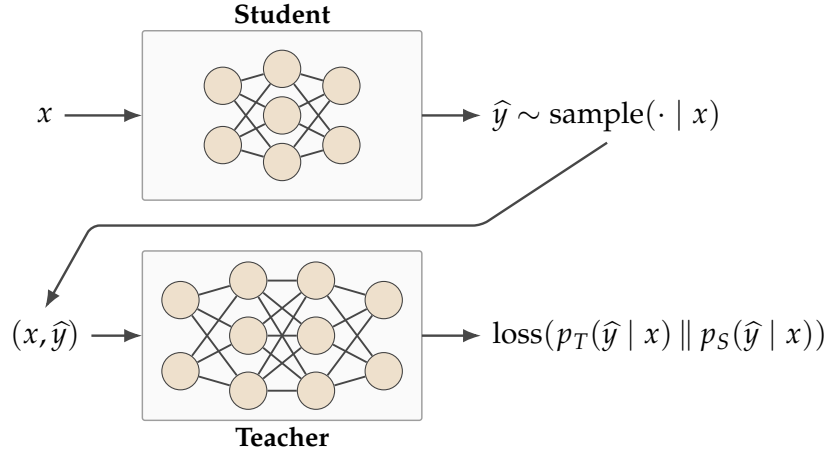


Figure 2.7: On-policy distillation. The student first samples an output, then the teacher evaluates this student-generated output.

2.1.5.4 Co-Distillation

2.2 ACTIVATION SPARSITY

While post-training compression of LLMs has proved to be an effective approach to create small LLMs, larger LLMs nevertheless remain largely more performant. They generalize better, contain more knowledge, and are much better at in-context learning. However, as discussed, they are also more costly to train and to serve for inference. Can we get some of the benefits of large models, but with a cheaper training and inference cost? Sparse activation is one answer to this question. The model dynamically selects which parameters should be used for the current input token. The selected parameters are called **active parameters**, while the others remain inactive for this specific input. When associated with efficient GPU Kernels, sparsity reduces the activation memory, so it gives more room for larger batches and hence higher training and inference **throughput**. It can also accelerate the end to end decoding speed.

Sparse activations make it possible to partly decouple model size from computation, and provide new scaling axes compared to dense models:

1. **Sparsity scaling axis:** keeping the number of active parameters per input fixed, while increasing the total number of model parameters.
2. **Computation scaling axis:** keeping the total number of parameters fixed, while increasing the computation by activating more parameters per input.

3. **Granularity scaling axis:** increasing the number of experts while keeping the number of activated parameters and the model parameters fixed.

Training sparse models is difficult because the process of selecting which parameters to activate is often discrete and therefore not directly differentiable. However, the know-how has evolved a lot, especially for sparse Mixture-of-Experts, which is now the most successful class of sparse models at scale. We will present different classes of sparse models, their pros and cons, and particularly the optimization tricks that make their training possible.

2.2.1 Sparse Mixture of Experts (MoE)

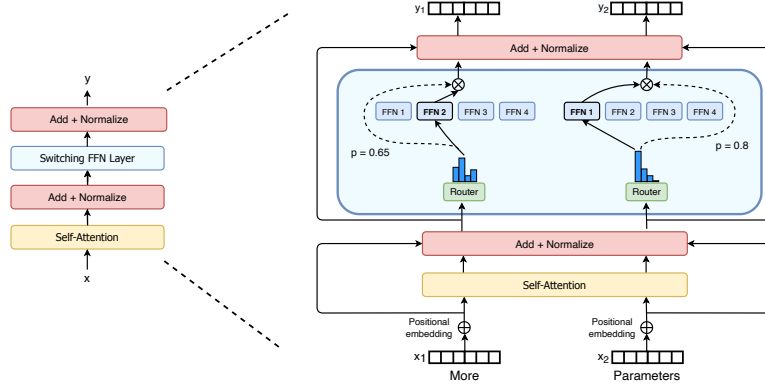


Figure 2.8: Switch Transformer encoder block, where the dense feed-forward layer is replaced by a sparse Switch FFN layer. Figure adapted from Fedus, Zoph, and Shazeer [39].

2.2.1.1 The MoE layer

In a sparse MoE, the weight matrix $\mathbf{W}$ is split into e subweights called experts, $\mathbf{W}_1, \dots, \mathbf{W}_e$. The model is trained to decide which experts are useful for the current input $\mathbf{x}$, and only computes these experts. Thus, the model can contain many parameters, while each token only activates a small fraction of them. The selection of the experts is performed by a router network R_θ and the top- k operator. More formally, let $\mathbf{x} \in \mathbb{R}^{d_1}$ be the input, and let each expert $\mathbf{W}_j \in \mathbb{R}^{d_1 \times d_e}$. The router produces one logit, or score, per expert, $\mathbf{r} = R_\theta(\mathbf{x}) \in \mathbb{R}^e$. The top- k operator is first applied on these router logits to select the subset of active experts, $S = \{j \mid r_j \in \text{TopK}(\mathbf{r}, k)\}$. Then, the routing weights are computed with a softmax only over these selected experts. Thus, for each selected expert $j \in S$, we have $g_j = \exp(r_j) / \sum_{\ell \in S} \exp(r_\ell)$.

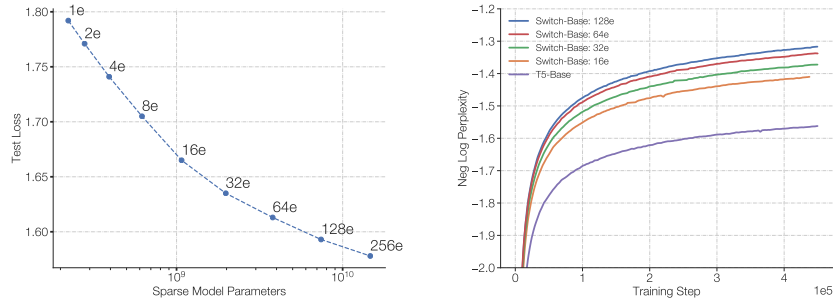
The output of a sparse MoE layer is then the weighted sum of these selected experts:

$$o = \text{MoE}(\mathbf{x}) = \sum_{j \in S} g_j \mathbf{W}_j^\top \mathbf{x}. \quad (2.1)$$

The output of the MoE layer is then projected back to the model dimension d_{model} using a projector weight $\mathbf{W}_o \in \mathbb{R}^{d_e \times d_2}$.

2.2.1.2 The nice scaling properties of MoEs

MoEs give a way to increase the capacity of the model without increasing the computation by the same factor. Switch Transformer [39] shows that at FLOP-matched budget, increasing the number of experts improves the overall performance. This means increasing the model parameters while keeping the same number of activated parameters is beneficial. Figure 2.9 illustrates this. On the left plot, we can see that keeping the number of activated parameters fixed, while increasing the number of experts, leads to strong improvement. However, it doesn't seem linear, and at some point increasing the number of parameters without increasing the number of activated parameters stop being beneficial. And since bigger models converges faster, large MoEs accelerate also the training as shown by the left plot.



(a) Validation loss as the number of parameters increases. (b) Negative log-perplexity during training.

Figure 2.9: Scaling properties of Switch Transformer. Increasing the number of experts improves performance and sample efficiency at equal computational budget. Figure from Fedus, Zoph, and Shazeer [39].

The sparsity has been pushed much farther in recent sparse MoEs. The DeepSeek series particularly pushed fine-grained experts and very sparse activation. DeepSeek-V3 uses 256 routed experts and activates only 8 of them for each token, which represents approximately 97% of sparsity [31]. DeepSeek-V4-Pro pushes this even further: 384 routed experts and activates only 6 per token, so more than 98% of the routed experts are inactive at each step [32]. Since MoEs decouple

parameters scaling from computation, recent models therefore scale to multi-trillion-parameter sparse MoEs, such as Kimi-K3 with 2.8T parameters and 104B active parameters [71], or Qwen3.8-Max with 2.4T parameters and 95B active parameters [107].

Beyond the scaling axis between active parameters and total parameters, MoEs also introduce another important axis: **granularity**, namely how many experts are available. This is different from sparsity. For example, with 50% sparsity, one can activate 1 expert out of 2, or 1024 experts out of 2048. Both designs activate the same fraction of experts, and can have the same total number of parameters, but the second design is much more fine-grained. It gives the router many more possible combinations of experts. In Krajewski et al. [74], the authors study this effect while keeping the model size and the number of training tokens fixed. as illustrated in Figure 2.10, their main result is that increasing granularity usually improves loss, probably because it gives to the router more flexibility to combine experts. The model can represent a more precise mixture of skills.

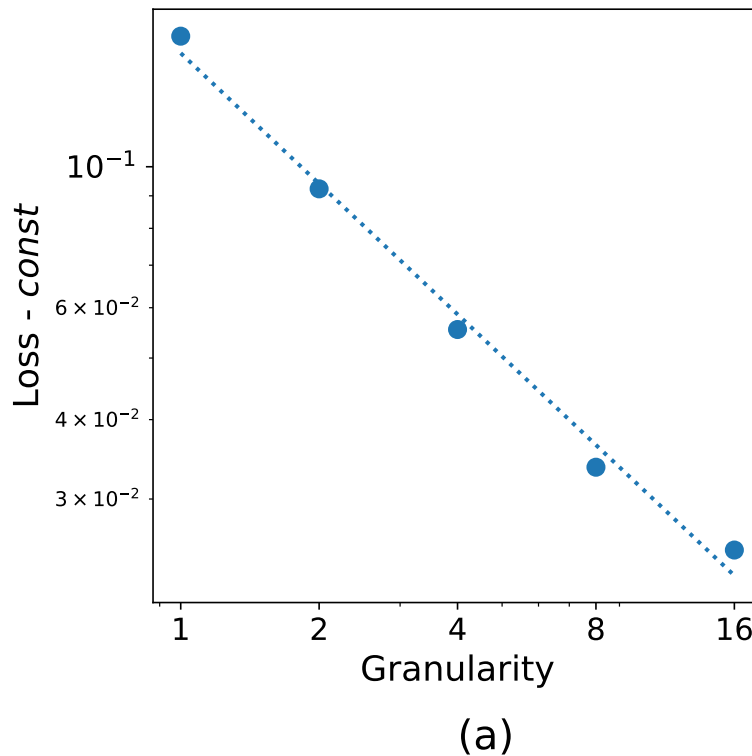


Figure 2.10: Effect of MoE granularity on loss for fixed model size and number of training tokens. Figure adapted from Krajewski et al. [74].

2.2.1.3 Stabilizing MoEs

While pushing large, sparse and granular MoEs results in strong efficient models, it also results in challenging training. This is true in

terms of infrastructure, since more experts creates a communication overhead between nodes and GPUs, and irregular GPUs memory loading when some experts are used more than other. But it is also true in terms of optimization dynamics, since the topk-based router is not a differentiable operation, it hurts the optimization dynamics and require a lot of tricks to optimize training.

LOAD BALANCING. The router must learn to distribute tokens across experts. If most tokens are routed to the same few experts, these experts become overloaded, while the other experts are almost not trained. Their hosting GPUs also receive more work than the others, which unbalances the GPU cluster. Switch Transformer avoids this by adding what is called a **load balancing** auxiliary loss, a loss added to the language modeling loss to encourage more uniform token routing at the batch level. There are several load balancing approaches. DeepSeek-V2 uses three types of auxiliary load balancing losses [30]:

- *Expert-level load balance*: encourages tokens to be distributed more uniformly across experts to avoid router collapse, so that no expert is overloaded and no expert remains undertrained.
- *Device-level load balance*: encourages the total expert load to be balanced across devices, since groups of experts are stored on different GPUs or nodes.
- *Communication balance*: avoids sending too many tokens to the same device during routing.

These are auxiliary losses added to the main task loss, which is the language modeling loss. This can hurt the optimization of the main language modeling task, because the model is partly optimized to satisfy a routing constraint instead of only predicting the next token. For this reason, recent large sparse MoEs often move toward **auxiliary-loss-free load balancing**, where balancing is not directly added to the training loss. DeepSeek-V3 uses a bias-based auxiliary-loss-free balancing strategy [31, 137]. A bias $\mathbf{b} \in \mathbb{R}^e$ is added to the router scores for the top- k selection. The final routing weights are still computed from the original router scores, without this bias. During training, the expert load is monitored at each step: if an expert is overloaded, its bias is decreased by a small constant; if it is underloaded, its bias is increased. Hence the router is progressively pushed toward balanced expert usage, but without creating additional gradients for a balancing constraint. Kimi-K3 uses a related but more direct approach called **Quantile Balancing** [71], which computes an expert-specific threshold from the quantiles of the router scores such that overused experts receive a harder threshold and underused experts receive an easier one.

WAKING THE DEAD. Even with load balancing constraints, large-scale fine-grained sparse MoEs can lead to instabilities. And some of them are silent. In Step 3.5 Flash [125], the authors report what they call **dead experts**, in comparison to the **dead neurons** phenomenon observed in ReLU-like activations. An expert is dead when it receives almost no useful training signal. In the simplest case, the router does not dispatch tokens to this expert for a long period, so the expert receives almost no gradient. But the Step 3.5 Flash report also shows a more subtle case: the router statistics can look healthy, while the expert itself is collapsing. The expert is selected, but its activations are not meaningful, and its parameter norms become stagnant or decay. A recent blogpost³ on ultra-sparse MoEs makes a similar observation: lower-layer experts can silently collapse even when training loss, validation loss and load balancing metrics look normal [66]. Their proposed solution, **LM Loss as Auxiliary Loss (LLAL)**, is to temporarily compute the language modeling based on the output of the collapsing MoE layers, so they are "forced" to become useful.

3

<https://alltoall.notion.site/save-lower-layer-moe-experts-llal>

2.2.1.4 Are "experts" really experts?

There is a debate about the possibility of experts to specialize in some domain. So far, there is no clear observed **expert specialization** in MoE LLMs, at least not in the intuitive sense of "specialization" on domains such as code, math, literature, etc. Mixtral is among the first open sparse MoE LLMs to study this question directly [65]. The authors measure how often each expert is selected on different subsets of The Pile, such as ArXiv, GitHub, PubMed, PhilPapers or Wikipedia. If experts were domain experts, we would expect some experts to be often selected for some domains. But this is not what happens. As shown in Figure 2.11, the routing to different domains is mostly uniform. However, they find that neighboring tokens in the sequence tend to be assigned to the same experts.

However, expert specialization can be more salient on surface forms, such as scripts and language families. NLLB-200, which is a sequence-to-sequence MoE machine translation model, studies this by measuring the similarity between languages according to their gating decisions [99]. As shown in Figure 2.12, the late decoder MoE layer clearly separates languages: languages from the same family tend to select similar experts. The early encoder layer also shows some language-expert specialization. This suggests that expert specialization exists, but it is not necessarily a clean semantic specialization. This finding is somehow validated in OLMoE, where the authors observe a strong vocabulary-level specialization of experts [96]. Some experts specialize in scripts or alphabets, others in punctuation, religious terms, dates, places, family words, or technical tokens. In other words, experts can specialize, but not always in high-level domains such as math or code. The specialization can also reflect scripts, language

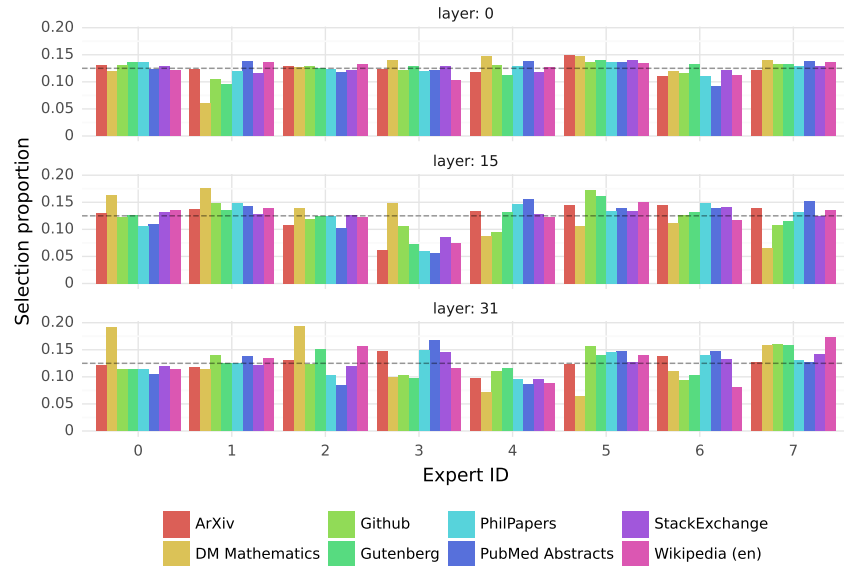


Figure 2.11: Expert selection proportions in Mixtral across domains and layers. The dashed line corresponds to uniform routing over the eight experts. Figure adapted from Jiang et al. [65].

families, token forms, and vary depending on the position of the MoE layer.

2.2.1.5 On the non-adaptivity of TopK

Liam Fedus, first author of Switch Transformer together with Noam Shazeer, reports that Shazeer once intuited: “Parameters is knowledge, FLOPs is intelligence.”⁴ In this view, increasing the number of experts while keeping the number of activated parameters fixed mainly increases the model’s knowledge capacity. It does not increase the computation spent on each token. So the gain should be weaker on tasks where the bottleneck is more compute than more memorized knowledge. The natural next step is therefore to make sparse LLMs adaptive: use more experts when the token or the task is hard, and use fewer experts when it is not.

This intuition is now well shared in the field, and many works propose ways to make MoE routing compute-adaptive⁵. Huang et al. [63] propose a routing strategy inspired by TopP decoding. Contrary to TopK, TopP does not activate a fixed number of experts. It sorts the router probabilities and keeps the smallest set of experts whose cumulative probability is above a threshold p . When the router is confident, only a few experts are used. When the distribution is flatter, more experts are activated.

Another approach is to replace the hard TopK operation with a continuous ReLU router [139]. In that case, experts with positive router scores are active, and sparsity is encouraged during training

⁴ <https://x.com/LiamFedus/status/2090363702042304847?s=20>

⁵ See this blogpost for an overview of the approaches <https://huggingface.co/blog/Spico/dynamic-routing>

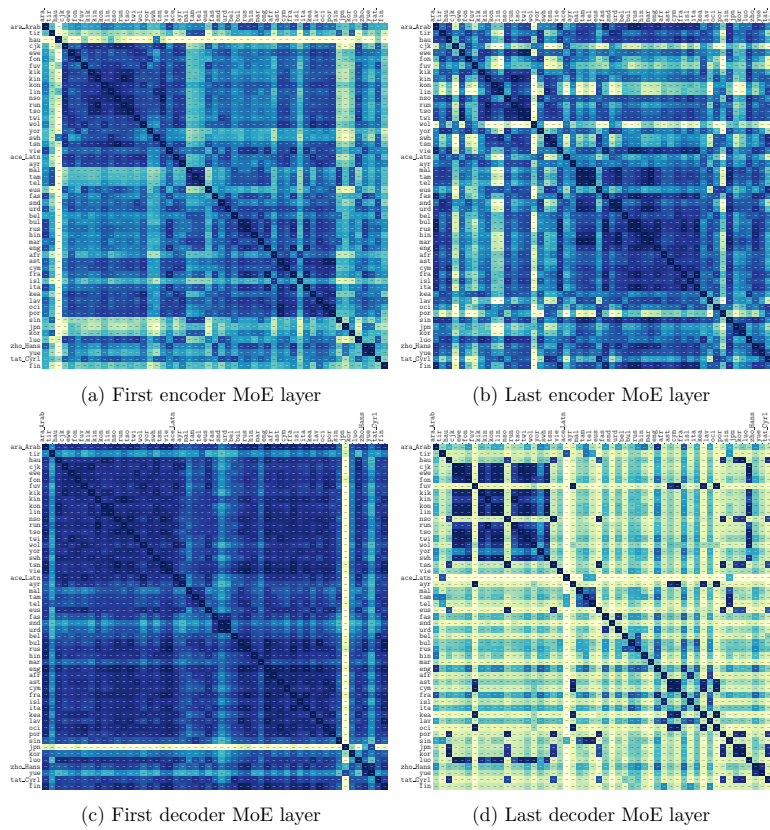


Figure 2.12: Cosine similarity between languages based on expert-choice patterns in NLLB-200. Figure adapted from NLLB Team et al. [99].

with a regularization on these scores. The number of active experts is therefore not fixed, but emerges from the input and from training.

The most commonly used adaptive sparse MoE approach at scale is probably the **zero-computation expert**. This method adds identity experts that let the input pass through without computation. This approach is for example used in LongCat-Flash [91], where they encourage a sparsity budget by selecting K experts among both FFN experts and zero-computation experts. The constraint is on the average number of real FFN experts selected by the router. The auxiliary-free bias is then adjusted during training to push this average toward the target budget. In practice, the model still routes each token to K experts, but some of these experts cost almost nothing, so the effective compute can change from token to token.

2.2.2 Sparse Memory Layers

We discussed **granularity**, and have shown that increasing the number of experts, while keeping both the total model size and the number of activated parameters fixed, can greatly improve performance (see

Expert ID	Input token IDs					Token IDs generated while the expert is active				
27	© (100%)	† (100%)	³ (100%)	í (100%)	ŀ (100%)	§ (100%)	© (100%)	Ĵ (100%)	ij	
	ĩ (100%)	◌̇ (100%)	ᵑ (100%)	क (100%)	(100%)	○ (100%)	𐌆 (100%)	र (100%)		
	Ш (100%)	ИH (100%)	а (100%)		क (100%)	◌̇ (100%)	𐌆 (100%)			
58	(“ (100%)	”((100%)	‘ (94%)	’ (92%)	“ (88%)	such (100%)	486 (100%)	see (95%)		
	(92%)	((92%)	" (90%)	' (89%)	“ (88%)	which (91%)	driving (91%)	UK (90%)		
	\$ (87%)	[(87%)	pound (86%)			who (88%)	including (88%)	normal (88%)		
7	Him (100%)	inde (100%)	Jesus (98%)			rella (100%)	Him (94%)	sin (90%)		
	God (90%)	pray (81%)	Holy (80%)			prince (80%)	glory (72%)	Jesus (69%)		
	Quran (80%)	God (77%)	Lord (76%)			Lord (68%)	Christ (65%)	Spirit (55%)		
	glory (75%)	Spirit (66%)	Christ (65%)			Holy (53%)	God (50%)	Prayer (50%)		
37	Sunday (100%)		Tuesday (100%)			days (91%)	anniversary (90%)	month		
	Thursday (100%)		Olympic (100%)			(88%) week (84%)	mpi (83%)	semester		
	Christmas (100%)		rugby (100%)			(81%) mand (80%)	Olympics (78%)			
	Championship (100%)		weekends			cent (76%)	season (76%)	perm (75%)		
3	grandmother (92%)		brother (91%)			hood (36%)	mother (35%)	inde (31%)		
	Daisy (83%)		daughter (78%)			boy (29%)	girl (28%)	married (27%)		
	(75%) father (72%)		wife (70%)			husband tri (21%)	Gab (20%)	died (18%)		
	(70%) lady (63%)		dad (62%)			boy (61%) taught (14%)	lived (13%)	knew (10%)		

Table 2.3: Vocabulary specialization in the seventh layer of OLMoE-1B-7B.
Table adapted from Muennighoff et al. [96].

Figure 2.10). The extreme case of granularity would be at the neuron level: each column of the MLP weights can be seen as a small expert, and only a few of them are activated for a given token. Doing so turns the dense MLP into a **sparse memory layer**. We mentioned earlier, in the introduction, that the MLP in a Transformer can be seen as a key-value memory, and that several empirical results support this view. If this is true, then we can also argue that not all key-value entries are needed for every input token. Some entries should be useful for one token, and irrelevant for another one. The natural idea is therefore to make the MLP sparse. While this approach is less common than MoE to build sparse Transformers, it has some interesting advantages.

The idea to augment neural networks with a large memory is not new [142], but in the context of LLMs it has been systemized by Lample et al. [77], where the approach is made efficient and scalable to modern LLM scale.

Using the notation of Lample et al. [77], a memory layer is a function $m : \mathbb{R}^d \rightarrow \mathbb{R}^n$. Given an input $x \in \mathbb{R}^d$, a query network produces a query vector $q(x) \in \mathbb{R}^{d_q}$. This query is compared to a set of trainable

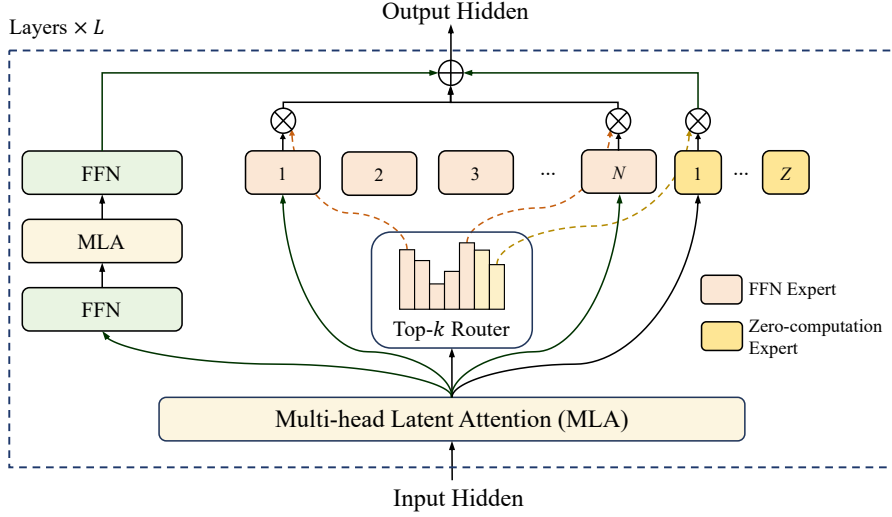


Figure 2.13: LongCat-Flash architecture with zero-computation experts. The router still selects K experts, but some selected experts are identity paths, which makes the effective compute adaptive. Figure from Meituan LongCat Team et al. [91].

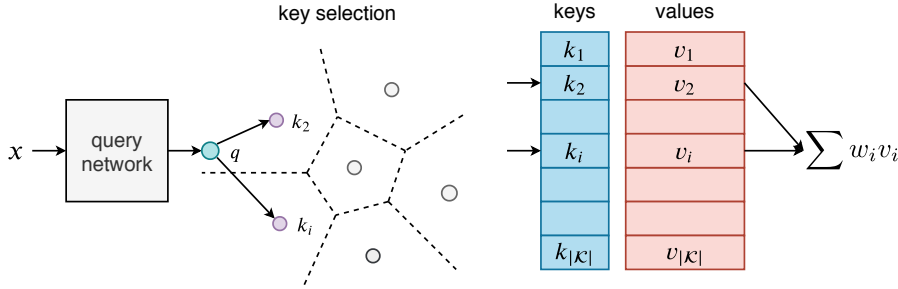


Figure 2.14: Overview of a key-value memory layer. The input is projected into a query, compared to the memory keys, and only the values associated with the selected keys are used for computation. Figure from Lample et al. [77].

keys $\mathcal{K} = \{k_1, \dots, k_{|\mathcal{K}|}\}$, with $k_i \in \mathbb{R}^{d_q}$, and each key is associated with a trainable value $v_i \in \mathbb{R}^n$. The memory does not read all values. It first selects the k keys whose inner product with the query is the largest:

$$\mathcal{I} = \mathcal{T}_k \left(\left(q(x)^\top k_i \right)_{i=1}^{|\mathcal{K}|} \right), \quad (2.2)$$

where $\mathcal{T}_k$ denotes the TopK operator. Then, the scores of the selected keys are normalized:

$$w = \text{Softmax} \left(\left(q(x)^\top k_i \right)_{i \in \mathcal{I}} \right), \quad (2.3)$$

and the output of the memory layer is the weighted sum of the selected values:

$$m(x) = \sum_{i \in \mathcal{I}} w_i v_i. \quad (2.4)$$

The query decides which memory entries are relevant for the input, and only these entries are read and updated. The issue is that, in this simple version, selecting the TopK keys requires comparing the query to all keys. This is extremely costly for modern LLM scaling. And the promise of Sparse Memory Layers is to scale the memory database to millions or even billions of entries, which is impossible to achieve in the current form. There are some efficient alternatives to this, which we will present next.

2.2.2.1 *Scaling Memory Layer*

PRODUCT-KEY MEMORY Product-Key Memory (PKM) Lample et al. [77] makes the keys search scalable by using a product structure. Instead of searching through one huge set of keys, PKM trains two smaller sets of keys, $\mathcal{K}_1$ and $\mathcal{K}_2$. A full key is not stored directly. It is formed by combining one key from the first set and one key from the second set. If $|\mathcal{K}_1| = |\mathcal{K}_2| = m$, this gives m^2 possible memory entries while storing only $2m$ key vectors. The query is also split in two, $q(x) = [q_1(x), q_2(x)]$, and the score of a composed key is computed as $q_1(x)^\top k_i^1 + q_2(x)^\top k_j^2$. So the system first finds the best keys in $\mathcal{K}_1$ and the best keys in $\mathcal{K}_2$, then combines these candidates to recover the best pairs. Each selected pair points to a value vector, and the output is still the weighted sum of the selected values, exactly as in the ordinary memory layer. In a standard memory, each memory entry has its own explicit key, and the query must be compared to all keys. The cost of retrieval therefore grows directly with the number of memory entries. In PKM, the memory is still large, but its keys are factorized. The model first searches in two small key tables, which is cheap, and only then forms a small set of candidate pairs. The expensive comparison with all possible composed keys is avoided.

ULTRA SPARSE MEMORY Ultra-Sparse Memory, or UltraMem, is a modern attempt to make PKM actually competitive at LLM scale [64] by improving several practical weak points of PKM. First, they increase the diversity of the values that can be activated. They achieve this by halving the value dimension D_v , which allows them to double the number of values while keeping the number of activated value parameters fixed. The model can access to more smaller values, and compose a richer output. Second, they observe that PKM is **memory-bound** during training. A very large sparse memory layer is cheap in FLOPS, but it can be expensive in memory access and communication. By splitting the huge memory into several smaller memories placed across the network, UltraMem avoids concentrating all memory access in a single blocking operation. The memory lookups become smaller and more distributed, making it easier to overlap them with the standard Transformer computation during training. Finally, they revisit

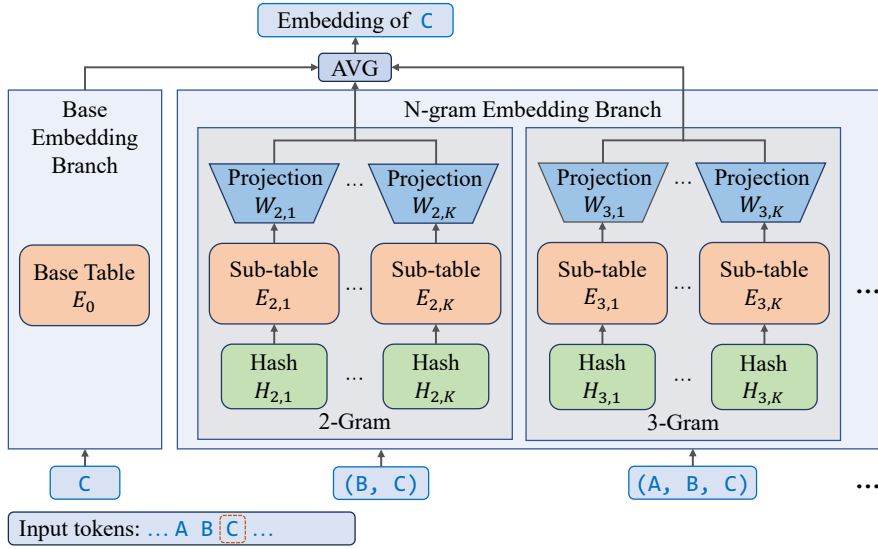


Figure 2.15: Architecture of an n -gram embedding layer. The token embedding is augmented with hashed n -gram embedding branches. Figure from Liu et al. [86].

the product-key decomposition itself with a richer approach for the query-key retrieval.

MIXTURE OF MILLION EXPERTS Another direction is to keep the product-key retrieval mechanism, but remove the key-value memory interpretation. This is the idea of PEER (Parameter Efficient Expert Retrieval), proposed by He [57]. Conceptually, PEER is similar to a very fine-grained MoE. Instead of a simple linear layer, the router uses product keys to retrieve experts efficiently, as in PKM, but each retrieved item is not a value vector stored in memory but a tiny MLP expert. Each expert is reduced to the smallest possible MLP: one up projection vector, one non-linearity, and one down projection vector. So an expert is a single hidden neuron. The layer retrieves only a few of these tiny experts and sums their outputs with router weights. The difference with PKM is therefore what is retrieved. PKM retrieves values from a key-value memory whereas PEER retrieves tiny experts. This makes the layer more naturally connected to MoE scaling and granularity: instead of having a few large experts, PEER pushes the idea to the extreme by using a large number of very small experts, while product-key retrieval keeps the routing cost cheap. Listing 2.1 gives the intuition of the forward pass, following the pseudo-code of He [57].

2.2.3 Embedding Memory Layers

Some solutions exist to augment the LLM memory without the complexity and cost of searching in a large key-value store. Embedding

Listing 2.1: Pseudo-code implementation of a PEER layer forward pass.

```

def peer_forward(self, x):
    # Embedding layers storing the down/up projection weights of
    # all experts
    self.w_down_embed = nn.Embed(num_embeddings=self.n_experts,
                                  features=self.d_model)
    self.w_up_embed = nn.Embed(num_embeddings=self.n_experts,
                                features=self.d_model)

    # Retrieve the weights of the top matching experts using
    # product keys
    # indices and scores have shape 'bthk',
    # where h is the number of heads and k the number of
    # retrieved experts
    indices, scores = self.get_indices(self.query_proj(x),
                                       self.sub_keys,
                                       top_k=self.k)

    w_down = self.w_down_embed(indices)
    w_up = self.w_up_embed(indices)

    # Compute weighted average of expert outputs
    x = jnp.einsum('btd,bthkd->bthk', x, w_down)
    x = self.activation(x)
    x = x * nn.softmax(scores)
    x = jnp.einsum('bthk,bthkd->btd', x, w_up)
    return x

```

memory layers can be seen as a special case of sparse key-value memory layers, where the keys are not learned dense vectors but fixed discrete indexes. For a standard embedding table, the key is simply the token index, and the value is the corresponding embedding vector. During the forward pass, each token x_i maps to exactly one index. In that sense, the retrieval is a TopK with $K = 1$, but without any query-key similarity to compute. Embedding memory layers are used in several recent LLMs as a simple trick to augment the model memory without increasing the forward cost too much. A simple version appears in the modded-nanoGPT speedrun as *value embeddings* where extra token embeddings are summed with the value vectors in the attention layers.⁶ More generally, the embedding table can store not only token embeddings, but also n -gram embeddings [86, 113] to augment the model memory. This approach is used and has been studied in many architectures, including LongCat-Flash [86] or Qwen3.8-Flash-Next⁷ Figure 2.15 shows how N-gram Embedding is used in LongCat-Flash-Lite. The embedding memories are used only in the input embedding module, before the Transformer layers. For each token position i , the model forms left-context n -grams ending at the current token, such as (B, C) or (A, B, C) . These n -grams are

⁶ <https://github.com/KellerJordan/modded-nanogpt>

⁷ https://github.com/QwenLM/Qwen3.8-Flash-Next/blob/main/tech_report.pdf

mapped through hash functions to embedding tables. The retrieved unigram and n-gram embeddings are projected when needed, averaged, and used as the initial hidden representation $x_i^{(0)}$ passed to the Transformer stack.

There are several ways to inject embedding-memory information into a Transformer architecture. LongCat-Flash-Lite [86] uses it to construct the input token representations to the Transformer backbone. Qwen3.8-Flash-Next instead injects the retrieved n-gram vectors inside the Transformer, at a selected layer. The authors find that no single layer position They also found that when the n-gram vocabulary is scaled by adding extra parameters, loss decreases monotonically, while downstream performance stagnates, as shown in Table 2.4.

Vocab. Scale	Loss	Knowledge			STEM	
		MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K
None	1.585	62.78	33.43	20.97	32.52	59.21
20×	1.553	64.14	34.46	21.83	37.38	65.09
50×	1.541	64.71	35.80	21.49	37.32	64.00
100×	1.534	64.70	35.87	21.93	36.98	63.08
200×	1.526	64.85	35.21	21.11	35.34	62.96

Table 2.4: Effect of n -gram vocabulary scaling in Qwen3.8-Flash-Next. Vocabulary scales are measured relative to the base tokenizer vocabulary size. Larger vocabularies add parameters. Table adapted from Qwen Team [106].

2.2.4 Post-training Activation Sparsity

We presented several approaches that induce activation sparsity during training, but sparsity can also be introduced after pre-training. These post-training activation-sparsity methods are used to reduce the inference cost of dense pre-trained LLMs.

Most modern LLMs use gated MLPs with smooth activation functions such as GELU, SiLU, or Swish, which produce few exact zeros. A natural way to recover activation sparsity is therefore to replace, after training, these smooth activations with ReLU-like activations. This process is called **ReLUfication**. In gated MLPs, this makes parts of the intermediate activation exactly zero, so the corresponding intermediate channels in the up and down projections can be skipped during inference. However, directly changing the activation function creates a distribution shift in the pre-trained model, so additional calibration or continued training is usually needed to recover performance.

Instead of replacing the MLP activation function, other approaches exploit the fact that many intermediate activations are concentrated near zero and can therefore be sparsified by magnitude thresholding.

TEAL [87] applies training-free magnitude pruning to input activations of the MLP, using layer-dependent thresholds estimated from calibration data. CATS [79] thresholds the absolute value of the gate activation $\text{SiLU}(xW_{\text{gate}})$ and sets small values to zero. The resulting mask makes it possible to skip the corresponding channels in the up and down projections during inference, improving latency. Learn-To-be-Efficient (LTE) proposes a training-aware approach to learn sparsity via sigmoid-router thresholding [164].

There are also approaches that transform dense pre-trained models into MoEs. **MoEfication** is a method for converting dense FFN layers into routed experts, while **upcycling** more generally means initializing an MoE from one or several dense checkpoints and then continuing training. For example, Qwen1.5-MoE-A2.7B is upcycled from Qwen-1.8B, and Qwen2 extends this line with a larger MoE model [105, 150]. Combining several dense models is not trivial, because their layers, notably attention layers, can differ a lot. Several studies therefore focus on how to combine different dense models into an MoE. BAM [159] proposes to reuse both FFN and attention parameters from dense models, using a mixture-of-attention variant. Nexus studies a continual domain-adaptation setting, where new domain experts are added to an existing MoE using an adaptive router initialized from domain embeddings [52].

2.2.5 Matryoshka Learning

Activation sparsity reduces the number of parameters used for each token, but it does not reduce the memory footprint of the model, since the full model parameters must be hosted in memory. Another line of approach is **Matryoshka Learning**, where a single model is trained to nest multiple model or embedding sizes within it. At inference time, it is possible to use the embedding dimension or model size that fits the compute and memory budget. The idea first emerged in representation learning as **Matryoshka Representation Learning** (MRL) [75]. Given a set of target embedding dimensions $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$, MRL trains these dimensions jointly by summing the task loss over the different representation sizes: $\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{D}} \mathcal{L}(y, f_d(x))$ where x is the full representation and f_d selects or projects it to a d -dimensional representation. Most of time, this function is simply a slicing operation that selects the first d dimensions of the representation, but it can also be a learned projection. MRL framework is illustrated in Figure 2.16 and is used in a lot in retrieval models. It helps reduce the disk usage of the embedding store, and allows a trade off between retrieval speed and accuracy. MRL can be seen as an activation sparsity since slicing a vector $x_{[:d]}$ means implicitly setting the remaining dimensions $x_{[d:]}$ to zero.

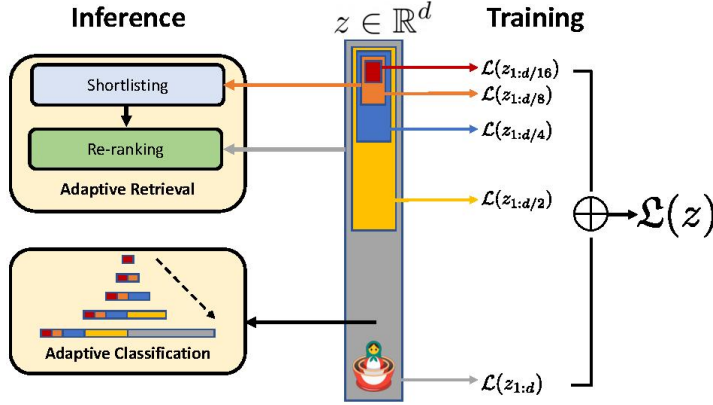


Figure 2.16: Matryoshka Representation Learning trains a representation to be useful at several embedding dimensions. Figure adapted from Kusupati et al. [75].

More recent work extends the same principle to LLM architectures. These methods train nested Transformer submodels of different sizes inside a larger model [35]. This design is used in Gemma 3n E2B and E4B⁸ and Gemma 4 E2B and E4B.⁹ At inference time, it is possible to select a submodel of the desired size depending on the compute and memory budget.

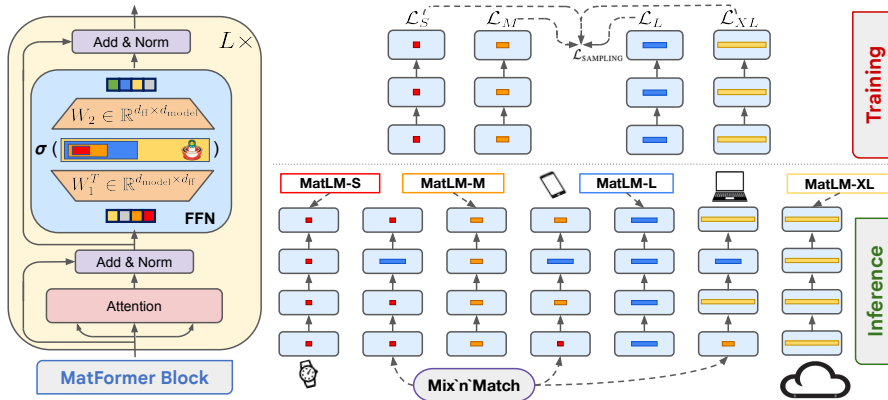


Figure 2.17: MatFormer nests several FFN sizes inside the Transformer block and trains the corresponding submodels jointly. Figure from Devvrit et al. [35].

⁸ <https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/>.

⁹ https://ai.google.dev/gemma/docs/core/model_card_4.

2.2.6 Speculative Decoding and Diffusion LLMs: Trading Compute for Memory

1. New Gemma paper: profiling of AR vs Diffusion
2. why prefer speculative decoding over diffusion LLMs: <https://x.com/barrowjoseph/status/2086933>

Part II

COMPRESSION OF TRANSFORMER MODELS

You can put some informational part preamble text here.
Illo principalmente su nos. Non message *occidental* anglo-romanian da. Debitas effortio simplicate sia se, auxiliar summarios da que, se avantiate publicationes via. Pan in terra summarios, capital interlingua se que. Al via multo esser specimen, campo responder que da. Le usate medical addresses pro, europa origine sanctificate nos se.

3.1 INTRODUCTION

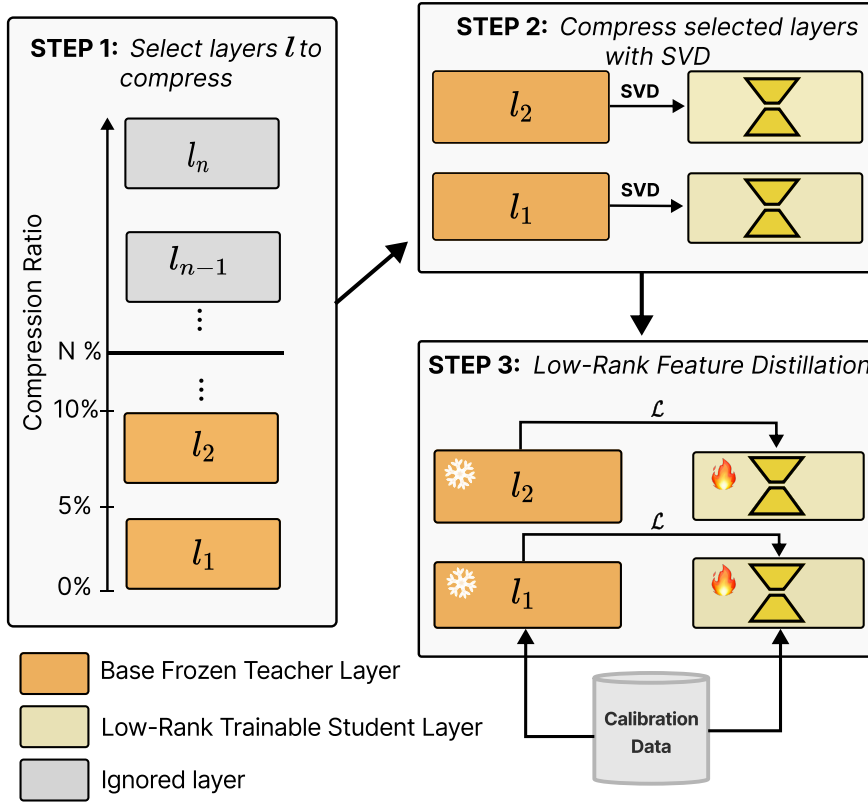


Figure 3.1: **Lillama approach**: **STEP 1** selects layers to compress for a target compression ratio (e.g., N%) using various strategies (see Section 3.5). **STEP 2** compresses and initializes the chosen parameters via SVD. **STEP 3** distills the low-rank weights with a small calibration dataset.

As discussed earlier, compressing Large Language Models is a crucial challenge that can be achieved with several complementary approaches: quantization, pruning, and distillation. In this section, we introduce a new one-shot compression method that locally distills low-rank weights and satisfies the three objectives:

1. The compression algorithm must be compute-efficient, e.g., it should require minimal computational resources and run in a reasonable amount of time. For instance, the iterative magnitude pruning algorithm as done in Frankle and Carbin [40], is too costly to be applied to large-scale LLMs.

Method	No Custom Kernel	No Ampere-only	Memory Gain	Is One-Shot	Calib. Data
SparseGPT [42]	✗	✗	✗	✓	-
Wanda [127]	✗	✗	✗	✓	-
Teal [87]	✗	✗	✗	✓	-
ShearedLLaMA [147]	✓	✓	✓	✗	52B
Minitron [123]	✓	✓	✓	✗	94B
Lillama	✓	✓	✓	✓	0.013B

Table 3.1: Positioning of our approach within the compression literature. **No Custom Kernel** indicates that a custom GPU kernel is not required to observe speedup and **No Ampere-only** is whether the method requires Ampere GPU to achieve speedup. **Calib. Data** is the total number of tokens in billions used for compression.

2. It must achieve fast convergence using as little training data as possible; this objective is often an issue with many model pruning methods that often require significant retraining after pruning to recover the lost performance.
3. The method should not cause a significant drop in performance; ideally, all the capabilities of the original LLM should remain intact.

To address the first constraint, we compress the model with a local distillation objective, as opposed to a global objective that is more costly. We further reduce the cost by compressing and distilling only a subset of all layers. About the second constraint, we initialize the compressed layers with Singular Value Decomposition (SVD), which enables reducing the required number of gradient steps as well as the calibration dataset size. We further improve convergence by combining Teacher and Student activations with a joint distillation loss. We experimentally show the robustness of our approach by compressing several state-of-the-art small and large Transformers, Mixture-of-Experts, and Mamba LLMs. Figure 3.1 gives an intuitive overview of the proposed method, which is detailed in Section 3.5 and validated in Section 3.6.

Our approach proposes to overcome some limitations of the compression approaches presented in Section 2.1. Compared to unstructured pruning and semi-structured pruning, which result in models with sparse weights and do not achieve speedup without specially designed hardware-specific kernels, our approach, which shrinks the LLM weights, saves memory across all hardware and leads to speedup, as fewer computations are performed. Structured pruning approaches can be costly: for example, the pruning step of Sheared-LLaMA Xia et al. [146] is five times as expensive as standard LM training, accord-

ing to the authors. In contrast, thanks to local gradient updates, our approach is computationally efficient, allowing us to compress a 47B model within minutes on a single A100 GPU.

There are also some low-rank decomposition approaches that compress models by approximating the pretrained matrices with lower-dimensional ones. Based on the observation by Li et al. [80] that neural networks have lower intrinsic dimensions, Aghajanyan, Zettlemoyer, and Gupta [4] show that Transformer language models also require lower intrinsic dimensions depending on the language task. However, Yu and Wu [154] and Chen et al. [20] show that the weights in some pretrained Transformer models are full-rank, unlike their activations, and propose activation-aware compression methods. Our work is related to these approaches, as we also decompose the matrices of the models. However, we apply a lightweight layer-wise feature distillation objective to better recover from compression and propose different distillation strategies to accelerate convergence and improve performance.

Standard knowledge distillation is data-intensive when the student model’s parameters are randomly initialized. Hence, this process is as costly as a standard pretraining stage. Layer-wise distillation can be more suitable in some cases, as intermediate layers may contain certain information, such as speaker identity or phonemes in pretrained speech models Baevski et al. [10], Pasad, Chou, and Livescu [103], Chang, Yang, and Lee [18], Chang et al. [19]. We leverage the observation that layer activations are low-rank and approximate them with fewer parameters by distilling the Transformer layer activations into low-rank weights initialized from the SVD of the base weights. This layer-wise distillation approach enables us to perform local gradient updates, hence making the distillation process cheaper.

3.2 MOTIVATION: ACTIVATIONS ARE MORE LOW-RANK THAN WEIGHTS

In order to motivate the compression of the the models weights with SVD, the weights should have a low low-rank structure. However, previous studies Chen et al. [20], Yu and Wu [154] have shown that the weights of pretrained Transformer models are often full-rank, while their activations are more low-rank. We empirically confirm this observation on three recent models: Falcon-Mamba, Mistral-v0.1 7B, and Phi-2 3B. There are different measure for rank. In this study, we use a simpler one: stable rank (*srank*) as a proxy measure for rank. The stable rank of a matrix is defined as the following ratio:

$$\text{srank}(X) = \frac{\sum_{i=1}^r \sigma_i^2}{\sigma_1^2} \quad (3.1)$$

with $X \in \mathbb{R}^{d_1 \times d_2}$ being the input matrix (activations or weights), $r = \min(d_1, d_2)$, and $\sigma_1 \geq \dots \geq \sigma_r$ the singular values of X .

Figure 3.2 shows the stable rank of various weight matrices and layer activations for three models: Falcon-Mamba, Mistral-v0.1 7B, and Phi-2 3B. For all models, activations are significantly lower-rank than weight matrices, which confirms previously published results by Yu and Wu [154].

This does not only motivate the use of activations for compression, but also explains why applying SVD to weights alone does not work well as we will show in the experiments. How to explain the fact that weights are full-rank while activations are low-rank? This is an open question. We know that modern LLMs are pre-trained on large and diverse datasets from the web that represent lot of knowledge. Hence the weights of the model need to be full-rank to capture all this knowledge. However, for a given input of a specific task, only a subset of this knowledge is needed, which may explain the low-rank structure of activations.

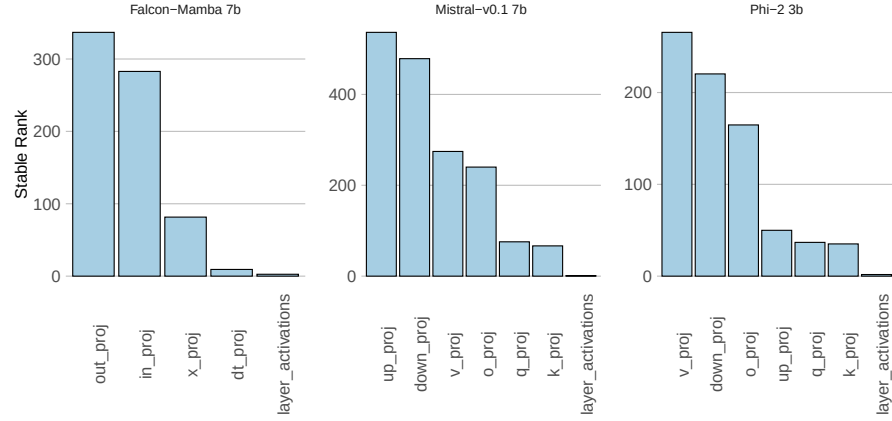


Figure 3.2: **Activations are low-rank.** Comparison of the stable rank of weights (*_proj) and layer activations (layer_activations). Each bar is the average stable rank across all layers.

Observation 1: Transformer layer activations are extremely low-rank

Transformer layer activations have extremely low stable rank compared to the weights. This extends the previous observations discussed above, which focused on linear activations, to the outputs of entire Transformer layers. Interestingly, the key projection weights also appear to have low stable rank.

3.3 BACKGROUND: LOW-RANK APPROXIMATION OF TRANSFORMER MODELS

Before presenting our approach, we first introduce the two methods for low-rank approximation of Transformer models: low-rank approximation from weights and from features.

3.3.1 Low-Rank Approximation from Weights

Given $W \in \mathbb{R}^{d_1 \times d_2}$, a pretrained weight matrix, and an example $x \in \mathbb{R}^{d_2}$, the activations $y \in \mathbb{R}^{d_1}$ are computed as $y = Wx$. It is possible to reduce the computations in this operation by using a low-rank approximation of W : $y \simeq \Delta W x = ABx$ with $\Delta W = AB$ the low-rank decomposition of W , composed of $A \in \mathbb{R}^{d_1 \times r}$ and $B \in \mathbb{R}^{r \times d_2}$. When the rank r is small enough, the number of parameters $r(d_1 + d_2)$ in the low-rank equation is smaller than $d_1 d_2$ in the full-rank equation. Estimating the low-rank matrix $\Delta W = AB$ can be formulated as a minimization problem to find the low-rank ΔW that best approximates W :

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \|W - \Delta W\|_F \quad (3.2)$$

It is well-known that this minimization problem can be approached using SVD, a low-rank approximation method that offers the optimal r -rank approximation of a given matrix with regard to the Frobenius norm $\|\cdot\|_F$. SVD approximates a matrix W into three matrices: $W = USV^T$, where $S \in \mathbb{R}^{d_1 \times d_2}$ is a diagonal matrix containing the singular values of W sorted in descending order, $U \in \mathbb{R}^{d_1 \times d_1}$ and $V \in \mathbb{R}^{d_2 \times d_2}$ are orthonormal matrices. The optimal low-rank approximation of $W \in \mathbb{R}^{d_1 \times d_2}$ can be obtained by keeping the first r singular values, with $r < \min(d_1, d_2)$:

$$\widehat{\Delta W} = (U_{:,r} S_{:,r,r}) V_{:,r} = AB \quad (3.3)$$

with $A = U_{:,r} S_{:,r,r}$ and $B = V_{:,r}$ and the notation $_{:,a:b}$ refers to the slicing operation.

3.3.2 Low-Rank Approximation from Feature

As discussed previously, many studies Chen et al. [20], Yu and Wu [154] have shown that the activations (i.e., features) of pretrained transformers are more low-rank than the weights. Recently, Liu et al. [87] show that transformer activations can be sparsified up to 60%. Hence the motivation to use the activations of the linear layers to estimate the low-rank matrices.

Let $\mathcal{D} = \{x_i \in \mathbb{R}^{d_2}\}_{1 \leq i \leq N}$ be a calibration dataset, and $W \in \mathbb{R}^{d_1 \times d_2}$ the parameters of a linear layer. Finding the low-rank matrix ΔW that best reproduces the activations $\{y = Wx\} \quad \forall x \in \mathcal{D}$ involves solving:

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \quad \frac{1}{N} \sum_{x \in \mathcal{D}} \|Wx - \Delta Wx\|_F \quad (3.4)$$

An analytic solution to this minimization problem is the eigendecomposition of the covariance matrix of the activations. We first collect all activations $\mathcal{Y} = \{y = Wx\}_{x \in \mathcal{D}}$, then the covariance matrix of the activations can be estimated as $\Sigma = \mathbb{E}_{y \in \mathcal{Y}} [yy^T] - \mathbb{E}[y]\mathbb{E}[y]^T$ with

$\Sigma \in \mathbb{R}^{d_1 \times d_1}$. Since Σ is a diagonalizable matrix, we can apply its eigendecomposition: $\Sigma = USU^T$, where $U \in \mathbb{R}^{d_1 \times d_1}$ contains the eigenvectors of Σ and $S \in \mathbb{R}^{d_1 \times d_1}$ is the diagonal matrix containing the eigenvalues sorted in decreasing order. As for the SVD equation above, we can keep the eigenvectors corresponding to the r largest eigenvalues, which gives us $A = U_{:,r}$ and $B = U_{:,r}^T W$.

Compared to the low-rank matrices in the SVD equation above, A and B weights here are learned to reproduce the activations of the base weight W , thanks to the minimization objective above. In the next section, we highlight the limitations of this approach and introduce our proposed method.

3.4 PROPOSED APPROACH

We identify and handle next three potential limitations of the approach presented in Section 3.3.2.

3.4.1 Non-linear Feature Approximation.

The previous analytical solution above is limited to activations from linear layers. To generalize to non-linear modules, we first observe that this objective can be seen as a feature distillation objective that may be optimized numerically by gradient descent rather than analytically by eigendecomposition. Given the input batch $X \in \mathbb{R}^{d \times b}$ of b examples, we denote the output activations of the i^{th} Teacher module $\mathcal{T}^{(i)}$ as

$$Y^{(i)} = \mathcal{T}^{(i)}(X; \Theta^{(i)}) \quad (3.5)$$

where $\Theta^{(i)}$ are the original pretrained matrices of the i^{th} Teacher, and $Y^{(i)} \in \mathbb{R}^{d \times b}$ its output activations. Similarly, we note the output activations of the i^{th} Student module $\mathcal{S}^{(i)}$ as:

$$\hat{Y}^{(i)} = \mathcal{S}^{(i)}(X; \Delta \Theta^{(i)}) \quad (3.6)$$

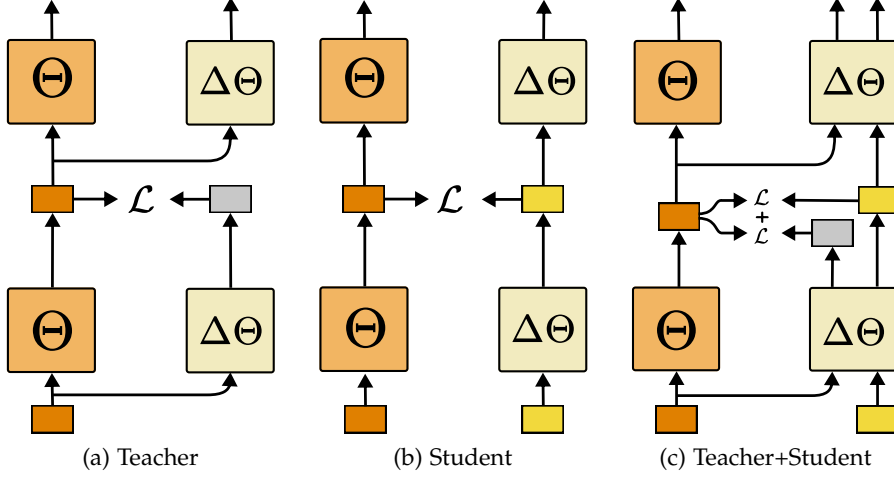


Figure 3.3: **Accelerating convergence by learning from Teacher and Student activations through a joint loss.** We compare three distillation strategies: **(a) Teacher**: the input to the compressed student layer comes from the output of the previous teacher layer; **(b) Student**: the input to the compressed student layer comes from the output of the previous student layer; **(c) Teacher+Student**: the compressed student layer is applied separately to the output of the previous teacher layer and to the output of the previous student layer, and the joint loss is the sum of the two corresponding distillation losses.

where the Student module is parametrized with the low-rank matrices $\Delta\Theta^{(i)}$ and $\hat{Y}^{(i)} \in \mathbb{R}^{d \times b}$ are the output activations. We can express the distillation objective of the i^{th} Student module as:

$$\widehat{\Delta\Theta}^{(i)} = \underset{\Delta\Theta^{(i)}}{\operatorname{argmin}} \mathcal{L}^{(i)}(Y^{(i)}, \hat{Y}^{(i)}) \quad (3.7)$$

where $\widehat{\Delta\Theta}^{(i)}$ are the estimated low-rank matrices of the i^{th} Student module and $\mathcal{L}^{(i)}(Y^{(i)}, \hat{Y}^{(i)})$ is the loss measuring the distance between the activations of the Teacher and the Student. We opted for the same loss as Chang, Yang, and Lee [18] because we observed that the loss yields instabilities:

$$\begin{aligned} \mathcal{L}^{(i)} &= \mathcal{L}_{\ell_1}^{(i)} + \mathcal{L}_{\cos}^{(i)} \\ &= \sum_{t=1}^b \left[\frac{1}{D} \left\| Y_t^{(i)} - \hat{Y}_t^{(i)} \right\|_1 - \log \sigma \left(\cos \left(Y_t^{(i)}, \hat{Y}_t^{(i)} \right) \right) \right] \end{aligned} \quad (3.8)$$

where D is the hidden vectors dimension, σ is the sigmoid activation and $\cos(\cdot, \cdot)$ is the cosine similarity. We propose to approximate the low-rank weights of the Students through Gradient Descent. In the linear case, this should converge towards the eigendecomposition solution described in Section 3.3.2. However, as we show later, we extend

the distillation process to non-linear modules, which requires numerical optimization. We also show that initializing the Student’s low-rank parameters with SVD, rather than randomly, improves convergence.

3.4.2 Beyond Teacher-only Activations.

The second potential limitation of the approach described in Section 3.3.2 and equations above is that the Student and the Teacher modules take the same input X , which is the output of the previous layer of the Teacher. This means that the Student is trained from the activations of the Teacher module only, which are not available at inference time. Yu and Wu [154] use this approach to approximate the Linear Layers of Transformer models, referring to it as Atomic Feature Mimicking. Let $\hat{Y}_T^{(i)} = \mathcal{S}^{(i)}(Y_T^{(i-1)}; \Delta\Theta^{(i)})$ be the output activations of the Student module when fed $Y_T^{(i-1)}$, the output activations of the Teacher at the previous layer $i - 1$. The loss can then be written as:

$$\mathcal{L}_T^{(i)} = \mathcal{L}^{(i)}(Y_T^{(i)}, \hat{Y}_T^{(i)}) \quad (3.9)$$

where $Y_T^{(i)}$ are the gold activations of the current Teacher. This approach can lead to a fast convergence as the Student modules learn from gold and high-quality activations produced by the Teacher modules. However, it can cause a performance drop during inference as the Teacher activations are not available. An alternative approach is to have the Student module take as input the activations of previous Student modules rather than those of Teacher modules:

$$\mathcal{L}_S^{(i)} = \mathcal{L}^{(i)}(Y_T^{(i)}, \hat{Y}_S^{(i)}) \quad (3.10)$$

where $\hat{Y}_S^{(i)} = \mathcal{S}^{(i)}(\hat{Y}_S^{(i-1)}; \Delta\Theta^{(i)})$ is the output activations of the current Student module when taking as input the output activations $\hat{Y}_S^{(i-1)}$ of the Student module at the previous layer, and $Y_T^{(i)}$ are the gold activations of the corresponding Teacher module. However, although we found this loss performs generally better than the previous one, it leads to a slow convergence due to the errors in the activations of the Student modules. To address both issues, we propose to combine the two losses:

$$\mathcal{L}_{T+S}^{(i)} = \mathcal{L}_T^{(i)} + \mathcal{L}_S^{(i)} \quad (3.11)$$

This distillation approach is illustrated in Figure 2c. We don’t introduce any hyperparameter for controlling the importance of one loss over another in the joint loss. We leave this for future work.

3.4.3 Module-wise Distillation.

Finally, we observe that the functions can be any other module in the neural network than linear layers: any sub-part of the model that

outputs low-rank activations can be compressed with our method. In our experiments, we distill at the Transformer layer level and leave comparisons with other distillation levels for future work. We also experiment with other models than pretrained Transformer-based language models.

3.5 FAST AND MEMORY-EFFICIENT COMPRESSION

Computing the optimal compression rank of each matrix individually may be costly: while search algorithms can be a solution for smaller models, they can be difficult to scale to LLMs with billions of parameters. Assuming a target compression rate N , we consider three simple strategies: **uniform**, **top**, and **bottom**-first compression. Figure 3.4 summarizes the strengths and weaknesses of each strategy.

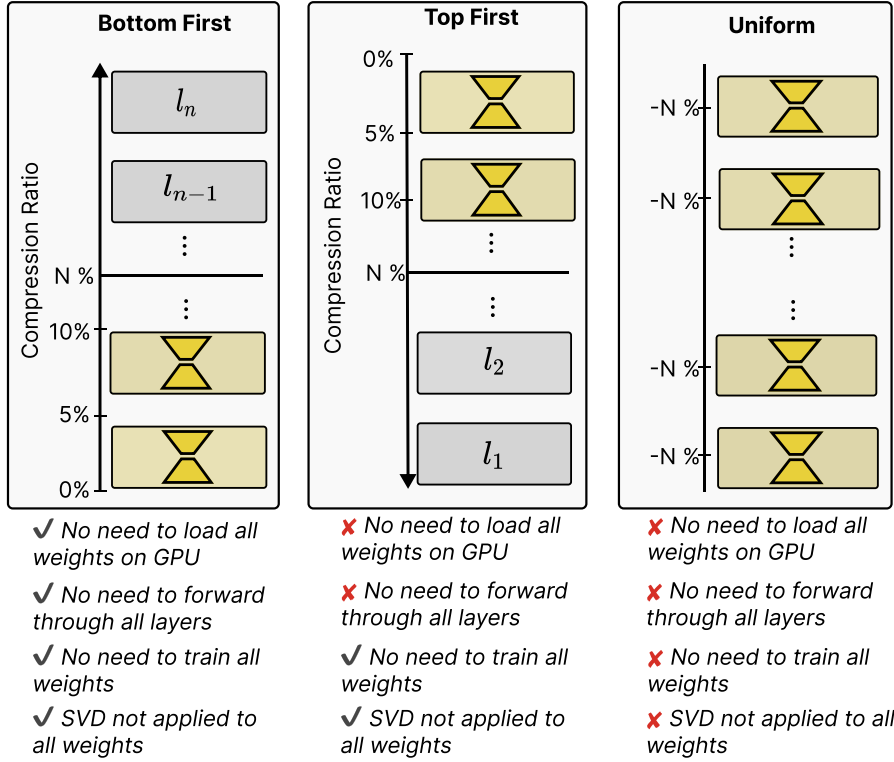


Figure 3.4: Illustration of 20% compression when using the three strategies: bottom, top, and uniform. Green boxes are compressed layers with low-rank matrices while gray boxes are non-compressed ones. $l_1, l_2, \dots, l_{n-1}, l_n$ are the layer indexes.

3.5.1 Uniform Weights Compression

This approach removes $N\%$ parameters from each weight matrix in the model. This approach doesn't make any assumption about the importance of different layers but is also costly because it requires

computing the low-rank approximation of all weight matrices. It also requires to train all the model's layers, hence requires more GPU memory and computation as it requires forwarding and backwarding through all layers of the model.

3.5.2 Top Layers First Compression

This approach removes $N\%$ parameters from the model by prioritizing the top layers. This is more efficient than the uniform one as it requires computing the low-rank approximation of fewer weight matrices and training fewer layers. However, it still requires loading the full model in GPU memory and forwarding through all the layers although only a subset of them are compressed and trained.

3.5.3 Bottom Layers First Compression

This approach removes $N\%$ parameters from the model by prioritizing the bottom layers. Compared to the previous two approaches, this one is more efficient as it requires computing the low-rank approximation of fewer weight matrices and training fewer layers. It also allows to load only a subset of the model in GPU memory and forward through only a subset of the first compressed layers, which is not the case for the previous two approaches. As we will show it, this approach can compress big models that could not fit into GPU memory. The Algorithm 1 describes the bottom-first compression approach.

Algorithm 1: Bottom Layers First Compression Algorithm

Input: $\mathcal{M}$ is the base model; S is the target size of the compressed model; k is the minimum possible rank to set; m the increment when generating ranks

Result: Low-rank compressed $\mathcal{M}$

```

1  $\mathcal{R} \leftarrow \emptyset;$ 
2 for each weight matrix  $W \in \mathbb{R}^{d_1 \times d_2}$  in  $\mathcal{M}$  do
3    $b \leftarrow \min(d_1, d_2);$ 
4   for  $r = k; r \leq b; r = r + m$  do
5     if  $r \times (d_1 + d_2) < d_1 \times d_2$  then
6        $\mathcal{R} \leftarrow \mathcal{R} \cup \{(r, W)\};$ 
7 sort  $\mathcal{R}$  by layer index in increasing order, then by rank in decreasing order;
8 while  $|\mathcal{M}| > S$  do
9   get the next  $(r, W)$  from  $\mathcal{R};$ 
10  compute  $A, B$  from  $W$  using Eq. 3.3 with rank  $r;$ 
11  replace  $W$  by  $A, B$  in  $\mathcal{M};$ 
12 return  $\mathcal{M};$ 

```

3.5.4 Ease of Implementation of the approach

The approach is straightforward to implement. Figure 3.5 shows a simple pseudo-code implementation that can be easily integrated into most recent PyTorch models.

```
def low_rank_layer(layer, ranks):
    for name, module in layer.named_modules():
        if isinstance(module, torch.nn.Linear):
            # get the rank of this matrix, as computed with Algorithm 1
            r = ranks[name]
            # create the Up-Down linears for the low-rank matrices
            l1 = torch.nn.Linear(module.in_features, r, bias=False)
            l2 = torch.nn.Linear(r, module.out_features, bias=False)
            # initialize the Low-Rank matrices with SVD
            u, s, v = torch.linalg.svd(module.weight)
            w1 = torch.diag(s)[:r, :r] @ v[:r, :]
            w2 = u[:, :r]
            l1.weight = torch.nn.Parameter(w1)
            l2.weight = torch.nn.Parameter(w2)
            # replace the old linear layers with the low-rank linear layer
            low_rank_linear = torch.nn.Sequential(l1, l2)
            setattr(layer, name, low_rank_linear)

# 'ranks' are the rank values computed using Algorithm 1
# 'base_layer' is the uncompressed pretrained layer.
compressed_layer = low_rank_layer(copy(base_layer), ranks)
optimizer = torch.optim.AdamW(compressed_layer.parameters())

def feature_approximation_hook(module, inputs, outputs):
    optimizer.zero_grad()
    outputs_ = compressed_layer(inputs)
    loss = loss_fn(outputs_, outputs)
    loss.backward()
    optimizer.step()

# attach the function to the base layer as a forward hook
# so the function is called at each forward call of the base layer
base_layer.register_forward_hook(feature_approximation_hook)
```

Figure 3.5: Example of PyTorch implementation of our approach using the Teacher-only distillation strategy.

3.6 EXPERIMENTS ON TRANSFORMERS

3.6.1 Setup

3.6.1.1 Models.

We evaluate our method on several Transformer-based LLMs: a 47B Mixture-of-Experts model (Mixtral-8x7B-v0.1), medium-sized LLMs (Mistral-v0.1 7B and Phi-3 14B), and a smaller model (Phi-2 3B).

3.6.1.2 Data.

For calibration, we use 13 million tokens randomly sampled from Slim-Orca Lian et al. [81], an open-source replication of Orca Mukherjee et al. [97]. In preliminary experiments, we also tested other datasets

such as RedPajama Computer [25], but observed better results with instruction data.

3.6.1.3 Implementation Details.

We implement our approach using version 4.44.2 of the Huggingface Transformers library [143] and PyTorch version 2.4.0+cu121. For all experiments, we use the AdamW optimizer with a learning rate of $8.6e^{-4}$. Note that since our approach is local, each student layer has its own optimizer. This also allows us to perform local gradient updates and avoid storing the entire large PyTorch computation graph in memory. All experiments were conducted on a single A100 GPU, regardless of the model.

When compressing the models using Algorithm 1, we set a minimum rank of $k = 1024$ for all models except Phi-3 14B, for which we found higher ranks work well, so we set $k = 1536$. For all models, we set $m = 256$ for the increment to generate ranks.

We apply the Teacher+Student loss, as illustrated in Figure 2c, to all models, as we found that this loss converges faster and generally performs better. Additionally, we use the bottom-first compression strategy, which proved to be effective and less memory-intensive than other strategies.

3.6.1.4 Evaluation.

To measure inference speed, we measure the *prefill speed*, which is the time to forward a batch of prompts through the entire model and present the results in tokens per second. Using lm-evaluation-harness Gao et al. [44], we evaluate zero-shot performance on 9 downstream tasks: TruthfulQA (TruQA) Lin, Hilton, and Evans [82], Social IQa (SIQA) Sap et al. [115], LogiQA Liu et al. [88], WinoGrande (WinoG) Sakaguchi et al. [114], ARC Easy (ARC-E) Clark et al. [24], ARC Challenge (ARC-C) Clark et al. [24], BoolQ Clark et al. [23], PIQA Bisk et al. [13], and OpenBookQA (OBQA) Mihaylov et al. [95]. We compare non-compressed (0%) and compressed variants. We provide in Table 3.2 the metrics we used to evaluate the models in Section 3.6.

3.6.2 Results

ZERO-SHOT EVALUATION UNDER DIFFERENT COMPRESSION RATIOS. Table 3.3 reports zero-shot results for base models and 20%, 25%, and 30% compressed models, showing that the method remains robust as compression increases.

LOW-RANK MODELS ARE LIGHTER AND FASTER. Table 3.4 shows memory gain (VRAM) and speedup of compressed models. We can see that 20% compressed models are lighter and up to 20% faster.

Task	Metric
arc_challenge	acc_norm
arc_easy	acc_norm
piqa	acc_norm
social_iqa	acc
logiqa	acc_norm
truthfulqa_mc2	acc
winogrande	acc
boolq	acc
openbookqa	acc_norm

Table 3.2: Benchmarks and corresponding metrics that were used to evaluate the models.

OUR APPROACH CAN EFFICIENTLY CREATE SMALL LANGUAGE MODELS. We compressed Phi-2 3B to 1.7B (40% parameter reduction) and compare its performance to recent models of similar size in Table 3.5.

COMPRESSED MODELS RECOVER WELL WITH FINE-TUNING. Table 3.6 shows that fine-tuning these compressed models leads to performance recovery.

Observation 2: Low-rank compressed LLMs can continue to improve

As shown by the Mistral fine-tuning results in Table 3.6, the bottleneck representation created by the low-rank structure does not seem to prevent the model from improving. With longer fine-tuning, it could potentially recover most of the lost performance.

3.7 OTHER ARCHITECTURES AND MODALITIES

In this section, we show the generalizability of our approach by evaluating it on the Mamba architecture for text Gu and Dao [53] and on the speech modality with Whisper Radford et al. [109]. We further evaluate it on a lower-resource African language in the appendix.

3.7.1 Mamba Architecture

The attention mechanism in the Transformer is limited by its increasing complexity as a function of the input sequence length. Therefore, Linear Attention Models have been proposed as alternatives to

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Phi-3 14B	0%	57.63	57.06	37.94	76.01	81.36	61.60	88.56	81.34	50.40	65.77
	20%	57.88	50.26	34.10	72.53	83.54	59.22	85.32	80.30	49.20	63.59
	25%	56.37	51.38	32.26	70.56	80.98	57.51	83.88	79.76	47.00	62.19
	30%	55.32	50.41	31.18	70.32	78.70	55.55	84.10	76.99	45.00	60.98
Phi-2 3B	0%	44.40	55.42	30.57	76.16	78.16	54.18	83.21	79.11	51.20	61.38
	20%	46.04	52.56	29.80	71.51	72.81	46.50	75.05	76.55	45.60	57.38
	25%	44.55	51.64	30.88	71.35	71.93	45.99	79.79	74.43	43.20	57.08
	30%	44.12	51.59	30.41	70.72	69.99	44.37	78.17	74.54	43.20	56.35
Mistral-v0.1 7B	0%	42.60	46.57	29.80	73.80	79.55	53.92	83.70	82.10	44.00	59.56
	20%	44.14	45.50	28.26	66.69	73.11	46.33	77.00	77.37	41.20	55.51
	25%	40.52	44.93	30.11	68.11	65.07	40.61	78.44	73.99	40.20	53.55
	30%	40.08	44.37	30.57	66.93	65.53	40.61	77.98	73.18	39.00	53.14

Table 3.3: **Zero-shot evaluation results** for the base non-compressed models (0%), and the 20%, 25%, and 30% compressed models.

Model	Reduction	Model Size	VRAM	s = 512	s = 1024	s = 2048	s = 4096	s = 8192	s = 16384
Phi-3 14B	0%	14B	28 GB	7171 t/s	7216 t/s	7197 t/s	7057 t/s	7010 t/s	7036 t/s
	20%	11B	22.8 GB	8622 t/s	8686 t/s	8509 t/s	8349 t/s	8268 t/s	8269 t/s
Mixtral-8x7B-v0.1 47B	0%	47B	OOM	OOM	OOM	OOM	OOM	OOM	OOM
	20%	37B	73.8 GB	6515 t/s	8143 t/s	8444 t/s	OOM	OOM	OOM
Phi-2 3B	0%	2.8B	6.8 GB	29949 t/s	30895 t/s	30184 t/s	30403 t/s	26636 t/s	21499 t/s
	20%	2.2B	5.7 GB	32451 t/s	34561 t/s	34276 t/s	32479 t/s	30351 t/s	23735 t/s
Mistral-v0.1 7B	0%	7.2B	15.2 GB	13385 t/s	13537 t/s	13650 t/s	13265 t/s	12603 t/s	12399 t/s
	20%	5.8B	12.6 GB	15416 t/s	15832 t/s	15947 t/s	15446 t/s	14589 t/s	14304 t/s

Table 3.4: **Compressed models save memory and speedup computation.**

Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 using a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All the models were loaded in bf16 and used Flash-Attention2. OOM stands for Out-Of-Memory.

Transformers, in particular State Space Models, such as the Mamba architecture Gu and Dao [53], which have shown promising results.

METHOD. We tested our approach on two Mamba architectures: Mamba 3B Gu and Dao [53] (<https://huggingface.co/state-spaces/mamba-2.8b-hf>) and Falcon-Mamba 7B (<https://huggingface.co/tiiuae/falcon-mamba-7b>). We compressed these models by 20% using the same dataset and hyperparameters as in previous experiments with Transformers, with a minimum rank set to 1024 for both models.

RESULTS. Table 3.7 shows that, as for Transformers, Mamba-based Language Models can be efficiently compressed. Interestingly, the 20% compressed Mamba 3B maintains 99% of the performance. Table 3.8 further shows that compressed Mamba models are lighter and faster than their original versions.

Model	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
StableLM-2 1.6B	38.90	48.52	26.73	63.30	68.39	38.57	74.80	76.93	39.00	52.79
Qwen-1 1.8B	38.09	45.19	31.80	59.12	58.33	34.98	65.93	73.23	33.60	48.92
Qwen-2 1.5B	45.93	45.85	31.18	66.22	60.56	36.09	72.26	75.35	36.40	52.20
Lillama (Phi-2 1.7B)	44.08	47.80	25.65	68.27	63.22	38.82	76.02	72.36	39.20	52.82

Table 3.5: **40% compressed Phi-2 3B competes with models of similar size while being compressed using only 13M tokens.** Compressing Phi-2 3B to 1.7B (by removing 40% of the parameters) and comparing its zero-shot performance to other recent language models of similar sizes.

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Phi-2 3B	0%	44.40	55.42	30.57	76.16	78.16	54.18	83.21	79.11	51.20	61.38
	40%	44.08	47.80	25.65	68.27	63.22	38.82	76.02	72.36	39.20	52.82
	40% FT	45.43	49.49	31.80	70.09	60.86	37.29	67.68	73.23	42.00	53.10
Mistral-v0.1 7B	0%	42.60	46.57	29.80	73.80	79.55	53.92	83.70	82.10	44.00	59.56
	40%	41.69	43.65	30.57	62.35	61.66	34.56	75.23	71.49	34.80	50.63
	40% FT	41.60	52.25	29.65	66.61	66.75	39.93	79.39	74.37	38.29	54.32

Table 3.6: **Compressed models recover well when finetuned.** Zero-shot performances of the base non-compressed models (0%) and the 40% compressed models, without and with fine-tuning (FT). Models were fine-tuned on the whole Slim-Orca dataset (191 million tokens).

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Falcon-Mamba 7B	0%	53.40	52.66	31.34	74.66	81.73	58.96	83.12	81.56	48.60	62.89
	20%	52.34	50.10	30.72	66.69	76.73	49.23	77.22	78.02	44.80	58.43
Mamba 3B	0%	35.88	43.24	26.88	63.38	64.02	36.26	65.63	75.90	39.40	50.07
	20%	37.50	42.43	27.04	61.40	62.16	35.58	64.89	73.29	40.00	49.37

Table 3.7: **Mamba-based Language Models can be efficiently compressed.** Zero-shot evaluation results for the non-compressed (0%) and for the compressed Mamba-based Language Models (20%)

3.7.2 Speech modality: Whisper

We evaluate next our approach on a state-of-the-art speech model: *whisper-medium.en* Radford et al. [109], a 764M parameters encoder-decoder speech recognition model trained on English.

METHOD. We take inspiration from the deep encoder, shallow decoder approach Kasai et al. [69], where the authors observe that the decoder in encoder-decoder models does not need to be very deep to achieve good performance. Therefore, we keep the encoder unchanged and remove 70% of the decoder’s parameters. We then approximate the low-rank matrices using the same method and hyperparameters described in Section 3.6.

Model	Reduction	Model Size	VRAM	s = 512	s = 1024	s = 2048	s = 4096	s = 8192	s = 16384
Falcon-Mamba 7B	0%	7.27B	15.3 GB	8725 t/s	9346 t/s	9390 t/s	9695 t/s	9908 t/s	9486 t/s
	20%	5.82B	12.6 GB	10227 t/s	10427 t/s	10477 t/s	10814 t/s	11108 t/s	10647 t/s
Mamba 3B	0%	2.77B	6.7 GB	17994 t/s	18556 t/s	18819 t/s	19618 t/s	19231 t/s	18602 t/s
	20%	2.22B	5.6 GB	19013 t/s	19993 t/s	20378 t/s	21218 t/s	20808 t/s	20100 t/s

Table 3.8: **Compressing mamba models saves memory and speeds up computation.** Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All models were loaded in bf16 and used Flash-Attention2.

DATA. For calibration, we used 60% of randomly sampled examples from Librispeech-train-100 Panayotov et al. [101]. We also fine-tuned the compressed model on a subset of 10 hours of speech randomly sampled from Librispeech-train-100.

RESULTS. We evaluate the models’ Word Error Rate (WER), memory footprint and latency on an out-of-distribution test dataset: the test split of the English Fleurs dataset Conneau et al. [27].

Reduction	Size	WER ($\downarrow$)	VRAM	Speed ($\uparrow$)
0%	764M	25.40	3.5GB	2395t/s
37%	485M	30.84	2.4GB	2717t/s
0% FT	764M	9.20	3.5GB	2395t/s
37% FT	485M	12.14	2.4GB	2717t/s

Table 3.9: **The 37% compressed Whisper is faster, $1.46\times$ lighter.** The WER is computed on the English test subset of the Fleurs dataset. The speed (tokens/second) corresponds to the transcription of 400 tokens from a dataset containing 2,048 speech examples, using a batch size of 128. A single A100 GPU is used and the model is loaded in FP32, without Flash-Attention. We also give the GPU Memory (VRAM) occupied by each model.

Table 3.9 gives the WER for compressed and uncompressed models. We can see that a 37% compressed whisper maintains 82% of the performance.

3.8 ANALYSIS

In this section, we give some ablation studies. In all experiments, the models are compressed for 20%.

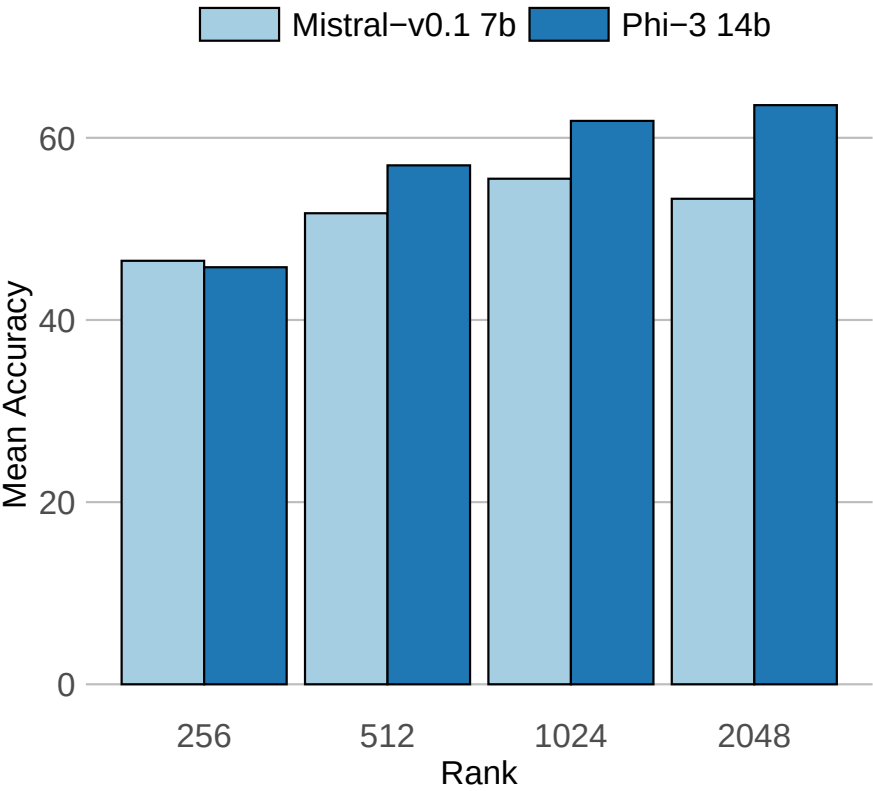


Figure 3.6: **Higher ranks give generally better results.** Mean accuracy over our test benchmarks as a function of the minimum rank k in Algorithm 1

Model	Teacher	Student	Tea+Stu
Phi-2 3B	57.38	56.44	57.38
Phi-3 14B	62.54	63.20	63.36
Mistral-v0.1 7B	51.25	55.46	55.51
Mixtral-8x7B 47B	58.87	60.49	60.19
Average	57.51	58.90	59.11

Table 3.10: **The joint loss generally performs better:** Zero-shot performances of the three losses using the same setup for each model, as described in Section 3.6.

3.8.1 Which distillation loss to use?

Table 3.10 shows that the joint loss depicted in Figure 2c generally produces the best results. Although the differences between the Student and Teacher+Student losses may not seem significant, Figure 3.8 shows that the Teacher+Student loss leads to faster convergence than the

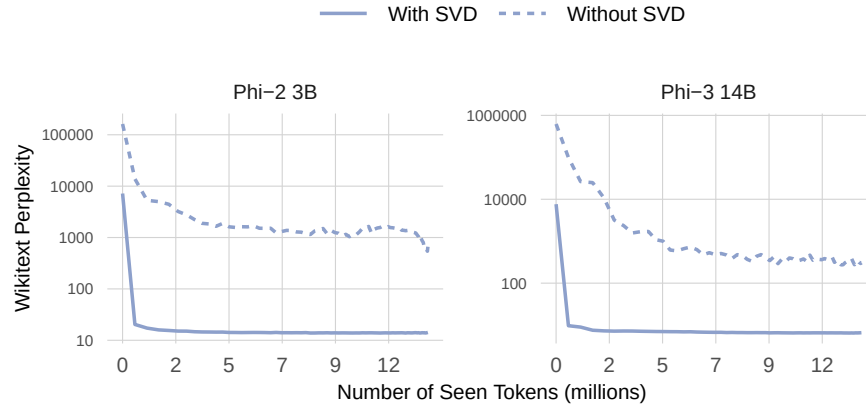


Figure 3.7: **SVD Initialization improves the sample-efficiency.** Convergence when initializing low-rank weights randomly (without SVD) or with SVD.

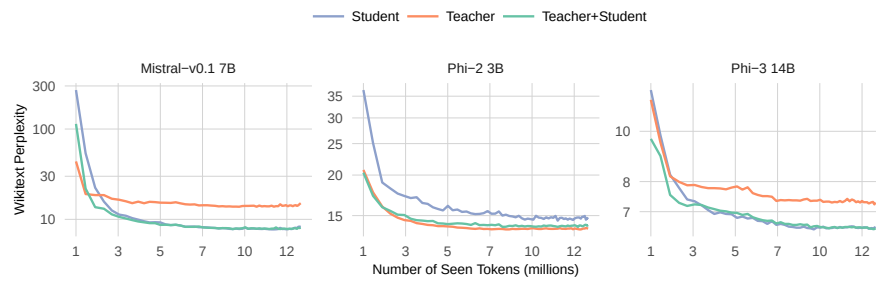


Figure 3.8: **The joint loss converges generally better.** Convergence of the three losses illustrated in Figure 3.3, evaluated on the Wikitext2 test corpus perplexity during distillation.

Student loss alone. It also generally results in a lower final perplexity than using the Teacher loss only.

Observation 3: Combining Teacher and Student activations improves average performance

Combining Teacher and Student activations in a joint loss achieves the highest average zero-shot score across the evaluated models (Table 3.10). It also converges faster than the Student-only loss and generally reaches a lower final perplexity than the Teacher-only loss (Figure 3.8).

As shown in Figure 3.8, the Teacher+Student loss combines the best of both worlds: fast convergence due to the Teacher activations and high performance at inference thanks to the Student activations. We illustrate this in Table 3.11 for the first 2M training tokens. It shows that the Teacher+Student loss converges faster than the Student-only

loss. While the Teacher-only loss also converges fast, it reaches a higher plateau and does not converge beyond that point.

Model	Seen Tokens	Teacher	Student	Tea+Stu
Phi-2 3B	0	7185.31	7185.31	7185.31
	532480	20.74	36.56	20.34
	1056768	17.69	25.11	17.26
	1559879	16.00	18.97	15.92
	2000506	15.21	18.19	15.51
Mistral-v0.1 7B	0	12365.44	12365.44	12365.44
	532480	43.79	270.16	114.02
	1056768	19.07	53.90	21.90
	1579121	18.56	22.40	13.64
	2056609	18.55	15.72	13.11
Phi-3 14B	0	46362.82	46362.82	46362.82
	532480	9.77	12.94	8.89
	1056768	8.33	8.76	7.99
	1580799	7.70	7.85	7.44
	2069488	7.61	7.77	7.34

Table 3.11: Comparing the Wikitext perplexity convergence of the three losses for 2M training tokens.

3.8.2 How to choose the rank?

In our bottom-first compression strategy, a small rank in Algorithm compresses more severely the bottom layers and leaves more upper layers uncompressed, hence saving GPU memory. However, a small may also destroy knowledge and the model will struggle to recover during the distillation phase. Figure 3.6 compares 4 rank values for Mistral-v0.1 7B and Phi-3 14B and shows that higher ranks lead to better performance, but are also more costly as more layers are distilled.

3.8.3 Is SVD initialization necessary?

Figure 3.7 shows the perplexity curves for Phi-2 3B and Phi-3 14b when the low-rank matrices are initialized with SVD or randomly. SVD initialization enables faster convergence, with 8 million tokens being sufficient, i.e., roughly 4k examples of 2048 tokens.

3.8.4 Cost.

All models in Section 3.6 are compressed in less than one hour on a single A100 GPU. The precise cost depends on the number of layers distilled, which is determined by the compression strategy (bottom-first, top-first, or uniform) and the minimum rank k in Algorithm 1.

Figure 3.4 intuitively illustrates the three compression strategies evaluated in our method: uniform, bottom-first, and top-first, explaining their respective advantages and drawbacks. We present here empirical results on differences in efficiency and performance using the same hyperparameters as in Section 3.6.

DIFFERENCES IN RUNNING TIME AND MEMORY. The bottom-first compression strategy allows compressing all models, including Mixtral-8x7B-v0.1 47B. While the top-first strategy can compress Phi-3 14B, the uniform compression strategy fails to compress Phi-3 14B and Mixtral-8x7B-v0.1 (47B). Table 3.12 gives the distillation time for each strategy and each model.

Model	Bottom	Top	Uniform
Phi-2 3B	25min	32min	50min
Mistral-v0.1 7B	26min	46min	105min
Phi-3 14B	30min	50min	OOM
Mixtral-8x7B-v0.1 47B	47min	OOM	OOM

Table 3.12: Compression times for the three compression strategies at 20% parameter reduction on a single A100 GPU. For the bottom and top-first compression strategies, we used the same hyperparameters as in Section 3.6.

Observation 4: Bottom-first compression is faster and more memory efficient

Bottom-first compression reduces computation and GPU memory requirements by processing only the required bottom layers. It is the fastest strategy across all evaluated models (Table 3.12). It is also the only strategy that compresses Mixtral 47B on a single A100 GPU, taking 47 minutes.

DIFFERENCES IN PERFORMANCES. Table 3.13 presents the mean accuracies for each compression strategy. The bottom-first compression strategy performs well and is also more memory efficient. The top-first strategy does not improve accuracy while having higher time and memory complexity compared to the bottom-first approach.

3.8.5 Examples of generated texts

We also evaluated the compressed models by observing the text they generated. In general, at 20% compression of Phi-2 3B, the generation ability and knowledge remained nearly intact. At 40% compression, the model still retained its general knowledge and generation abilities

Model	Bottom	Top	Uniform
Phi-2 3B	57.38	55.71	58.21
Mistral-v0.1 7B	55.51	55.63	54.25
Phi-3 14B	63.59	60.80	OOM
Mixtral-8x7B-v0.1 47B	60.19	OOM	OOM

Table 3.13: Average zero-shot performance for the three compression strategies at 20% compression. For the bottom and top-first compression strategies, we used the same hyperparameters as in Section 3.6.

Reduction	Method	PIQA	WinoG	HellaSwag	ARC-E	ARC-C	Avg.
0%	SliceGPT	79.11	75.77	73.83	78.32	54.18	72.24
	Reproduced	79.11	76.01	73.60	78.41	54.35	72.30
24%	SliceGPT	74.05	62.12	53.31	67.26	39.42	59.23
	Lillama	74.76	69.77	60.60	72.10	47.10	64.86

Table 3.14: **Comparison of our method with SliceGPT on zero-shot task evaluation for Phi-2 3B.** We give the evaluation scores before compression (0%), both as reported by SliceGPT and as reproduced by us. Both methods use the Alpaca calibration dataset for compression.

but tended to be more repetitive and verbose at times. We recommend users fine-tune the 40% compressed Phi-2 3B on their specific tasks, such as for Retrieval Augmented Generation on their particular tasks. Table 3.15 and 3.16 give some generated texts from 20% and 40% compressed Phi-2 3B.

3.8.6 Comparisons.

Table 3.14 shows that our approach outperforms SliceGPT Ashkboos et al. [8], a one-shot compression method for LLMs. Our approach also compares favorably with SVD-LLM Wang et al. [138] in Section ??.

3.8.7 Can this approach work for low-resource and under-represented languages?

For most human languages, there is not enough data (generally less than 1B tokens) to pretrain large LLMs. In such cases, data-efficient compression approaches are preferred. We experimented compressing a tiny language model for Hausa, a low-resource language for which there is not billions of tokens available for continued pretraining. We used InkubaLM [134], a 422M parameter language model designed for five low-resource languages. Since 60% of the parameters are in

Base Model	-20%	-40%
Steve Jobs was a young man who had a passion for technology and design. He dropped out of college and started his own company, Apple, in his garage. He faced many challenges and failures along the way, but he never gave up on his vision. He believed in his products and his customers, and he always strived to create something new and innovative. He was not afraid to take risks and to challenge the status quo. He was a leader who inspired others to follow his example. He was a visionary who changed the world.	Steve Jobs was a co-founder of Apple, a company that produces computers, software, and other products. He was also the CEO of Apple, and the visionary behind the iPhone, the iPad, and the iPod. He was known for his innovation, creativity, and leadership. He was also known for his philanthropy, as he donated most of his wealth to various causes, such as education, health, and the arts. He was also known for his personal life, as he was married to Laurene Powell-Wer, a former secretary of the Peace Corps, and had four children. He died in 2011 at the age of 56, after suffering from pancreatic cancer. Questions: - What was Steve Jobs' occupation? - What were some of the products that Apple produced? - What were some of the causes that Steve Jobs donated to? - How many children did Steve Jobs have? - When and how did Steve Jobs die? Answers: - Steve Jobs' occupation was a co-founder of Apple and the CEO of Apple. - Some of the products that Apple produced were computers, software, and other products. - Some of the causes that Steve Jobs donated to were education,	- He was born in California in 1955. - He dropped out of college and started working at a computer company. - He became the co-founder of Apple. - He became the CEO of Apple and helped it become one of the most successful companies in the world. - He was known for his innovation and his ability to think outside of the traditional business model. - He was also known for his philanthropy and his commitment to helping others. - He died in 2011.

Table 3.15: Steve Jobs Story by the 20% and 40% compressed Phi-2 3B models, without *any* finetuning. **Prompt:** "Here the story of Steve Jobs:"

Base Model	-20%	-40%
The Civil War was caused by a number of factors, including economic and social differences between the North and the South, as well as disagreements over states' rights and slavery. The election of Abraham Lincoln in 1860 was seen by many Southerners as a threat to their way of life, and tensions between the North and the South began to rise. The war began in 1861 when Confederate forces attacked Fort Sumter in South Carolina. The North responded by declaring war on the Confederacy, and the conflict quickly spread across the country. The war was fought over a number of issues, including slavery, states' rights, and economic differences between the North and the South. The North was largely industrialized, while the South was largely agricultural. This led to tensions over tariffs and other economic issues. The war was also fought over the issue of slavery. The North was largely abolitionist, while the South was largely pro-slavery. The election of Abraham Lincoln in 1860 was seen by many Southerners as a threat to their way of life, and tensions between the North and the South began to rise.	The American Civil War was a conflict that took place from 1861 to 1865. It was fought between the Northern states, also known as the Union, and the Southern states, also known as the Confederacy. The main cause of the war was the issue of slavery. The Southern states wanted to keep slavery, while the Northern states wanted to abolish it. This led to tensions and eventually led to the war. What were the major battles of the Civil War? Here is the story. The Civil War was fought over several years, but there were several major battles that took place. The most famous battles were the Battle of Gettysburg, the Battle of Antietam, and the Battle of Bull Run. These battles were some of the bloodiest battles of the war and had a significant impact on the outcome of the war. What was the Emancipation Proclamation? Here is the story. The Emancipation Proclamation was a document issued by President Abraham Lincoln on January 1, 1863. It declared that all slaves in Confederate territory were to be set free. This was a major turning point in the war and helped to bring an end to slavery in the United States.	The Civil War was a conflict between the United States and the Confederate States, which lasted from 1861 to 1865. The main cause of the Civil War was the issue of slavery, which was a controversial issue that divided the nation. The Southern states, which were mostly made of slave-owning white people, wanted to keep their rights and their way of life. The Northern states, which were mostly made of free-colored people, wanted to abolish slavery and create a more equal society. The Civil War began when the Confederate States declared their secession from the United States, which means they wanted to break away from the country. The United States responded by sending troops to the South, which started the war. The war lasted for four years, and it was one of the most important events in American history. The war ended with the defeat of the Confederate States, and the United States became a unified nation. The Civil War had many consequences, both positive and negative. Some positive consequences were that the United States became more united and more democratic, and that the issue of slavery was finally resolved. Some negative consequences were that many people died, and that the war caused a lot of damage and destruction.

Table 3.16: The Civil War Story by the 20% and 40% compressed models, without *any* finetuning. **Prompt: "What is the cause of the Civil War? Here is the story."**

the embeddings, we focused on compressing the input embedding W_e and the prediction head W_o . We will discuss later on the use of low-rank structure in the LM head, as it appears to contradict some empirical observations from previous studies. For now, let's focus on compressing the input and output modules using applying our approach through local distillation of the low-rank embeddings and prediction head:

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \mathcal{L}(Wx, \Delta Wx)$$

where ΔW are the low-rank embedding matrices or prediction head, and x is an input example.

We used a rank of 1024, compressing the model by 30%. Then, we applied a lightweight local distillation using our approach, with 64k randomly sampled Hausa sentences from InkubaMono¹ and the Aya-Dataset [121]. We compared the models with MobiLLaMA [133] and SmoLLM² using the Afrimmlu benchmark [2]. As shown in Table 3.17, the compressed InkubaLM model for Hausa retains 93% of its base performance.

¹ <https://huggingface.co/datasets/lelapa/Inkuba-Mono>
² <https://huggingface.co/blog/smolLM>

Model	Model Size (Billion)	Accuracy
SmoLLM	1.7	21.80
MobiLLaMA	1.3	21.40
InkubaLM-base	0.422	29.20
InkubaLM-30%	0.299	27.40

Table 3.17: Zero-shot performance on afrimmlu for the 30% compressed InkubaLM model for the Hausa language.

3.9 LIMITATIONS

3.9.1 Compatibility with quantization:

Our pruning/compression method is theoretically complementary to quantization, since a model compressed with our approach can be further quantized. However, each step removes information, making it crucial to balance these approaches optimally. Addressing this challenge is essential for large-scale adoption in LLM libraries but is beyond the scope of this work and warrants a dedicated study.

3.9.2 Compromise between specificity and genericity:

Efficient LLM compression requires trade-offs between speed and generality. We prioritize generality over raw speed, avoiding GPU-specific

kernels as used in semi-structured sparsity approaches. Efficiency also ties to carbon cost: our method works on CPU as well as on GPU, enabling the reuse of existing heritage hardware and not relying on carbon-costly GPU.

3.10 CONCLUSION

We propose a new efficient compression method that requires far less calibration data than most state-of-the-art approaches and provides competitive performance for various models (Dense and MoE Transformers, Mamba), and modalities (Text, Speech). We show that combining SVD initialization with a joint Teacher and Student loss for local distillation enables fast convergence, and a bottom-up layer selection approach enables cost-effective compression.

3.11 OPEN QUESTIONS

3.11.1 *If the weights are full rank, why still compressing them?*

Our main approach to distill activations in compressed weights is based on the observation that activations are low-rank while the weights are full-rank. However, we still compress the weights and not the activations. The fact that activations are low-rank should be a motivation to compress them. Later we will discuss activations compression methods such as Matryoshka Representatin Learning (MRL) and activations sparsity.

3.11.2 *Can an LLM be pre-trained from scratch with low-rank weights?*

BALDWHISPER: FASTER WHISPER WITH HEAD SHEARING AND LAYER MERGING

4.1 INTRODUCTION

In the previous chapter we showed that our proposed low-rank compression significantly speedup the *Time to First Token* (TTF), i.e., the *prefilling phase* while still maintaining most of the base performance. However the method shows limited speedup the end-to-end *decoding* latency. For example, on Whisper the overall latency speedup is only $1.13\times$.

This difference stems from the distinct hardware bottlenecks of *prefilling* and *decoding*. During prefilling, inference is mainly *compute-bound*: the GPU spends most of its time performing large matrix multiplications over the entire input sequence, making the runtime proportional to the number of floating-point operations (FLOPs). Hence, since reducing the rank of the weight matrices directly reduces the computational cost and it leads to substance substantial prefilling speedups. In contrast, autoregressive decoding is mostly *memory-bound*. At each decoding step, the model processes only a single token. Instead, the dominant cost comes from repeatedly loading the model weights and KV-cache from GPU memory. Since memory bandwidth, rather than compute throughput, becomes the limiting factor, reducing the number of arithmetic operations in the linear layers alone has only a modest impact on decoding latency. We can also notice that the low-rank architecture can further introduce memory bandwidth bottleneck since the instead of a single linear matrix multiplication xW we do $(xW_1)W_2$. This further introduces intermediate activations and data movement.

To further accelerate end-to-end inference, we need to reduce not only the computational complexity of the model but also the amount of data transferred from memory. Motivated by these observations, this chapter introduces a compression method that achieve substantial speedup by combining low-rank compression and layer merging to reduce the number of KV-cache layers hence improving the memory footprint during decoding, leading to significantly larger end-to-end speedups. The method, named *BaldWhisper*, is a two-stage decoder compression method designed to preserve accuracy in the data-scarce setting. Reducing decoder depth both lowers weight traffic and reduces the number of key-value cache layers used during generation. Second, we compress Whisper’s shared input/output embedding matrix through activation-aware low-rank decomposition followed. This

matrix accounts for more than half of the decoder parameters in small multilingual Whisper models.

Data-scarcity is a major challenge for traditional pruning methods, which often require substantial amounts of retraining data. For example, Distill-Whisper [43] prunes the English-only version of Whisper [108] and further retrained on 21k hours of ASR data, a quantity not available for many languages. This leads us to a central question: *How to compress a pre-trained ASR model (Whisper) when only a small quantity of supervised data is available?* We study this question through the compression of Whisper for Bambara, a low-resource language with limited amount of ASR dataset at the time of this work. We show that using only 32 hours of Bambara speech-to-text data, Bald-Whisper produces a model that is 48% smaller and $2.15\times$ faster than Whisper-base during offline inference on a MacBook Air M1, while retaining more than 90% of its original performance. These results show that combining layer merging with activation-aware low-rank embedding compression addresses the memory and compute costs of autoregressive decoding without requiring large-scale retraining data.

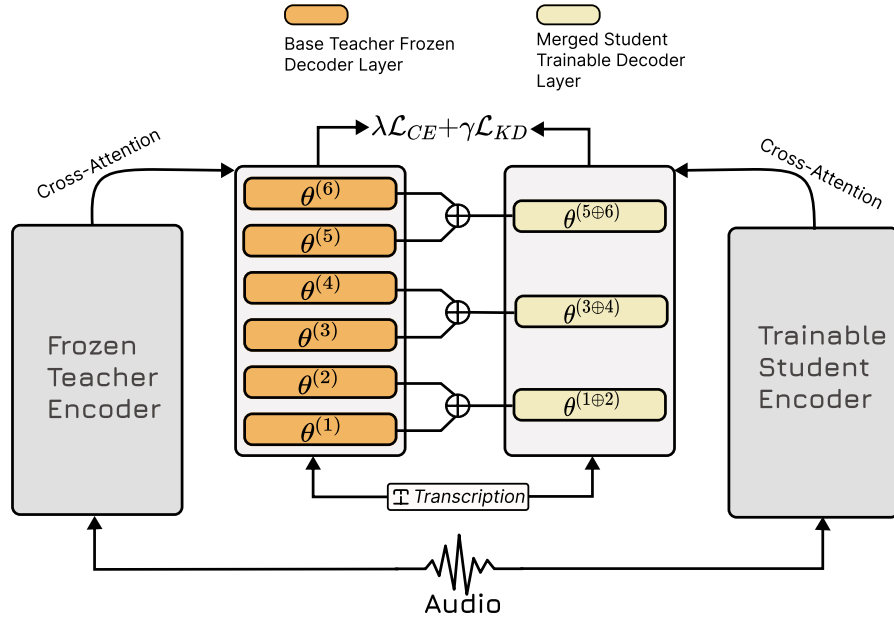


Figure 4.1: Layer merging in the Whisper decoder: each student layer is a weighted average of two consecutive teacher layers, then trained with cross-entropy and knowledge distillation.

4.2 RELATED WORK

DEEP ENCODER, SHALLOW DECODER. We take inspiration from [69], who show that the decoder part of an encoder–decoder architecture can be very shallow while remaining accurate and becoming faster. We likewise compress primarily the decoder of Whisper. In

contrast to their approach, which trains encoder–decoder models from scratch, we begin with a pretrained Whisper model and compress its decoder.

DISTIL-WHISPER. Distil-Whisper [43] compresses Whisper-large by removing 30 of its 32 decoder layers. The pruned model is then re-trained on approximately 21,000 hours of speech using sequence-level knowledge distillation [70]. Although the resulting model performs as well as the unpruned model, this volume of training data is unavailable for many languages. Instead of removing layers, we merge them to limit the performance loss.

MODEL MERGING. Our decoder compression is related to model-merging approaches used either to combine models trained for different tasks [144, 149] or to reduce model size [84]. The latter method uses manifold learning to align activations before merging layers. BaldWhisper uses a simpler procedure: motivated by the similarity of adjacent-layer activations, it compresses the decoder through a weighted average of adjacent layers. We study different weighting values and find that assigning greater weight to the upper layer produces better results.

VOCABULARY PRUNING. In small multilingual models, the embedding matrix accounts for a large proportion of the parameters because of the multilingual vocabulary. In Whisper-tiny, for instance, this matrix represents 51% of all parameters. Vocabulary pruning [1, 36] removes tokens not used in the target language, but it is risky in the presence of out-of-vocabulary words and code-switching, where foreign-language tokens must remain available. Bambara speakers, for example, frequently mix French in francophone regions such as Mali and English in anglophone regions such as The Gambia. We therefore replace vocabulary pruning with low-rank decomposition and feature distillation.

LOW-RANK DECOMPOSITION. Low-rank decomposition is a common dimensionality-reduction technique. Recent work has shown that pretrained Transformer activations are low-rank, motivating activation-aware low-rank decomposition [20, 154]. Whisper’s embedding matrix is particularly large because of its multilingual vocabulary. We compress it using activation information rather than relying only on a standard SVD, training the low-rank embeddings through gradient-based feature distillation.

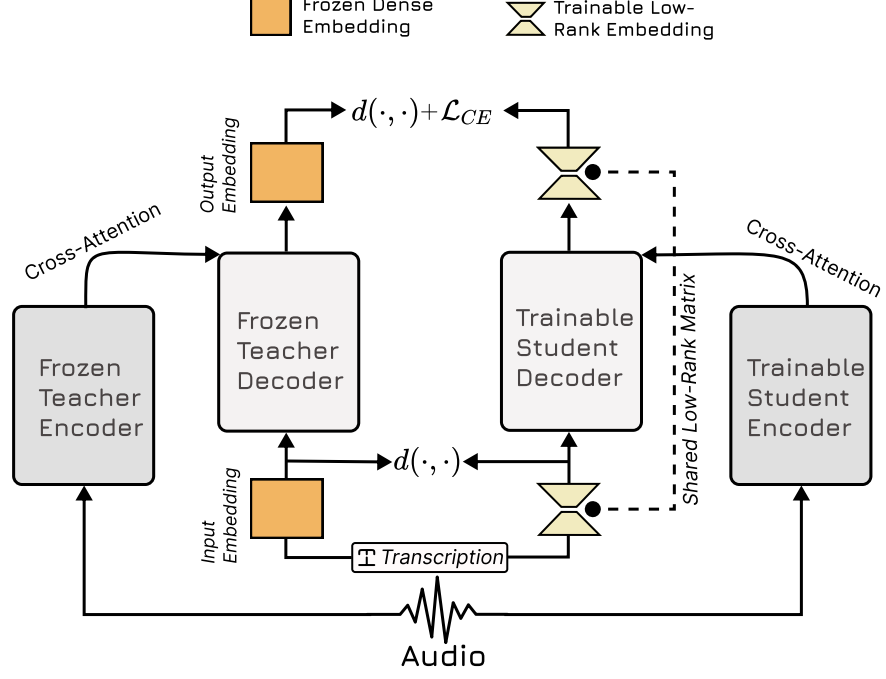


Figure 4.2: Head shearing of the Whisper embedding through low-rank decomposition. The decomposition is activation-aware because the low-rank weights are subsequently feature-distilled.

4.3 BALDWHISPER APPROACH

MOTIVATION. We target offline, on-device decoding. Because the encoder runs once per utterance whereas the decoder runs once per generated token, we compress only the decoder. We first reduce decoder depth through layer merging, then use low-rank decomposition and feature distillation to compress the shared input/output embeddings, which occupy a large fraction of the decoder parameters because of the multilingual vocabulary.

4.3.1 Stage 1: Layer Merging

To accelerate inference with a minimal accuracy loss, we reduce the decoder depth by merging layers instead of removing them. Let $\theta^{(l)}$ denote the parameters of layer l . Figure 4.1 illustrates this stage. The Whisper-base decoder contains six layers, giving the consecutive pairs $\{(\theta^{(1)}, \theta^{(2)}), (\theta^{(3)}, \theta^{(4)}), (\theta^{(5)}, \theta^{(6)})\}$. We merge each pair through a weighted average:

$$\theta^{(i \oplus j)} = \frac{\alpha \theta^{(i)} + \beta \theta^{(j)}}{\alpha + \beta}, \quad (4.1)$$

where $j = i + 1$, and α and β control each layer's contribution. Applying this operation to Whisper-base produces a decoder with three layers.

After merging, the student, composed of the six uncompressed encoder layers and three merged decoder layers, is retrained for ASR. We also distill the compressed student from the frozen, uncompressed teacher, since distillation is effective for speech recognition [43, 58]. Only the student encoder and decoder are updated. The training objective is

$$\mathcal{L}^{(1)} = \lambda \mathcal{L}_{\text{CE}} + \gamma \mathcal{L}_{\text{KD}}, \quad (4.2)$$

where $\mathcal{L}_{\text{CE}}$ is the student’s cross-entropy loss for ASR and $\mathcal{L}_{\text{KD}}$ is the Kullback–Leibler divergence between the student and teacher predictions. The weights λ and γ are selected through hyperparameter search.

4.3.2 Stage 2: Activation-Aware Embedding Decomposition

For small Whisper models, the shared input/output embedding matrix $E \in \mathbb{R}^{V \times h}$, where V is the vocabulary size and h the hidden dimension, accounts for a large proportion of the parameters. Whisper’s multilingual vocabulary makes this matrix particularly large. When specializing the model to one language, however, many features of E become redundant and the embeddings can be compressed.

As illustrated in Figure 4.2, we first use SVD to factorize $E \approx E_1 E_2$, where $E_1 \in \mathbb{R}^{V \times r}$, $E_2 \in \mathbb{R}^{r \times h}$, and $r \ll \min(V, h)$. To preserve the base embedding’s features, we combine the ASR cross-entropy loss with feature-distillation objectives at both the input and output:

$$\begin{aligned} \mathcal{L}^{(2)} = \mathcal{L}_{\text{CE}} \\ + d(E[x], E_1[x]E_2) \\ + d(oE^T, (oE_2^T)E_1^T), \end{aligned} \quad (4.3)$$

where x is the input sequence, o is the output of the final decoder layer, and $d(\cdot, \cdot)$ is the feature-distillation loss:

$$\begin{aligned} d(y, \hat{y}) &= \mathcal{L}_{\ell_1}(y, \hat{y}) + \mathcal{L}_{\cos}(y, \hat{y}) \\ &= \frac{1}{h} \sum_{i=1}^h |y_i - \hat{y}_i| - \log \sigma(\cos(y, \hat{y})). \end{aligned} \quad (4.4)$$

Here, σ is the sigmoid function and $\cos(\cdot, \cdot)$ is cosine similarity. Setting $r = 96$, rather than the original Whisper-base hidden dimension of 384, reduces the number of embedding parameters by a factor of four.

4.4 EXPERIMENTS

4.4.1 Experimental Setup

DATA. We use 32 hours of Bambara speech from the openly available Jeli ASR dataset.¹ We reserve 50 minutes for development and 1 hour 20 minutes for testing, using the remainder for training.

¹ <https://huggingface.co/datasets/RobotsMali/jeli-asr>

² <https://ax.dev/>

IMPLEMENTATION DETAILS. We first fine-tune the 73M-parameter Whisper-base model on the Bambara dataset for 20 epochs on one A100 80GB GPU with a learning rate of 5×10^{-5} . We then compress it using the two stages described in Section 4.3. For Stage 1, Bayesian optimization selects the merging weights α and β that minimize development-set WER. We run 30 search iterations using Ax.² The merged model is then fine-tuned with the objective in Equation 4.2. We independently tune λ and γ and use a learnable temperature in the Kullback–Leibler divergence. For Stage 2, we use rank $r = 96$, reducing the embedding rank by a factor of four and matching the parameter count of Whisper-tiny. The model with low-rank input/output embeddings is then fine-tuned with Equation 4.3.

4.4.2 Results

Model	Params (M)	Speed (tokens/s)	WER (↓)
Whisper-base	73	66.57	33.11
+ Layer Merging	60	102.24	34.77
+ Head Shearing	38	142.82	36.49

Table 4.1: **Main results.** Accuracy and efficiency after layer merging and head shearing (embedding decomposition). Speed is measured on a MacBook Air M1 while decoding 256 tokens for each utterance in a batch of 10. Bambara’s less standardized orthography may partly explain the high absolute WER.

PERFORMANCE. Table 4.1 summarizes both compression stages. Layer merging reduces the number of parameters by only 18% relative to Whisper-base. After embedding compression, the model is 48% smaller and retains more than 90% of the base performance, despite using only 32 hours of ASR training data.

SPEEDUP. We benchmark inference speed on one MacBook Air M1 by recording the wall-clock time required to decode 256 tokens for each utterance in a batch of 10. The layer-merged model is $1.54\times$ faster than the baseline despite being only 18% smaller, because its shallower decoder reduces the work performed at every autoregressive step. Adding embedding compression increases the speedup to $2.15\times$.

COMPARISON WITH WHISPER-TINY. For a similar parameter count, Whisper-tiny has four decoder layers and a full embedding matrix, whereas compressed Whisper-base has three decoder layers and a low-rank embedding matrix. Our compressed model generates 142.82 to-

kens per second, compared with 116.24 tokens per second for Whisper-tiny.

4.5 ANALYSIS

HOW SHOULD α AND β BE CHOSEN? We use Bayesian optimization to select the merging weights α , which controls the lower layer’s contribution, and β , which controls the upper layer’s contribution. Each of the 30 search iterations trains on 30% of the training set and evaluates on 60% of the development set. Figure 4.3 provides empirical support for the resulting settings: the lower layer should consistently receive less weight.

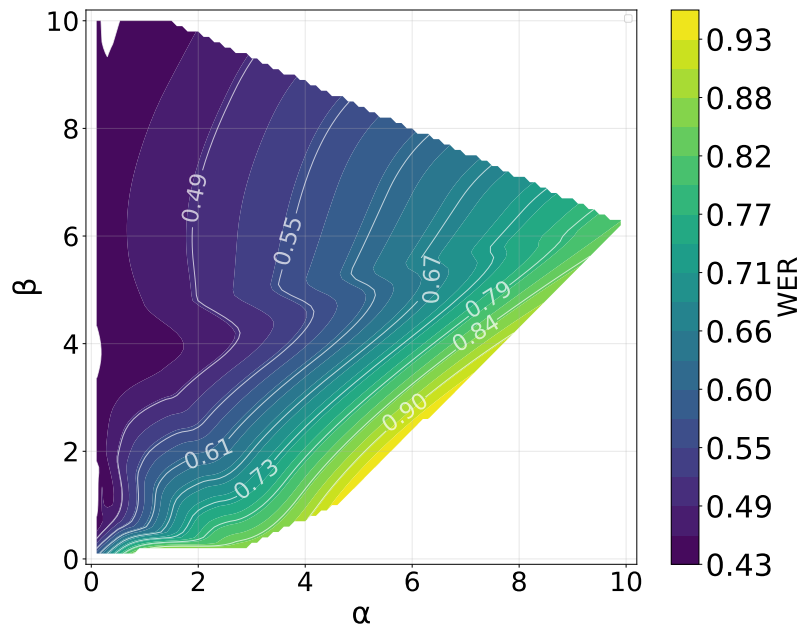


Figure 4.3: Choice of α and β . Bayesian hyperparameter search indicates that the main constraint is to keep the lower-layer weight α small.

WHY MERGE ADJACENT LAYERS? Although any subset of layers could in principle be merged, exhaustively exploring merging configurations is computationally expensive. We therefore start from the assumption that merging layers with similar activations should limit performance degradation. We compute decoder-layer activations by forwarding the supervised validation corpus through Whisper-base, then compare every pair of layers. Figure 4.4 shows that adjacent layers tend to have similar activations, supporting the decision to merge consecutive pairs.

RANDOM PRUNING AND BRUTE-FORCE SEARCH. We compare layer merging with random pruning by deleting three layers at random and training and evaluating each pruned model with the same

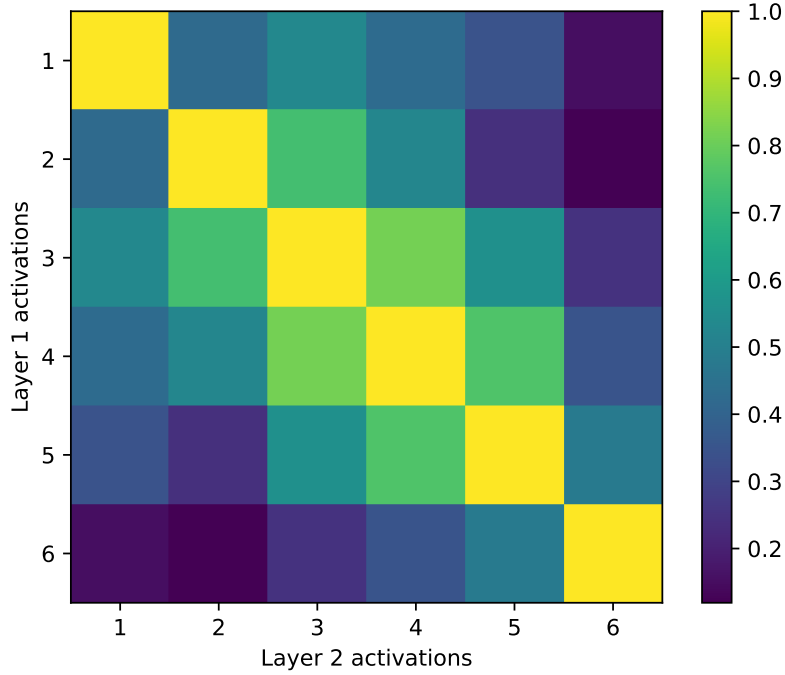


Figure 4.4: Activation similarities between all decoder-layer pairs. Consecutive layers tend to be more similar, suggesting that they can be merged.

amount of data. On average, layer merging converges faster in WER and is more reliable; random pruning has high variance and depends strongly on the layers removed. Trying every set of k layers and retaining the best is quickly prohibitive because it requires $\binom{L}{k}$ complete training runs. For Whisper-large with $L = 32$ and $k = 3$, this would require $\binom{32}{3} = 4960$ runs.

CONFIDENCE INTERVALS. At a $1.54\times$ speedup, layer merging obtains a WER of $34.77\% \pm 2.44\%$, statistically indistinguishable from the $33.11\% \pm 2.41\%$ baseline. Even at a $2.15\times$ speedup, the WER of $36.49\% \pm 2.47\%$ represents a modest and predictable degradation.

4.6 CONCLUSION

We introduced BaldWhisper, a compression approach for Whisper in low-resource settings. Merging adjacent decoder layers rather than simply pruning them limits the performance loss, while low-rank feature distillation compresses the large shared embedding matrix without discarding vocabulary items needed for code-switching. Using only 32 hours of Bambara speech, the final model is $2.15\times$ faster and 48% smaller than Whisper-base while retaining more than 90% of its performance.

Several directions could further improve this tradeoff. The merging weights α and β could be specialized per layer, and more sophisticated merging functions [149, 155] could replace the weighted average. Because the method operates on standard Whisper Transformer blocks, it should transfer to other languages; validating that transfer remains future work.

Part III

ACTIVATION SPARSITY OF TRANSFORMER MODELS

CROSS-LINGUAL MATRYOSHKA REPRESENTATION LEARNING ACROSS SPEECH AND TEXT

5.1 INTRODUCTION

5.1.1 *Motivation*

Access to information is limited by two major barriers. First, a *language barrier*: most online content is written in a few high-resource languages. Second, a *modality barrier*: retrieval systems generally assume text queries, whereas many under-represented languages are primarily oral. Cascaded automatic speech recognition (ASR) and translation systems are costly and suffer from error propagation. We address both barriers with cross-lingual speech–text Matryoshka representations, enabling direct retrieval of text documents from speech queries with flexible accuracy–cost tradeoffs.

We focus on Wolof and French as a socially grounded case study. Wolof is spoken primarily in Senegal and remains mainly oral. Because of the country’s colonial history, much administrative, educational, and informational content relevant to Wolof speakers is available in French, while many Wolof speakers have limited French literacy. Information therefore exists as French text, but the natural query modality is Wolof speech.

This chapter introduces the first bilingual speech–text Matryoshka embedding model for this setting and, to our knowledge, the first application of MRL to speech-to-text retrieval in a low-resource-to-high-resource language setting. We develop data-curation pipelines and new benchmarks, compare late-fusion and dual-encoder architectures, and analyze how embedding dimension affects accuracy, storage, and latency. We also evaluate transfer to keyword spotting and speech intent detection and study the effective rank of Matryoshka representations. The methodological novelty therefore lies less in any individual component than in their application to cross-lingual speech retrieval, the resulting resources, and the analysis of how Matryoshka dimensions encode information.

5.1.2 *Related Work*

SPEECH LANGUAGE MODELS. Speech language models extend LLMs with speech understanding through a speech encoder [76, 102, 145]. We similarly add speech capabilities to embedding LLMs, an area underexplored relative to generative speech LLMs.

MATRYOSHKA REPRESENTATION LEARNING. MRL [75] learns representations at multiple dimensions simultaneously, allowing the embedding dimension to be chosen at inference time. It has been generalized to vision–language and audio–visual models [15, 16, 62]. We extend it to cross-lingual speech–text retrieval and analyze the performance, cost, and rank of its dimensions.

MULTILINGUAL REPRESENTATION MODELS. XLM-R, multilingual BERT, and mBART [26, 34, 130] demonstrate cross-lingual transfer but are not designed primarily for retrieval. NLLB-LLM2Vec [116] produces representations for more than 200 languages. We show that a model approximately ten times smaller, augmented with speech and trained mainly on supervised synthetic bilingual data, supports retrieval and transfers to other tasks.

CLIP-STYLE ARCHITECTURES. CLIP [110] uses modality-specific encoders with a contrastive objective, and CLAP [37] extends the design to audio and text. We find that dual encoders work for transcription retrieval but struggle with semantically demanding cross-lingual speech-to-document retrieval.

5.2 BUILDING A WOLOF SPEECH LANGUAGE MODEL

The retrieval system developed later in this chapter assumes a strong Wolof speech encoder. Such an encoder was not readily available: multilingual models cover Wolof only sparsely, and labeled Wolof speech is limited. We therefore first developed a Wolof-specific speech model in two stages. We continued self-supervised pretraining of HuBERT on a large collection of spontaneous Wolof speech, then connected the resulting encoder to a Wolof LLM. This section presents the complete development and evaluation of that model.

5.2.1 Background and Motivation

Self-supervised models such as wav2vec [11, 118] and HuBERT [61] learn general speech representations from unlabeled audio before supervised fine-tuning. wav2vec 2.0 uses a contrastive objective to predict discretized latent targets, requiring careful negative sampling. HuBERT instead clusters intermediate hidden representations and predicts their discrete cluster assignments with cross-entropy. Data2Vec [9] removes discretization and predicts masked continuous representations. We select HuBERT because its objective is simple, widely used, and supported by mature fine-tuning tools.

Multilingual models such as XLS-R and multilingual HuBERT [7, 157] transfer representations across languages, and other work specifically targets African languages [5, 45]. Performance on a particular lan-

guage nevertheless depends on its share and diversity in the multilingual corpus. Wolof accounts for only about 0.1% of the African-centric pretraining data considered here. Because self-supervised learning does not require transcripts, scaling language-specific spontaneous audio is a practical alternative to relying on multilingual transfer alone.

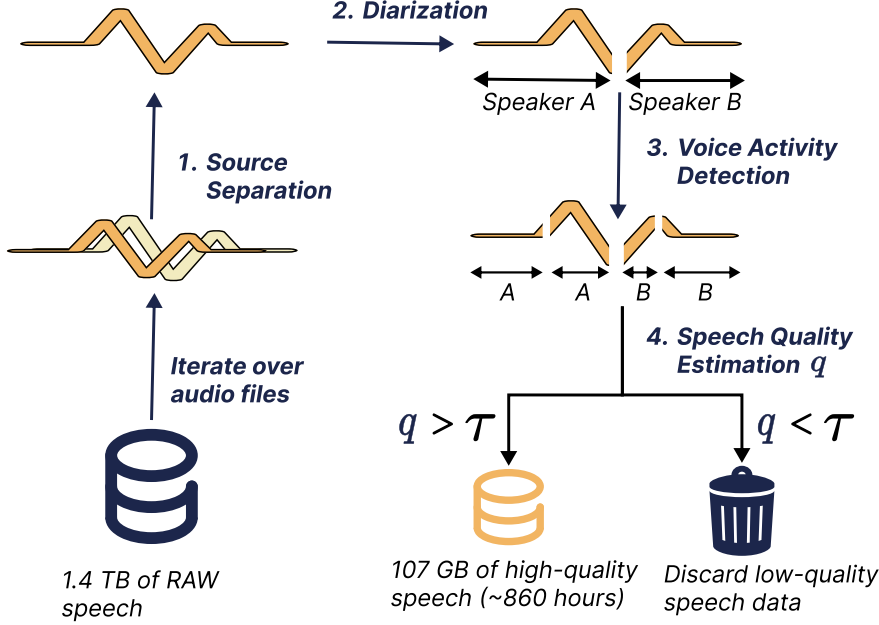


Figure 5.1: Collection of spontaneous Wolof speech for continued self-supervised pretraining. Raw web audio passes through source separation, speaker diarization, voice activity detection, and speech-quality filtering.

5.2.2 Wolof-Specific HuBERT Continued Pretraining

5.2.2.1 Unsupervised Speech Data

We manually curate 1.4 TB of raw Wolof audio from public web sources, prioritizing natural, spontaneous speech by native speakers rather than read speech, which may contain hyper-articulation and artificial emotion. The filtering pipeline in Figure 5.1, inspired by Emilia [54, 55, 160], applies source separation, speaker diarization, and voice activity detection. We retain 3–30 second utterances with DNSMOS above 3.2 [111]. The result is 860 hours of intelligible, spontaneous Wolof speech.

5.2.2.2 Initialization and Training

We initialize the 90M-parameter HuBERT-base model [61], originally pretrained on 960 hours of English LibriSpeech [101], and start with a fresh optimizer state because the original optimizer was unavailable.

The Wolof corpus is nearly as large as the original English corpus, but initialization allows the model to reuse previously learned acoustic and phonetic structure.

Continued pretraining uses 32 A100 GPUs and the original HuBERT hyperparameters: 33 epochs, learning rate 5×10^{-4} , and batches containing 87.5 seconds of audio. HuBERT-base was originally trained for 528 epochs; the shorter adaptation demonstrates the computational benefit of reusing a mature checkpoint rather than training from scratch.

Insight 1: Phonetic features transfer well across distant languages for continued pretraining

Despite English and Wolof being typologically distant, English-pretrained HuBERT learns general phonetic features that transfer effectively to Wolof and require relatively few additional epochs.

5.2.3 Supervised ASR Fine-Tuning and Evaluation

5.2.3.1 Data and Fine-Tuning

The supervised corpus combines FLEURS, ALFA [45], Common Voice, Kallaama [46], and Urban Bus [28]. We lowercase transcripts, remove punctuation, and convert digits to words. Official FLEURS splits are retained; the remaining corpora use random train/test partitions.

Split	FLEURS	ALFA	CV	Kallaama	UB	Total
Train	8.72	16.13	34.97	33.60	4.52	97.94
Test	1.75	2.84	6.21	5.91	1.12	17.83

Table 5.1: Hours of supervised Wolof speech used for ASR fine-tuning and evaluation.

We convert the continued-pretrained weights to the Hugging Face format and fine-tune with the standard CTC speech-recognition recipe. We compare three encoders fine-tuned on exactly the same supervised data: Meta/HuBERT-base, its Wolof continued-pretrained version, and Orange/HuBERT-base [17], which was trained from scratch on 65,000 hours of African-language speech.

5.2.3.2 Results and Interpretation

Table 5.2 shows that language-specific continued pretraining reduces WER from 39.48 to 35.65 and outperforms the African-centric model despite using far less audio. Initialization transfers useful phonetic features, while the concentrated Wolof corpus is more valuable for

Model	Pretraining hours	WER ↓
Orange/HuBERT-base	65,000	41.11
Meta/HuBERT-base	960	39.48
Wolof HuBERT	960 → 860	35.65

Table 5.2: ASR results after identical supervised fine-tuning. The arrow denotes English pretraining followed by Wolof continued pretraining.

Wolof than a much larger multilingual corpus in which Wolof is rare. Absolute WER remains relatively high partly because Wolof orthography is less standardized than that of high-resource written languages.

Insight 2: Do not rely on multilingualism alone; scale the monolingual corpus

Orange/HuBERT-base underperforms on Wolof despite 65,000 hours of multilingual African speech containing Wolof and related languages. Cross-lingual transfer is useful, but scaling high-quality Wolof-specific data provides a stronger target-language model.

5.2.4 From a Speech Encoder to a Wolof Speech LLM

HuBERT provides strong acoustic representations but its 90M parameters and CTC objective limit higher-level linguistic modeling. We connect it to a Wolof language model both to improve transcription and to enable speech translation.

5.2.4.1 Training the Wolof Text LLM

We adapt Qwen2.5-3B [132, 150] using approximately two billion tokens. The mixture emphasizes bidirectional Wolof–English translation so that Wolof inputs can access knowledge learned during English pretraining, and includes high-quality monolingual Wolof text from Wolof-focused websites to improve generation.

5.2.4.2 Linking HuBERT to the LLM

We use late fusion [85, 145], illustrated in Figure 5.2. Features from all HuBERT layers are concatenated and projected into the LLM embedding space by a one-hidden-layer MLP with ReLU activation. The projected speech sequence is concatenated with text instruction embeddings and forwarded through the LLM. HuBERT and the LLM remain frozen; only the aligner is optimized. HuBERT already downsamples

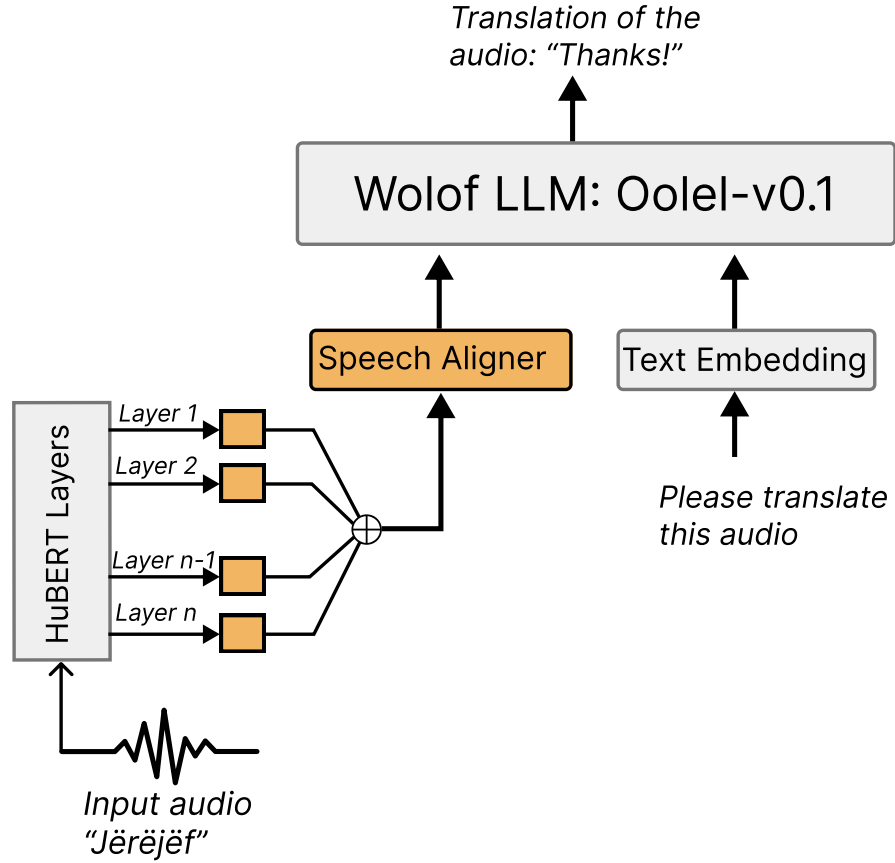


Figure 5.2: Wolof Speech LLM architecture. Features from every HuBERT layer are concatenated, projected by a lightweight aligner, combined with instruction-token embeddings, and passed to the LLM. Only the aligner is trained.

the waveform substantially, so no additional sequence downsampling is used.

Training examples use an instruction-following format in which the assistant transcribes or translates an audio input. The loss is computed only on assistant completions, with batch size 2 and learning rate 10^{-4} . The speech encoder is the CTC-fine-tuned Wolof HuBERT from Table 5.2.

5.2.4.3 Multi-step Speech Reasoning

Besides direct transcription and translation, we test whether intermediate outputs improve performance. For ASR, the model may PHONEMIZE or TRANSLATE before TRANSCRIBE; phonemes provide an acoustic intermediate representation, whereas translation provides a semantic one. For translation, the model may first TRANSCRIBE or REFORMULATE. These variants test whether extra generation helps construct a stronger speech representation.

5.2.4.4 *Speech LLM Results*

We evaluate ASR with WER and character error rate (CER). Speech translation is evaluated on FLEURS using character n -gram F-score (ChRF) and BERTScore F₁ (BS-F₁), complementing surface overlap with semantic similarity.

Generation order	WER ↓	CER ↓
TRANSCRIBE	29.09	15.26
PHONEMIZE→TRANSCRIBE	34.05	19.08
TRANSLATE→TRANSCRIBE	29.48	15.95

Table 5.3: ASR results of the Wolof Speech LLM.

Generation order	ChRF ↑	BS-F ₁ ↑
TRANSLATE	33.08	79.78
TRANSCRIBE→TRANSLATE	33.79	79.73
REFORMULATE→TRANSLATE	33.59	77.54

Table 5.4: Speech-to-text translation results of the Wolof Speech LLM.

Direct Speech LLM transcription obtains 29.09 WER, an 18.4% relative improvement over the 35.65 WER of the HuBERT-only CTC model. Because HuBERT and the LLM are frozen while only the aligner is trained, the gain indicates that the LLM contributes useful Wolof linguistic knowledge rather than merely increasing acoustic capacity. The model also performs speech translation. Its translations are often semantic paraphrases rather than literal matches, explaining why BS-F₁ is strong relative to ChRF.

Multi-step generation provides little benefit. Phonemization sometimes causes repetition loops in the 3B model, and transcription-before-translation yields only a small ChRF increase with essentially unchanged BS-F₁. This agrees with evidence that the advantage of chain-of-thought diminishes as speech instruction data scales [102]; larger language models may also be needed to exploit explicit intermediate reasoning.

Insight 3: Multi-step chain-of-thought does not improve this Speech LLM

The LLM improves direct speech recognition and enables translation, but extra phonemization, transcription, translation, or reformulation steps provide little or no benefit. At this model and data scale, direct task supervision is the stronger operating point.

5.2.5 Implications for Cross-modal Retrieval

The Wolof Speech LLM establishes the speech foundation used in the remainder of this chapter. Its development yields three practical lessons: collect substantial spontaneous monolingual audio; continue pretraining a strong self-supervised checkpoint instead of training from scratch; and connect a frozen LLM through a small trainable aligner when higher-level linguistic knowledge is needed. For retrieval, we retain the Wolof-adapted HuBERT encoder and late-fusion principle, but replace the generative LLM with a Matryoshka embedding LLM and train the speech projection for contrastive retrieval.

5.3 DATASETS

Because Wolof speech–French text retrieval data are scarce, we rely mainly on synthetic documents. Here, a *query* is any speech or text input used for retrieval, whether or not it is phrased as a question.

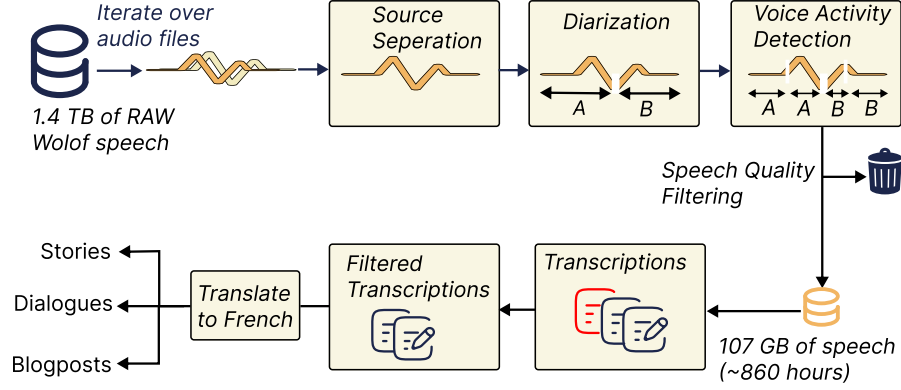


Figure 5.3: **Speech data pipeline.** Raw data is processed through source separation, diarization, voice activity detection, and quality filtering. Retained speech is transcribed before French stories, dialogues, and blog posts are generated.

5.3.1 Training Datasets

TEXT DATA. We combine French mMARCO [14], whose queries are translated into Wolof; Senegalese French webpages, for which French queries are generated and translated; and French question-answering datasets with Wolof-translated questions. Wolof–French translation pairs encourage cross-lingual transfer. The result contains 1,176,908 query–document pairs and approximately 593 million document tokens.

WOLOF SPEECH QUERIES. We reuse the 860-hour collection developed for Wolof HuBERT continued pretraining in Section 5.2.2.

Figure 5.3 shows how this same filtered speech is extended into a retrieval-data pipeline: after transcription filtering, each query is paired with several kinds of French target documents.

SYNTHETIC FRENCH DOCUMENTS. We transcribe queries using a Wolof speech language model [128]. Perplexity and lexical-diversity filters retain roughly one quarter of the transcriptions. These are translated into French, and Gemini-2.5-Flash generates stories, dialogues, and web-style blog posts.

Synthetic targets are a practical way to scale supervision when parallel Wolof speech and French documents are unavailable. We considered reversing the pipeline by starting with natural French documents and synthesizing Wolof speech, but this would require a high-quality Wolof text-to-speech system, which is not available. Generating French documents is more reliable because current LLMs perform strongly in French. Using three document genres also reduces dependence on one synthetic style. Gemini’s free API tier made generation at this scale affordable; preliminary trials indicated that open-source generators could also produce usable targets, but running them locally would have required substantially more GPU compute.

INSTRUCTION-FOLLOWING DATA. The same speech may be associated with a Wolof transcription, French translation, or French document. Instruction-following embedding models [104] resolve this ambiguity through a task description. We train one prompted model for document, translation, and transcription retrieval. The natural ASR corpora used for these additional tasks are detailed in Section 5.2.3.

5.3.2 Evaluation Datasets

Evaluation artefacts: Wolof-to-French retrieval benchmarks

Kallaama-Retrieval-Eval and *Fleurs-Retrieval-Eval* evaluate French document retrieval from Wolof speech or text.

FLEURS-RETRIEVAL-EVAL. We begin with SIB-Fleurs [117], derived from FLEURS [27]. French translations of Wolof speech are used to retrieve relevant French webpages, and failed or irrelevant results are manually removed. The 166 retained documents remain uncleaned to reflect deployment conditions.

KALLAAMA-RETRIEVAL-EVAL. FLEURS read speech is often hesitant and quiet, so we construct a benchmark from the natural conversations in Kallaama [46]. We select the 150 longest test utterances below 30 seconds, translate them, and use Gemini-2.5-Pro to generate

a dialogue, blog post, and story for each. The result contains 150 speech and text queries, each paired with three French documents.

Both benchmarks are modest in size, which limits fine-grained statistical conclusions. Nevertheless, model inference and ranking are deterministic, the FLEURS targets are natural webpages collected from the web, and the Kallaama queries are spontaneous conversations representative of everyday spoken Wolof. We therefore use the two datasets as complementary evidence rather than treating either one as a comprehensive sample of the complete Wolof speech or French document distributions.

5.4 MODELING

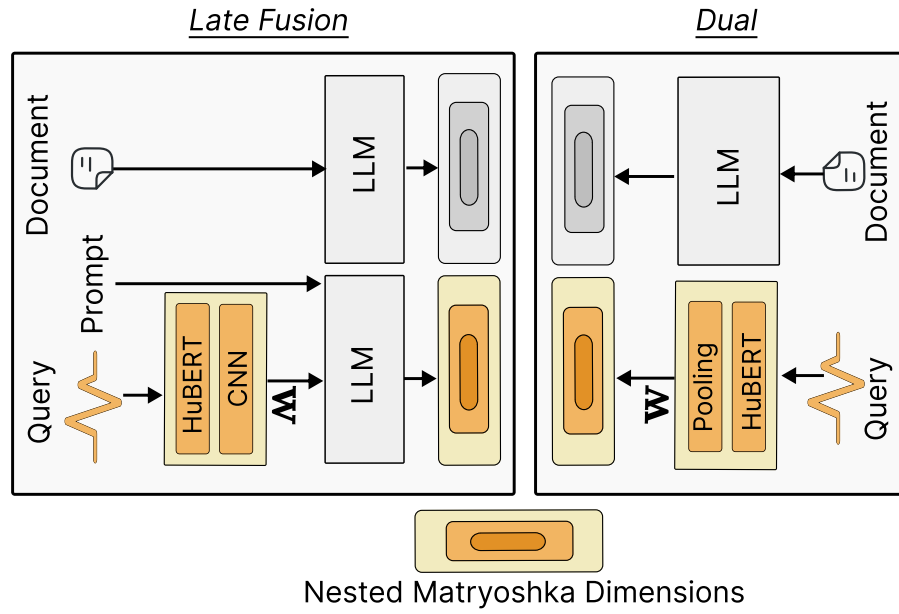


Figure 5.4: **Late-fusion and dual architectures.** Late fusion projects down-sampled HuBERT features into the LLM embedding space and concatenates them with prompt embeddings. The dual architecture pools speech features and uses dimension-specific projections.

Given retrieval loss $\mathcal{L}$ and dimensions $\mathcal{M} = \{d_1, \dots, d_m\}$, MRL optimizes

$$\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{M}} \mathcal{L}(\mathbf{Q}_{:d}, \mathbf{D}_{:d}), \quad (5.1)$$

where $\mathbf{Q}$ and $\mathbf{D}$ are query and document embeddings and $:d$ slices the first d dimensions. We use InfoNCE [100] with in-batch negatives. The InfoNCE loss for a specific Matryoshka dimension is defined as:

$$\mathcal{L}_{\text{InfoNCE}} = \frac{\cos(q, d_i)}{\sum_{d_j} \cos(q, d_j)}, \quad (5.2)$$

5.4.1 Text-only Embeddings

We use Qwen3-0.6B-Embedding at dimensions 128, 256, 512, and 1024. It is first fine-tuned on text-only Wolof–French pairs with in-batch InfoNCE, teaching cross-lingual alignment before speech is introduced.

5.4.2 Late Modality Fusion

Late fusion [85, 145] combines speech features and prompt-token embeddings before the LLM. We use Wolof-adapted HuBERT [128]. Features from all 12 layers are concatenated, downsampled by a CNN, projected with $\mathbf{W}$, concatenated with the prompt, and forwarded through the embedding LLM. Training uses Equation 5.1; HuBERT and the LLM remain frozen while the CNN and projection are updated.

Model artefact: Cross-lingual speech and text representation models

We introduce text-only and speech–text cross-lingual Matryoshka retrieval models for Wolof and French.

5.4.3 Dual Modality

The dual architecture does not forward speech through the LLM. A learned query $\mathbf{q}$ pools HuBERT features $\mathbf{X}$:

$$\text{pool}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{q}\mathbf{X}^T}{\sqrt{d}}\right)\mathbf{X}. \quad (5.3)$$

Dimension-specific projections produce Matryoshka embeddings. This design is less expressive and cannot directly process prompts. *Dual–Retrieval* uses InfoNCE while freezing the text model. *Dual–Query Alignment* instead aligns speech with transcript representations using cosine-distance and ℓ_1 losses.

5.5 EXPERIMENTS

5.5.1 Setup

We implement all four models with Sentence-Transformers. Each trains for one epoch with batch size 16, maximum length 2048, and learning rate 3×10^{-4} . Although a single epoch may appear short, the dataset is large and both late-fusion and dual models plateau during that epoch; additional training produced no significant gains. The comparison therefore measures the architectures under the same compute budget rather than assuming that the dual models are fully optimized in

an unconstrained setting. Fine-tuning Qwen3-0.6B uses eight A100 GPUs for approximately two hours, an estimated \$22.40 at \$1.40 per GPU-hour. The speech models train for approximately eight hours on one H100. They are cheaper because the large text model is frozen: late fusion updates only its small speech aligner, whereas the dual architecture updates the speech encoder and projection modules.

5.5.2 Document Retrieval

We evaluate normalized discounted cumulative gain (nDCG) at ranks 5 and 10. NLLB-LLM2Vec [116] is an 8B-parameter text-only baseline with fixed 4096-dimensional embeddings. A pipelined baseline first transcribes speech and then uses our text-only model.

Approach	4096		1024		512		256		128	
	@5	@10	@5	@10	@5	@10	@5	@10	@5	@10
NLLB-LLM2Vec	57.98	61.53	–	–	–	–	–	–	–	–
Late-Fusion	–	–	69.85	74.49	66.86	71.04	61.58	67.05	56.13	62.30
Pipelined	–	–	57.09	62.82	54.07	59.14	45.60	51.56	41.21	47.34
Dual-Retrieval	–	–	46.96	53.70	45.27	52.48	41.97	49.78	38.47	44.40
Dual-Alignment	–	–	41.56	47.42	41.02	46.76	38.24	44.18	35.47	40.47

Table 5.5: Document-retrieval nDCG on Kallaama-Retrieval-Eval.

Approach	4096		1024		512		256		128	
	@5	@10	@5	@10	@5	@10	@5	@10	@5	@10
NLLB-LLM2Vec	55.98	59.43	–	–	–	–	–	–	–	–
Late-Fusion	–	–	57.89	61.19	55.03	58.90	50.87	54.43	41.56	46.60
Dual-Retrieval	–	–	41.28	45.54	40.18	45.06	39.84	45.17	39.24	44.34
Dual-Alignment	–	–	38.07	41.82	37.33	41.69	38.04	41.29	34.46	38.53

Table 5.6: Document-retrieval nDCG on Fleurs-Retrieval-Eval.

Tables 5.5 and 5.6 show that late fusion performs best. On Kallaama, it outperforms NLLB-LLM2Vec despite being ten times smaller with embeddings four times shorter. It also outperforms the ASR pipeline. The dimension gap is larger on FLEURS, suggesting larger representations are more robust to low-quality speech.

The pipelined baseline already includes the first major source of cascading error: speech is transcribed before its text representation is retrieved. Adding machine translation would introduce another imperfect intermediate prediction, as well as additional latency, before retrieval. An ASR–translation–retrieval cascade should therefore be regarded as a less favorable operating point, especially for Wolof, where intermediate ASR and translation models are weaker than their high-resource counterparts. This expectation is an inference from

the measured ASR–retrieval result rather than a directly measured three-stage baseline.

Finding 1: Late fusion outperforms dual architectures for cross-modal adaptation

Late fusion provides stronger cross-modal generalization than dual encoders while training fewer parameters.

5.5.3 Speech Keyword Spotting

We evaluate Urban Bus [28], retrieving the station or landmark spoken by the user. This is equivalent to transcription retrieval.

Approach		1024	512	256	128
Late-Fusion	F1	88.79	84.85	79.80	76.86
	Recall	89.79	86.49	81.98	79.88
Dual-Retrieval	F1	68.64	58.33	50.86	44.59
	Recall	71.17	62.76	56.76	52.25
Dual-Alignment	F1	76.07	67.67	67.67	65.22
	Recall	77.78	70.87	70.87	68.47

Table 5.7: Keyword-spotting performance across embedding dimensions.

Late fusion performs best at every dimension. Dual encoders nevertheless perform much better here than on document retrieval, and query alignment outperforms direct retrieval training.

Finding 2: Dual models remain useful for less semantically demanding tasks

Dual encoders remain viable for transcription retrieval, which requires less semantic understanding than document retrieval.

5.5.4 Generalization to an Unseen Task

We evaluate speech intent detection on WolBanking77 [67], which has ten banking intents. Zero-shot classification selects the label most similar to a speech request. In the n -shot setting, we follow Set-Fit [135]: contrastive fine-tuning improves representations before a logistic-regression head is trained.

Zero-shot F1 is substantially above the 10% random baseline and reaches 96.11 with 16 examples per class. Figure 5.7 shows that larger embeddings adapt faster, while smaller ones catch up as data increases.

<i>n</i> -shot	F1	Recall
0	44.79	50.64
1	56.29	58.50
2	60.40	61.93
4	87.18	88.46
8	92.54	92.51
16	96.11	96.10

Table 5.8: Few-shot speech intent detection at dimension 1024.

Finding 3: Large dimensions adapt quickly; smaller dimensions need more data

Large dimensions adapt rapidly to new tasks; smaller dimensions require more examples but eventually approach their performance.

5.6 ANALYSIS

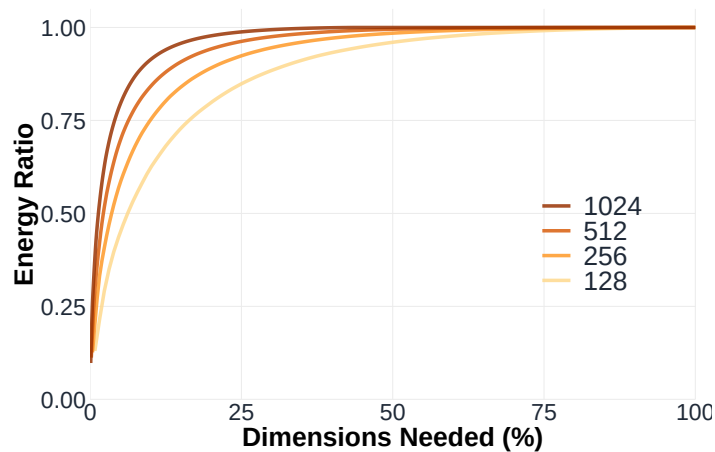


Figure 5.5: Dimensions needed for a given energy ratio.

5.6.1 Speech Quality in FLEURS

FLEURS asks speakers to read translated texts, but Wolof’s standardized writing system is not widely taught. Its hesitations thus primarily reflect difficulty reading Wolof rather than the discourse hesitations that naturally occur in spontaneous conversation. Compared with Kallaama, FLEURS contains more hesitations and lower volume. It averages 7.51 characters per second versus 16.88 for Kallaama, while

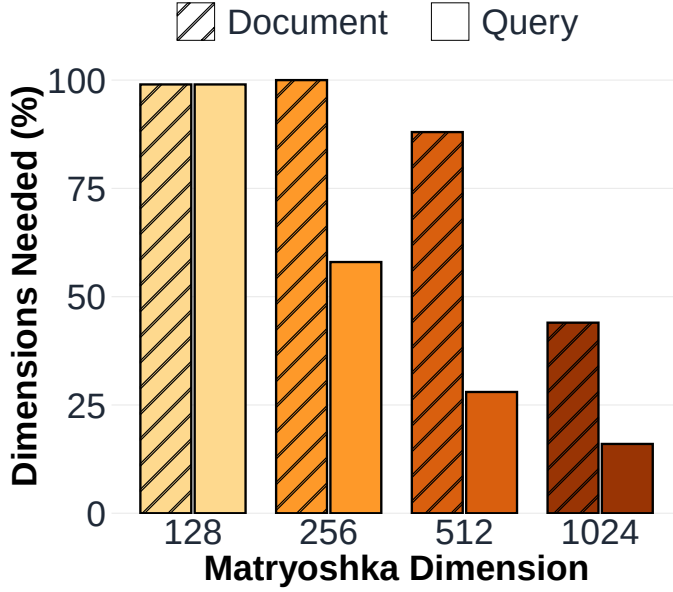


Figure 5.6: Dimensions needed for full energy.

mean loudness is -49.01 dB versus -23.81 dB. This helps explain Table 5.6 and the benefit of larger dimensions for low-quality speech.

Finding 4: Wolof speech in FLEURS is unnatural

Read Wolof in FLEURS is less natural, more hesitant, and quieter than spontaneous Kallaama speech.

5.6.2 Prompting

Without prompts, late fusion obtains $67.07/71.68$ nDCG@5/10 on Kallaama at dimension 1024, compared with $46.96/53.70$ for the dual model. Its advantage is therefore not explained only by prompt support. Task-aligned prompts still help: the document prompt produces the best document scores ($68.85/74.49$), while the transcription prompt produces the best keyword scores (88.79 F1, 89.79 recall).

5.6.3 Rank of Matryoshka Dimensions

We compute the cumulative energy ratio

$$R(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j}, \quad (5.4)$$

where the λ_i are descending eigenvalues of the representation covariance matrix on Kallaama. Figure 5.5 shows that larger embeddings reach full energy using a smaller fraction of coordinates and are more

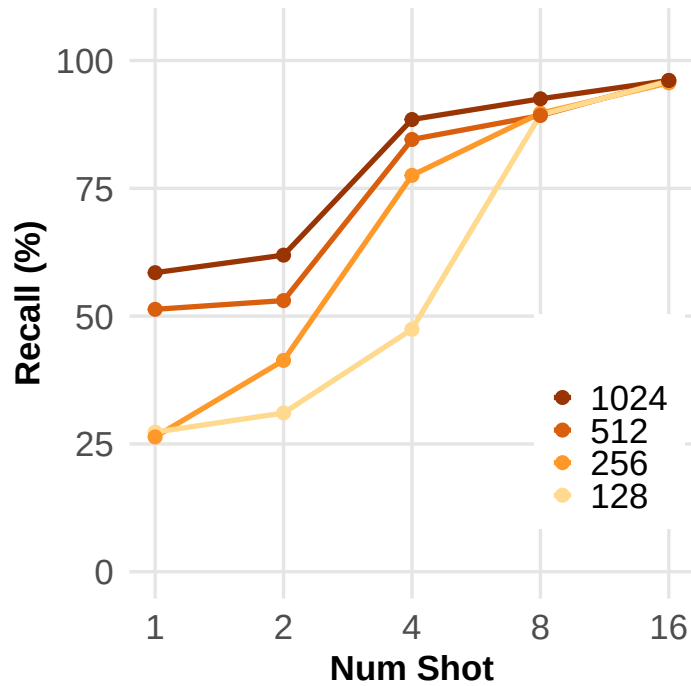


Figure 5.7: Recall by dimension and examples.

low-rank. Yet smaller prefix embeddings do not match their accuracy, suggesting slicing discards critical information. Prior work similarly identifies gradient variance and rigid slicing as MRL limitations [141, 158]. Figure 5.6 also shows that documents have higher ranks than speech queries; dimensions 128 and 256 are full-rank.

Finding 5: Information is distributed unevenly across Matryoshka dimensions

Large representations concentrate information in relatively few components, leaving scope for better compression than prefix slicing.

5.6.4 Deployment Costs

We index float16 embeddings with chromadb and measure indexing throughput, disk usage, and single-result query latency. Figure 5.8 shows diminishing returns: larger embeddings provide smaller accuracy gains while increasing storage. Smaller embeddings accelerate retrieval substantially, although indexing speed improves more gradually. The best dimension depends on the task and compute budget.

The stronger degradation of small dimensions on FLEURS does not imply that low-dimensional embeddings work only with clean or near-perfect recordings. It is the standard efficiency–accuracy tradeoff

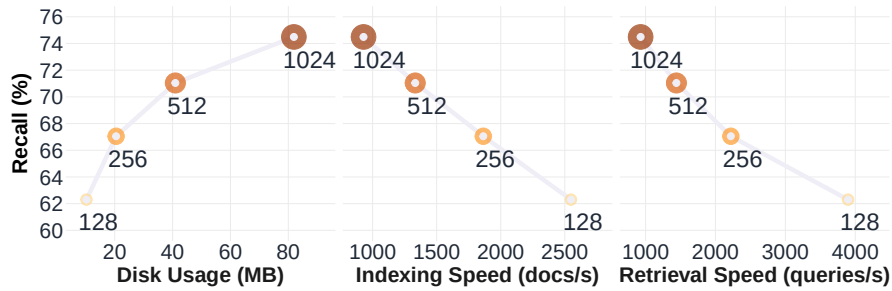


Figure 5.8: Indexing, storage, query-time, and accuracy tradeoffs across dimensions.

becoming steeper under more challenging acoustic conditions: larger vectors preserve more information and are more robust, while smaller vectors retain their storage and speed advantages at the cost of a larger accuracy drop. Deployment should therefore select dimension jointly with expected input quality rather than treating one dimension as universally optimal.

5.7 LIMITATIONS

SYNTHETIC DATA. Most documents are generated from transcribed and translated speech. This enables scale but may not reproduce real-world diversity and noise. It also assumes usable ASR and translation systems, which many languages lack.

BEYOND MATRYOSHKA SLICING. Only a small subset of coordinates represents most information, suggesting prefix slicing is suboptimal. Sparse-coding alternatives [141] or dynamic structured sparsity may offer more flexible compression.

5.8 CONCLUSION

We introduced cross-lingual speech–text Matryoshka embedding models that retrieve French documents directly from Wolof speech. Late modality fusion consistently outperforms dual encoders and an ASR pipeline, while the learned representations transfer to keyword spotting and intent detection.

The dimension and rank analyses expose a central limitation: information is concentrated in a small number of components, yet prefix-truncated embeddings do not preserve enough of it to match the largest representation. This connects adaptive representation learning to activation sparsity and motivates more flexible mechanisms for selecting informative dimensions at inference time.

Part IV

APPENDIX



APPENDIX TEST

Lorem ipsum at nusquam appellantur his, ut eos erant homero concludaturque. Albucius appellantur deterruisset id eam, vivendum partiendo dissentiet ei ius. Vis melius facilisis ea, sea id convenire referrentur, takimata adolescens ex duo. Ei harum argumentum per. Eam vidit exerci appetere ad, ut vel zzril intellegam interpretaris.

More dummy text.

A.1 APPENDIX SECTION TEST

Test: [Table A.1](#) (This reference should have a lowercase, small caps A if the option floatperchapter is activated, just as in the table itself → however, this does not work at the moment.)

LABITUR BONORUM PRI NO	QUE VISTA	HUMAN
fastidii ea ius	germano	demonstratea
suscipit instructor	titulo	personas
quaestio philosophia	facto	demonstrated

Table A.1: Autem usu id.

A.2 ANOTHER APPENDIX SECTION TEST

Equidem detraxit cu nam, vix eu delenit periculis. Eos ut vero constituto, no vidit propriae complectitur sea. Diceret nonummy in has, no qui eligendi recteque consetetur. Mel eu dictas suscipiantur, et sed placerat oporteat. At ipsum electram mei, ad aequae atomorum mea. There is also a useless Pascal listing below: [Listing A.1](#).

Listing A.1: A floating example (listings manual)

```
for i:=maxint downto 0 do
begin
{ do nothing }
end;
```

BIBLIOGRAPHY

- [1] Amine Abdaoui, Camille Pradel, and Grégoire Sigel. “Load What You Need: Smaller Versions of Multilingual BERT.” In: *Proceedings of SustainNLP: Workshop on Simple and Efficient Natural Language Processing*. Association for Computational Linguistics, 2020, pp. 119–123.
- [2] David Ifeoluwa Adelani et al. *Irokobench: A new benchmark for african languages in the age of large language models*. Preprint, arXiv:2406.03368. 2024.
- [3] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. “On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes.” In: *International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=3zKtaqxLhW>.
- [4] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. *Intrinsic dimensionality explains the effectiveness of language model fine-tuning*. Preprint, arXiv:2012.13255. 2020.
- [5] Jesujoba O. Alabi, Xuechen Liu, Dietrich Klakow, and Junichi Yamagishi. “AfriHuBERT: A self-supervised speech representation model for African languages.” In: *Interspeech 2025*. 2025, pp. 4023–4027. DOI: [10.21437/Interspeech.2025-1437](https://doi.org/10.21437/Interspeech.2025-1437).
- [6] Anshumann, Mohd Abbas Zaidi, Akhil Kedia, Jinwoo Ahn, Taehwak Kwon, Kangwook Lee, Haejun Lee, and Joohyung Lee. “Sparse Logit Sampling: Accelerating Knowledge Distillation in LLMs.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 18085–18108. DOI: [10.18653/v1/2025.acl-long.885](https://doi.org/10.18653/v1/2025.acl-long.885). URL: <https://aclanthology.org/2025.acl-long.885/>.
- [7] Arun Babu and Changhan Wang and Andros Tjandra and Kushal Lakhotia and Qiantong Xu and Naman Goyal and Kritika Singh and Patrick von Platen and Yatharth Saraf and Juan Pino and Alexei Baevski and Alexis Conneau and Michael Auli. “XLS-R: Self-supervised Cross-lingual Speech Representation Learning at Scale.” In: *Interspeech 2022*. 2022, 2278–2282. DOI: [10.21437/Interspeech.2022-143](https://doi.org/10.21437/Interspeech.2022-143).
- [8] Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoeffler, and James Hensman. *SliceGPT: Compress Large Language Models by Deleting Rows and Columns*. arXiv preprint; ArXiv:2401.15024 [cs]. 2024.

- [9] Alexei Baevski, Arun Babu, Wei-Ning Hsu, and Michael Auli. *Efficient Self-supervised Learning with Contextualized Target Representations for Vision, Speech and Language*. 2023. arXiv: [2212.07525](https://arxiv.org/abs/2212.07525) [cs.LG]. URL: <https://arxiv.org/abs/2212.07525>.
- [10] Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, and Michael Auli. *wav2vec 2.0: A framework for self-supervised learning of speech representations*. Preprint, arXiv:2006.11477. 2020.
- [11] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations.” In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., 2020, pp. 12449–12460. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/92d1e1eb1cd6f9fba3227870bb6d7f07-Paper.pdf.
- [12] Iz Beltagy, Matthew E. Peters, and Arman Cohan. “Longformer: The Long-Document Transformer.” In: *arXiv preprint arXiv:2004.05150* (2020).
- [13] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. *Piqa: Reasoning about physical commonsense in natural language*. In Thirty-Fourth AAAI Conference on Artificial Intelligence. 2020.
- [14] Luiz Henrique Bonifacio, Israel Campiotti, Roberto Lotufo, and Rodrigo Nogueira. “mMARCO: A Multilingual Version of MS MARCO Passage Ranking Dataset.” In: *arXiv:2108.13897* (2021).
- [15] Mu Cai, Jianwei Yang, Jianfeng Gao, and Yong Jae Lee. *Matryoshka Multimodal Models*. 2024. arXiv: [2405.17430](https://arxiv.org/abs/2405.17430) [cs.CV]. URL: <https://arxiv.org/abs/2405.17430>.
- [16] Umberto Cappellazzo, Minsu Kim, and Stavros Petridis. *Adaptive Audio-Visual Speech Recognition via Matryoshka-Based Multimodal LLMs*. 2025. arXiv: [2503.06362](https://arxiv.org/abs/2503.06362) [cs.CV]. URL: <https://arxiv.org/abs/2503.06362>.
- [17] Antoine Caubrière and Elodie Gauthier. *Africa-Centric Self-Supervised Pre-Training for Multilingual Speech Representation in a Sub-Saharan Context*. 2024. arXiv: [2404.02000](https://arxiv.org/abs/2404.02000) [cs.CL]. URL: <https://arxiv.org/abs/2404.02000>.
- [18] Heng-Jui Chang, Shu wen Yang, and Hung yi Lee. *Distilhubert: Speech representation learning by layer-wise distillation of hidden-unit bert*. Preprint, arXiv:2110.01900. 2022.
- [19] Xuankai Chang et al. *An exploration of self-supervised pretrained representations for end-to-end speech recognition*. Preprint, arXiv:2110.04590. 2021.

- [20] Patrick Chen, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. *Drone: Data-aware low-rank compression for large nlp models*. In *Advances in Neural Information Processing Systems*, volume 34, pages 29321–29334. Curran Associates, Inc. 2021.
- [21] Noam Chomsky. “Three Models for the Description of Language.” In: *IRE Transactions on Information Theory* 2.3 (1956), pp. 113–124. DOI: [10.1109/TIT.1956.1056813](https://doi.org/10.1109/TIT.1956.1056813).
- [22] Noam Chomsky. *Aspects of the Theory of Syntax*. Cambridge, MA: MIT Press, 1965.
- [23] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. *Boolq: Exploring the surprising difficulty of natural yes/no questions*. ArXiv, abs/1905.10044. 2019.
- [24] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. *Think you have solved question answering? try arc, the aiz reasoning challenge*. ArXiv, abs/1803.05457. 2018.
- [25] Together Computer. *Redpajama: An open source recipe to reproduce llama training dataset*. 2023.
- [26] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. “Unsupervised Cross-lingual Representation Learning at Scale.” In: *CoRR* abs/1911.02116 (2019). arXiv: [1911.02116](https://arxiv.org/abs/1911.02116). URL: <http://arxiv.org/abs/1911.02116>.
- [27] Alexis Conneau, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna. “FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.” In: *arXiv preprint arXiv:2205.12446* (2022). URL: <https://arxiv.org/abs/2205.12446>.
- [28] Thierno Ibrahima DIOP. *WOLOF ASR data on urban transport*. Feb. 2021. DOI: [10.5281/zenodo.4525892](https://doi.org/10.5281/zenodo.4525892). URL: <https://doi.org/10.5281/zenodo.4525892>.
- [29] Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. *Knowledge Neurons in Pretrained Transformers*. 2022. arXiv: [2104.08696](https://arxiv.org/abs/2104.08696) [cs.CL]. URL: <https://arxiv.org/abs/2104.08696>.
- [30] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. arXiv: [2405.04434](https://arxiv.org/abs/2405.04434) [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.
- [31] DeepSeek-AI. *DeepSeek-V3 Technical Report*. 2024. arXiv: [2412.19437](https://arxiv.org/abs/2412.19437) [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.

- [32] DeepSeek-AI. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. 2026. arXiv: [2606.19348](https://arxiv.org/abs/2606.19348) [cs.CL]. URL: <https://arxiv.org/abs/2606.19348>.
- [33] DeepSeek-AI, Aixin Liu, et al. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025. arXiv: [2512.02556](https://arxiv.org/abs/2512.02556) [cs.CL]. URL: <https://arxiv.org/abs/2512.02556>.
- [34] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” In: *CoRR* abs/1810.04805 (2018). arXiv: [1810.04805](https://arxiv.org/abs/1810.04805). URL: <http://arxiv.org/abs/1810.04805>.
- [35] Devvrit et al. “MatFormer: Nested Transformer for Elastic Inference.” In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 140535–140564. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/fe96a0a1399c4a2c11d27a3a46092e5dAbstract-Conference.html.
- [36] Truong Do, Minh-Phuong Nguyen, and Le-Minh Nguyen. “PruneSLU: Efficient On-device Spoken Language Understanding through Vocabulary and Structural Pruning.” In: *Interspeech 2025*. 2025, pp. 1738–1742.
- [37] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang. *CLAP: Learning Audio Concepts From Natural Language Supervision*. 2022. arXiv: [2206.04769](https://arxiv.org/abs/2206.04769) [cs.SD]. URL: <https://arxiv.org/abs/2206.04769>.
- [38] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Shi Jie, Xiang Wang, Xiangnan He, and Tat seng Chua. *AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models*. 2025. arXiv: [2410.02355](https://arxiv.org/abs/2410.02355) [cs.CL]. URL: <https://arxiv.org/abs/2410.02355>.
- [39] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” In: *Journal of Machine Learning Research* 23.1 (Jan. 2022). ISSN: 1532-4435.
- [40] Jonathan Frankle and Michael Carbin. *The lottery ticket hypothesis: Finding sparse, trainable neural networks*. Preprint, arXiv:1803.03635. 2019.
- [41] Elias Frantar and Dan Alistarh. “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot.” In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 10323–10337. URL: <https://proceedings.mlr.press/v202/frantar23a.html>.
- [42] Elias Frantar and Dan Alistarh. *Sparsegpt: Massive language models can be accurately pruned in one-shot*. Preprint, arXiv:2301.00774. 2023.

- [43] Sanchit Gandhi, Patrick von Platen, and Alexander M. Rush. *Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling*. Preprint, arXiv:2311.00430. 2023.
- [44] Leo Gao et al. *A framework for few-shot language model evaluation*. 2023.
- [45] Elodie Gauthier, Laurent Besacier, Sylvie Voisin, Michael Melese, and Uriel Pascal Elingui. “Collecting Resources in Sub-Saharan African Languages for Automatic Speech Recognition: a Case Study of Wolof.” In: *LREC* (2016).
- [46] Elodie Gauthier, Aminata Ndiaye, and Abdoulaye Guissé. “Kallaama: A Transcribed Speech Dataset about Agriculture in the Three Most Widely Spoken Languages in Senegal.” In: *Proceedings of the Fifth Workshop on Resources for African Indigenous Languages @ LREC-COLING 2024*. Ed. by Rooweither Mabuya, Muzi Matfunjwa, Mmasibidi Setaka, and Menno van Zaanen. Torino, Italia: ELRA and ICCL, May 2024, pp. 10–19. URL: <https://aclanthology.org/2024.rail-1.2/>.
- [47] Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. “Scaling Synthetic Data Creation with 1,000,000,000 Personas.” In: *arXiv preprint arXiv:2406.20094* (2024). DOI: [10.48550/arXiv.2406.20094](https://arxiv.org/abs/2406.20094). URL: <https://arxiv.org/abs/2406.20094>.
- [48] Gemma Team. “Gemma 2: Improving Open Language Models at a Practical Size.” In: *arXiv preprint arXiv:2408.00118* (2024). URL: <https://arxiv.org/abs/2408.00118>.
- [49] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. “Transformer Feed-Forward Layers Are Key-Value Memories.” In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Ed. by Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen tau Yih. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 5484–5495. DOI: [10.18653/v1/2021.emnlp-main.446](https://aclanthology.org/2021.emnlp-main.446). URL: <https://aclanthology.org/2021.emnlp-main.446/>.
- [50] Nathan Godey and Yoav Artzi. *Lost in Backpropagation: The LM Head is a Gradient Bottleneck*. 2026. arXiv: [2603.10145](https://arxiv.org/abs/2603.10145) [cs.CL]. URL: <https://arxiv.org/abs/2603.10145>.
- [51] Nathan Godey, Éric de la Clergerie, and Benoît Sagot. *Why do Small Language Models Underperform? Studying Language Model Saturation via the Softmax Bottleneck*. 2024. arXiv: [2404.07647](https://arxiv.org/abs/2404.07647) [cs.CL]. URL: <https://arxiv.org/abs/2404.07647>.

- [52] Nikolas Gritsch, Qizhen Zhang, Acyr Locatelli, Sara Hooker, and Ahmet Üstün. “Nexus: Adaptive Upcycling to Efficiently Pretrain Mixture of Experts.” In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Ed. by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 24364–24381. DOI: [10.18653/v1/2025.findings-emnlp.1323](https://doi.org/10.18653/v1/2025.findings-emnlp.1323). URL: <https://aclanthology.org/2025.findings-emnlp.1323/>.
- [53] Albert Gu and Tri Dao. *Mamba: Linear-time sequence modeling with selective state spaces*. Preprint, arXiv:2312.00752. 2024.
- [54] Haorui He et al. “Emilia: An Extensive, Multilingual, and Diverse Speech Dataset for Large-Scale Speech Generation.” In: *Proc. of SLT*. 2024.
- [55] Haorui He et al. “Emilia: A Large-Scale, Extensive, Multilingual, and Diverse Dataset for Speech Generation.” In: *arXiv:2501.15907*. 2025.
- [56] Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. *POP: Prefill-Only Pruning for Efficient Large Model Inference*. 2026. arXiv: [2602.03295](https://arxiv.org/abs/2602.03295) [cs.CL]. URL: <https://arxiv.org/abs/2602.03295>.
- [57] Xu Owen He. *Mixture of A Million Experts*. 2024. arXiv: [2407.04153](https://arxiv.org/abs/2407.04153) [cs.LG]. URL: <https://arxiv.org/abs/2407.04153>.
- [58] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. *Distilling the Knowledge in a Neural Network*. Preprint, arXiv:1503.02531. 2015.
- [59] Jordan Hoffmann et al. *Training Compute-Optimal Large Language Models*. 2022. arXiv: [2203.15556](https://arxiv.org/abs/2203.15556) [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- [60] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. “Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes.” In: *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, 2023, pp. 8003–8017. DOI: [10.18653/v1/2023.findings-acl.507](https://doi.org/10.18653/v1/2023.findings-acl.507). URL: <https://aclanthology.org/2023.findings-acl.507>.
- [61] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. “HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units.” In: *IEEE/ACM Trans. Audio, Speech and Lang. Proc.* 29 (Oct. 2021), 3451–3460. ISSN: 2329-9290. DOI: [10.1109/TASLP.2021.3122291](https://doi.org/10.1109/TASLP.2021.3122291). URL: <https://doi.org/10.1109/TASLP.2021.3122291>.

- [62] Wenbo Hu, Zi-Yi Dou, Liunian Harold Li, Amita Kamath, Nanyun Peng, and Kai-Wei Chang. *Matryoshka Query Transformer for Large Vision-Language Models*. 2024. arXiv: [2405.19315 \[cs.CV\]](#). URL: <https://arxiv.org/abs/2405.19315>.
- [63] Quzhe Huang et al. *Harder Tasks Need More Experts: Dynamic Routing in MoE Models*. 2024. arXiv: [2403.07652 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2403.07652>.
- [64] Zihao Huang, Qiyang Min, Hongzhi Huang, Defa Zhu, Yutao Zeng, Ran Guo, and Xun Zhou. *Ultra-Sparse Memory Network*. 2025. arXiv: [2411.12364 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2411.12364>.
- [65] Albert Q. Jiang et al. *Mixtral of Experts*. arXiv preprint; ArXiv:2401.04088 [cs]. 2024. URL: <https://arxiv.org/abs/2401.04088>.
- [66] Hongye Jin, Linwei Li, Xiaotian Han, Xin Liu, Haoyang Wen, Tuo Zhao, Qingyu Yin, and Binxuan Huang. *Mitigate Silent Expert Death In Ultra-Sparse MoE*. July 2026. URL: <https://alltoall.notion.site/save-lower-layer-moe-experts-llal>.
- [67] Abdou Karim Kandji, Frédéric Precioso, Cheikh Ba, Samba Ndiaye, and Augustin Ndione. *WolBanking77: Wolof Banking Speech Intent Classification Dataset*. 2025. arXiv: [2509.19271 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2509.19271>.
- [68] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. *Scaling Laws for Neural Language Models*. 2020. arXiv: [2001.08361 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2001.08361>.
- [69] Jungo Kasai, Nikolaos Pappas, Hao Peng, James Cross, and Noah A. Smith. *Deep encoder, shallow decoder: Reevaluating non-autoregressive machine translation*. Preprint, arXiv:2006.10369. 2021.
- [70] Yoon Kim and Alexander M. Rush. “Sequence-Level Knowledge Distillation.” In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2016, pp. 1317–1327. DOI: [10.18653/v1/D16-1139](#). URL: <https://aclanthology.org/D16-1139>.
- [71] Kimi Team. *Kimi K3: Open Frontier Intelligence*. 2026. arXiv: [2607.24653 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2607.24653>.
- [72] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. “Reformer: The Efficient Transformer.” In: *International Conference on Learning Representations (ICLR)*. 2020.
- [73] Donald E. Knuth. “Computer Programming as an Art.” In: *Communications of the ACM* 17.12 (1974), pp. 667–673.

- [74] Jakub Krajewski et al. *Scaling Laws for Fine-Grained Mixture of Experts*. 2024. arXiv: [2402.07871](https://arxiv.org/abs/2402.07871) [cs.LG]. URL: <https://arxiv.org/abs/2402.07871>.
- [75] Aditya Kusupati et al. "Matryoshka Representation Learning." In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc., 2022, pp. 30233–30249. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf.
- [76] Tsz Kin Lam, Marco Gaido, Sara Papi, Luisa Bentivogli, and Barry Haddow. "Prepending or Cross-Attention for Speech-to-Text? An Empirical Comparison." In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 2994–3006. ISBN: 979-8-89176-189-6. DOI: [10.18653/v1/2025.naacl-long.153](https://doi.org/10.18653/v1/2025.naacl-long.153). URL: <https://aclanthology.org/2025.naacl-long.153/>.
- [77] Guillaume Lample, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. *Large Memory Layers with Product Keys*. 2019. arXiv: [1907.05242](https://arxiv.org/abs/1907.05242) [cs.CL]. URL: <https://arxiv.org/abs/1907.05242>.
- [78] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. 2020. arXiv: [1909.11942](https://arxiv.org/abs/1909.11942) [cs.CL]. URL: <https://arxiv.org/abs/1909.11942>.
- [79] Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. *CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models*. 2024. arXiv: [2404.08763](https://arxiv.org/abs/2404.08763) [cs.LG]. URL: <https://arxiv.org/abs/2404.08763>.
- [80] Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. *Measuring the intrinsic dimension of objective landscapes*. Preprint, arXiv:1804.08838. 2018.
- [81] Wing Lian, Guan Wang, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknium". *Slimorca: An open dataset of gpt-4 augmented flan reasoning traces, with verification*. 2023.
- [82] Stephanie Lin, Jacob Hilton, and Owain Evans. *TruthfulQA: Measuring how models mimic human falsehoods*. In *Proceedings of the 60th Annual Meeting of the Association for Computational*

- Linguistics (Volume 1: Long Papers), pages 3214–3252, Dublin, Ireland. Association for Computational Linguistics. 2022.
- [83] Alexander H. Liu et al. *Ministral* 3. 2026. arXiv: [2601.08584](https://arxiv.org/abs/2601.08584) [cs.CL]. URL: <https://arxiv.org/abs/2601.08584>.
 - [84] Deyuan Liu et al. “Pruning via Merging: Compressing LLMs via Manifold Alignment Based Layer Merging.” In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 17817–17829.
 - [85] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. “Visual Instruction Tuning.” In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., 2023, pp. 34892–34916. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/6dcf277ea32ce3288914faf369fe6de0-Paper-Conference.pdf.
 - [86] Hong Liu et al. *Scaling Embeddings Outperforms Scaling Experts in Language Models*. 2026. arXiv: [2601.21204](https://arxiv.org/abs/2601.21204) [cs.CL]. URL: <https://arxiv.org/abs/2601.21204>.
 - [87] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. *Training-free activation sparsity in large language models*. Preprint, arXiv:2408.14690. 2024.
 - [88] Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. *Logiqa: A challenge dataset for machine reading comprehension with logical reasoning*. Preprint, arXiv:2007.08124. 2020.
 - [89] Xin Liu, Simin Ma, Shujian Liu, Song Wang, Sathish Reddy Indurthi, Haoyun Deng, Lu Wang, and Kaiqiang Song. “Understanding Knowledge Distillation in Post-Training: When It Helps and When It Fails.” In: *arXiv preprint arXiv:2606.22942* (2026). URL: <https://arxiv.org/abs/2606.22942>.
 - [90] Xinyin Ma, Gongfan Fang, and Xinchao Wang. “LLM-Pruner: On the Structural Pruning of Large Language Models.” In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023, pp. 21702–21720. URL: <https://openreview.net/forum?id=J8Ajf9WfXP>.
 - [91] Meituan LongCat Team et al. *LongCat-Flash Technical Report*. 2025. arXiv: [2509.01322](https://arxiv.org/abs/2509.01322) [cs.CL]. URL: <https://arxiv.org/abs/2509.01322>.
 - [92] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. “ShortGPT: Layers in Large Language Models are More Redundant Than You Expect.” In: *arXiv preprint arXiv:2403.03853* (2024). URL: <https://arxiv.org/abs/2403.03853>.

- [93] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. *Locating and Editing Factual Associations in GPT*. 2023. arXiv: 2202.05262 [cs.CL]. URL: <https://arxiv.org/abs/2202.05262>.
- [94] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. *Mass-Editing Memory in a Transformer*. 2023. arXiv: 2210.07229 [cs.CL]. URL: <https://arxiv.org/abs/2210.07229>.
- [95] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. *Can a suit of armor conduct electricity? a new dataset for open book question answering*. In EMNLP. 2018.
- [96] Niklas Muennighoff et al. *OLMoE: Open Mixture-of-Experts Language Models*. 2025. arXiv: 2409.02060 [cs.CL]. URL: <https://arxiv.org/abs/2409.02060>.
- [97] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. *Orca: Progressive Learning from Complex Explanation Traces of GPT-4*. 2023. DOI: 10.48550/arXiv.2306.02707. URL: <https://arxiv.org/abs/2306.02707>.
- [98] Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. “Compact Language Models via Pruning and Knowledge Distillation.” In: *arXiv preprint arXiv:2407.14679* (2024). URL: <https://arxiv.org/abs/2407.14679>.
- [99] NLLB Team et al. *No Language Left Behind: Scaling Human-Centered Machine Translation*. 2022. arXiv: 2207.04672 [cs.CL]. URL: <https://arxiv.org/abs/2207.04672>.
- [100] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. *Representation Learning with Contrastive Predictive Coding*. 2019. arXiv: 1807.03748 [cs.LG]. URL: <https://arxiv.org/abs/1807.03748>.
- [101] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. “Librispeech: An ASR corpus based on public domain audio books.” In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2015, pp. 5206–5210. DOI: 10.1109/ICASSP.2015.7178964.
- [102] Oriol Pareras, Gerard I. Gállego, Federico Costa, Cristina España-Bonet, and Javier Hernando. *Revisiting Direct Speech-to-Text Translation with Speech LLMs: Better Scaling than CoT Prompting?* 2025. arXiv: 2510.03093 [cs.CL]. URL: <https://arxiv.org/abs/2510.03093>.
- [103] Ankita Pasad, Ju-Chieh Chou, and Karen Livescu. *Layer-wise analysis of a self-supervised speech representation model*. Preprint, arXiv:2107.04734. 2022.

- [104] Letian Peng, Yuwei Zhang, Zilong Wang, Jayanth Srinivasa, Gaowen Liu, Zihan Wang, and Jingbo Shang. “Answer is All You Need: Instruction-following Text Embedding via Answering the Question.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 459–477. DOI: [10.18653/v1/2024.acl-long.27](https://doi.org/10.18653/v1/2024.acl-long.27). URL: <https://aclanthology.org/2024.acl-long.27/>.
- [105] Qwen Team. *Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters*. <https://qwenlm.github.io/blog/qwen-moe/>. Mar. 2024.
- [106] Qwen Team. *On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability*. 2026. arXiv: [2608.30320](https://arxiv.org/abs/2608.30320) [cs.CL]. URL: <https://arxiv.org/abs/2608.30320>.
- [107] Qwen Team. *Qwen3.8-2.4T-A95B*. <https://huggingface.co/Qwen/Qwen3.8-2.4T-A95B>. 2026.
- [108] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. *Robust Speech Recognition via Large-Scale Weak Supervision*. Preprint, arXiv:2212.04356. 2022.
- [109] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. *Robust speech recognition via large-scale weak supervision*. Preprint, arXiv:2212.04356. 2022.
- [110] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: [2103.00020](https://arxiv.org/abs/2103.00020) [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- [111] Chandan K A Reddy, Vishak Gopal, and Ross Cutler. *DNS-MOS: A Non-Intrusive Perceptual Objective Speech Quality metric to evaluate Noise Suppressors*. 2021. arXiv: [2010.15258](https://arxiv.org/abs/2010.15258) [cs.SD]. URL: <https://arxiv.org/abs/2010.15258>.
- [112] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. “Efficient Content-Based Sparse Attention with Routing Transformers.” In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 53–68. DOI: [10.1162/tacL_a-00353](https://doi.org/10.1162/tacL_a-00353).
- [113] Aurko Roy et al. *N-Grammer: Augmenting Transformers with Latent n-grams*. 2022. arXiv: [2207.06366](https://arxiv.org/abs/2207.06366) [cs.CL]. URL: <https://arxiv.org/abs/2207.06366>.
- [114] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. *Winogrande: An adversarial winograd schema challenge at scale*. arXiv preprint arXiv:1907.10641. 2019.

- [115] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. *Social iqa: Commonsense reasoning about social interactions*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 4463–4473. 2019.
- [116] Fabian David Schmidt, Philipp Borchert, Ivan Vulić, and Goran Glavaš. “Self-Distillation for Model Stacking Unlocks Cross-Lingual NLU in 200+ Languages.” In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 6724–6743. DOI: [10.18653/v1/2024.findings-emnlp.394](https://doi.org/10.18653/v1/2024.findings-emnlp.394). URL: <https://aclanthology.org/2024.findings-emnlp.394/>.
- [117] Fabian David Schmidt, Ivan Vulić, Goran Glavaš, and David Ifeoluwa Adelani. *Fleurs-SLU: A Massively Multilingual Benchmark for Spoken Language Understanding*. 2025. arXiv: [2501.06117](https://arxiv.org/abs/2501.06117) [cs.CL]. URL: <https://arxiv.org/abs/2501.06117>.
- [118] Steffen Schneider, Alexei Baevski, Ronan Collobert, and Michael Auli. *wav2vec: Unsupervised Pre-training for Speech Recognition*. 2019. arXiv: [1904.05862](https://arxiv.org/abs/1904.05862) [cs.CL]. URL: <https://arxiv.org/abs/1904.05862>.
- [119] Claude E. Shannon. “A Mathematical Theory of Communication.” In: *Bell System Technical Journal* 27:3 (1948), pp. 379–423.
- [120] Noam Shazeer. *GLU Variants Improve Transformer*. Preprint, arXiv:2002.05202. 2020.
- [121] Shivalika Singh et al. *Aya dataset: An open-access collection for multilingual instruction tuning*. Preprint, arXiv:2402.06619. 2024.
- [122] Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. *The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs*. 2026. arXiv: [2509.09677](https://arxiv.org/abs/2509.09677) [cs.AI]. URL: <https://arxiv.org/abs/2509.09677>.
- [123] Sharath Turuvekere Sreenivas, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. *Llm pruning and distillation in practice: The minitron approach*. Preprint, arXiv:2408.11796. 2024.
- [124] Sharath Turuvekere Sreenivas et al. “LLM Pruning and Distillation in Practice: The Minitron Approach.” In: *arXiv preprint arXiv:2408.11796* (2024). URL: <https://arxiv.org/abs/2408.11796>.

- [125] StepFun Team. *Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters*. 2026. arXiv: [2602.10604](https://arxiv.org/abs/2602.10604) [cs.CL]. URL: <https://arxiv.org/abs/2602.10604>.
- [126] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. “A Simple and Effective Pruning Approach for Large Language Models.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=PxoFut3dWW>.
- [127] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. *A simple and effective pruning approach for large language models*. Preprint, arXiv:2306.11695. 2024.
- [128] Yaya Sy, Dioula Doucouré, Christophe Cerisara, and Irina Illina. *Speech Language Models for Under-Represented Languages: Insights from Wolof*. 2025. arXiv: [2509.15362](https://arxiv.org/abs/2509.15362) [cs.CL]. URL: <https://arxiv.org/abs/2509.15362>.
- [129] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. “QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference.” In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 47901–47911.
- [130] Yuqing Tang, Chau Tran, Xian Li, Peng-Jen Chen, Naman Goyal, Vishrav Chaudhary, Jiatao Gu, and Angela Fan. “Multilingual Translation with Extensible Multilingual Pretraining and Fine-tuning.” In: (2020). arXiv: [2008.00401](https://arxiv.org/abs/2008.00401) [cs.CL].
- [131] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. *Stanford Alpaca: An Instruction-Following LLaMA Model*. 2023. URL: https://github.com/tatsu-lab/stanford_alpaca.
- [132] Qwen Team. *Qwen2.5: A Party of Foundation Models*. 2024. URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- [133] Omkar Thawakar, Ashmal Vayani, Salman Khan, Hisham Cholakkal, Rao Muhammad Anwer, Michael Felsberg, Timothy Baldwin, Eric P. Xing, and Fahad Shahbaz Khan. *Mobillama: Towards accurate and lightweight fully transparent gpt*. Preprint, arXiv:2402.16840. 2024.
- [134] Atnafu Lambebo Tonja et al. *Inkubalm: A small language model for low-resource african languages*. arXiv preprint arXiv:2408.17024. 2024.
- [135] Lewis Tunstall, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg. *Efficient Few-Shot Learning Without Prompts*. 2022. DOI: [10.48550/ARXIV.2209.11055](https://arxiv.org/abs/2209.11055). URL: <https://arxiv.org/abs/2209.11055>.

- [136] Keyu Wang, Tian Lyu, Guinan Su, Jonas Geiping, Lu Yin, Marco Canini, and Shiwei Liu. *When Fewer Layers Break More Chains: Layer Pruning Harms Test-Time Scaling in LLMs*. 2025. arXiv: 2510.22228 [cs.CL]. URL: <https://arxiv.org/abs/2510.22228>.
- [137] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. *Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts*. 2024. arXiv: 2408.15664 [cs.LG]. URL: <https://arxiv.org/abs/2408.15664>.
- [138] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. *SVD-LLM: Truncation-aware Singular Value Decomposition for Large Language Model Compression*. arXiv preprint arXiv:2403.07378. 2024.
- [139] Ziteng Wang, Jun Zhu, and Jianfei Chen. *ReMoE: Fully Differentiable Mixture-of-Experts with ReLU Routing*. 2025. arXiv: 2412.14711 [cs.LG]. URL: <https://arxiv.org/abs/2412.14711>.
- [140] Jason Wei et al. *Emergent Abilities of Large Language Models*. 2022. arXiv: 2206.07682 [cs.CL]. URL: <https://arxiv.org/abs/2206.07682>.
- [141] Tiansheng Wen, Yifei Wang, Zequn Zeng, Zhong Peng, Yudi Su, Xinyang Liu, Bo Chen, Hongwei Liu, Stefanie Jegelka, and Chenyu You. *Beyond Matryoshka: Revisiting Sparse Coding for Adaptive Representation*. 2025. arXiv: 2503.01776 [cs.LG]. URL: <https://arxiv.org/abs/2503.01776>.
- [142] Jason Weston, Sumit Chopra, and Antoine Bordes. *Memory Networks*. 2015. arXiv: 1410.3916 [cs.AI]. URL: <https://arxiv.org/abs/1410.3916>.
- [143] Thomas Wolf et al. *Huggingface’s transformers: State-of-the-art natural language processing*. Preprint, arXiv:1910.03771. 2020.
- [144] Mitchell Wortsman et al. “Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time.” In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. PMLR, 2022, pp. 23965–23998.
- [145] Jian Wu et al. *On decoder-only architecture for speech-to-text and large language model integration*. 2023. arXiv: 2307.03917 [eess.AS]. URL: <https://arxiv.org/abs/2307.03917>.
- [146] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. *Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning*. arXiv preprint; ArXiv:2310.06694 [cs]. 2023.

- [147] Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. “Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=09i0dae0zp>.
- [148] Zhangchen Xu, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin. “Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned Large Language Models with Nothing.” In: *The Thirteenth International Conference on Learning Representations*. 2025. URL: <https://openreview.net/forum?id=G7sN4c7Sw0>.
- [149] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. “TIES-Merging: Resolving Interference When Merging Models.” In: *Advances in Neural Information Processing Systems*. 2023.
- [150] An Yang et al. “Qwen2 Technical Report.” In: *arXiv preprint arXiv:2407.10671* (2024).
- [151] Yifei Yang, Zouying Cao, and Hai Zhao. “LaCo: Large Language Model Pruning via Layer Collapse.” In: *arXiv preprint arXiv:2402.11187* (2024). URL: <https://arxiv.org/abs/2402.11187>.
- [152] Zhilin Yang, Zihang Dai, Ruslan Salakhutdinov, and William W. Cohen. *Breaking the Softmax Bottleneck: A High-Rank RNN Language Model*. 2018. arXiv: 1711.03953 [cs.CL]. URL: <https://arxiv.org/abs/1711.03953>.
- [153] Lu Yin et al. “Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity.” In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 57101–57115. URL: <https://proceedings.mlr.press/v235/yin24e.html>.
- [154] Hao Yu and Jianxin Wu. *Compressing Transformers: Features Are Low-Rank, but Weights Are Not!* Proceedings of the AAAI Conference on Artificial Intelligence, 37(9):11007–11015. 2023.
- [155] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. “Language Models Are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch.” In: *Proceedings of the 41st International Conference on Machine Learning*. JMLR.org, 2024.
- [156] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. “Scaling Relationship on Learning Mathematical Reasoning with Large Language Models.” In: *arXiv preprint arXiv:2308.01825* (2023). DOI: 10.48550/arXiv.2308.01825. URL: <https://arxiv.org/abs/2308.01825>.

- [157] Marcely Zanon Boito, Vivek Iyer, Nikolaos Lagos, Laurent Besacier, and Ioan Calapodescu. “mHuBERT-147: A Compact Multilingual HuBERT Model.” In: *Interspeech 2024*. 2024, pp. 3939–3943. DOI: [10.21437/Interspeech.2024-938](https://doi.org/10.21437/Interspeech.2024-938).
- [158] Biao Zhang, Lixin Chen, Tong Liu, and Bo Zheng. “SMEC: Rethinking Matryoshka Representation Learning for Retrieval Embedding Compression.” In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Ed. by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 26220–26233. ISBN: 979-8-89176-332-6. DOI: [10.18653/v1/2025.emnlp-main.1332](https://doi.org/10.18653/v1/2025.emnlp-main.1332). URL: <https://aclanthology.org/2025.emnlp-main.1332/>.
- [159] Qizhen Zhang et al. “BAM! Just Like That: Simple and Efficient Parameter Upcycling for Mixture of Experts.” In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 56304–56321. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/68179e00db78ec1c708f6352c0f1724a-Abstract-Conference.html.
- [160] Xueyao Zhang et al. “Amphion: An Open-Source Audio, Music and Speech Generation Toolkit.” In: *Proc. of SLT*. 2024.
- [161] Yingtao Zhang, Haoli Bai, Haokun Lin, Jialin Zhao, Lu Hou, and Carlo Vittorio Cannistraci. “Plug-and-Play: An Efficient Post-training Pruning Method for Large Language Models.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=Tr0lPx9woF>.
- [162] Yuxin Zhang, Lirui Zhao, Mingbao Lin, Yunyun Sun, Yiwu Yao, Xingjia Han, Jared Tanner, Shiwei Liu, and Rongrong Ji. “Dynamic Sparse No Training: Training-Free Fine-Tuning for Sparse LLMs.” In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=1ndDmZdT4g>.
- [163] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. *Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models*. 2026. arXiv: [2601.18734](https://arxiv.org/abs/2601.18734) [cs.LG]. URL: <https://arxiv.org/abs/2601.18734>.
- [164] Haizhong Zheng, Xiaoyan Bai, Xueshen Liu, Z. Morley Mao, Beidi Chen, Fan Lai, and Atul Prakash. “Learn-To-be-Efficient: Build Structured Sparsity in Large Language Models.” In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024, pp. 101969–101991. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/b8f10193cab43d45df9bb810637333fd-Paper-Conference.pdf.

- [165] Gemma Team et al. *Gemma 2: Improving open language models at a practical size*. Preprint, arXiv:2408.00118. 2024.
- [166] Gemma Team et al. *Gemma 3 Technical Report*. Preprint, arXiv:2503.19786. 2025.
- [167] Gemma Team et al. *Gemma 4 Technical Report*. Preprint, arXiv:2607.02770. 2026.

DECLARATION

Put your declaration here.

Saarbrücken, February 2025

André Miede & Ivo Pletikosić

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst's seminal book on typography "*The Elements of Typographic Style*". `classicthesis` is available for both L^AT_EX and L^yX:

<https://bitbucket.org/amiede/classicthesis/>

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

<http://postcards.miede.de/>

Thank you very much for your feedback and contribution.